A COMPLETE PRACTITIONER'S TUTORIAL

Anomaly Detection from Scratch

Fraud Detection and Threat Intelligence Analysis with Machine Learning in Python

Statistical baselines **Isolation Forest** **LOF** **One-Class SVM** **Autoencoders** **Gradient Boosting** **MITRE ATT&CK**

Every code cell in this deck was executed; every output shown is the real printout of that code.

INTRODUCTION

How to read the code slides

Each code slide has three parts: the code, its output, and annotations that explain both.

PYTHON

```
print("hello, anomalies")  # code you can run
```

1

OUTPUT

```
hello, anomalies
```

2

1 Dark panel = Python code

Exactly what was run, in order. Highlighted lines carry a gold number that points to a note on the right.

2 Parchment panel = real output

The genuine printout of that cell. Orange numbers point to the notes that interpret the result.

No equations, ever

Every idea is explained in plain language, pictures and worked results. Where code computes something mathematical, the slide says what it means in words.

INTRODUCTION

What you will be able to do

1 Frame the problem

Explain what an anomaly is, why fraud and cyber-attacks look like anomalies, and why rare events break ordinary machine learning habits.

2 Build the data

Simulate realistic card transactions and security logs with known ground truth, then explore them like an analyst.

3 Engineer signals

Turn raw events into behavioural features: velocity, customer baselines, time context, location and device signals.

4 Detect without labels

Apply eight unsupervised methods, from Isolation Forest to autoencoders, and read their outputs critically.

5 Detect with labels

Train imbalanced classifiers, tune thresholds by cost and alert budget, calibrate and explain them.

6 Hunt threats

Detect password spraying, brute force, beaconing, DGA domains and exfiltration, mapped to MITRE ATT&CK.

INTRODUCTION

The roadmap: sixteen parts

1 Foundations of anomaly detection	**2** The fraud domain	**3** The threat-intelligence domain	**4** Building the fraud dataset
5 Exploratory data analysis	**6** Feature engineering	**7** How to measure a detector	**8** Statistical baselines
9 Unsupervised machine learning	**10** Supervised machine learning	**11** Thresholds, cost and calibration	**12** Leakage and validation
13 Explainability	**14** Threat-intelligence case study	**15** Production and monitoring	**16** Capstone and resources

INTRODUCTION

Who this is for, and what you need

You will get the most from this if you…

- can read basic Python (variables, functions, loops)
- have met pandas DataFrames, even briefly
- are curious about how fraud teams and security operations centres (SOCs) really find bad activity

You do not need…

- any prior machine learning experience: every algorithm is built up from intuition
- any mathematics beyond everyday arithmetic
- access to private data: we generate realistic data ourselves, with ground truth

INTRODUCTION

Two case studies run through the whole tutorial

Case study A: card-payment fraud

62,173 transactions from 2,000 customers over 90 days. Six realities are baked in: everyday spending, travel, night-shift workers, legitimate micro-purchase bursts — and five fraud schemes (card testing, account takeover, foreign card-not-present, ATM cash-out, lost or stolen cards).

Case study B: a security operations centre

27,000 authentication events, 108,000 hourly network summaries, 7,480 connection records and 8,000 domain names. Hidden inside: password spraying, brute force, impossible travel, a port scan, data exfiltration, command-and-control beaconing and algorithmically generated domains.

KEY INSIGHT

Because we create the data, we know exactly which events are malicious. That lets us measure every method honestly — something real-world projects can rarely do this cleanly.

Our toolkit: three libraries, nothing exotic

PYTHON

```
import numpy as np
import pandas as pd
import sklearn
print("numpy  ", np.__version__)
print("pandas ", pd.__version__)
print("sklearn", sklearn.__version__)
```

OUTPUT

```
numpy    2.5.3
pandas   3.0.5
sklearn 1.9.1
```

1 NumPy

Fast arrays of numbers and the random generator we use to simulate data reproducibly.

2 pandas

Tables (DataFrames): grouping, time windows, joins — the analyst's workbench.

3 scikit-learn

Every model in this tutorial: Isolation Forest, LOF, One-Class SVM, mixtures, PCA, neural networks, forests and boosting.

Versions printed

Record versions in every project; model behaviour and defaults change between releases.

INTRODUCTION

A promise: ideas in words, not equations

Anomaly detection is often taught through formulas. Here, every method is explained through three lenses instead:

The intuition

A plain-language picture of what the algorithm is really doing — usually an analogy from everyday life.

The mechanics

A labelled diagram built from text boxes and shapes showing the steps, inputs and outputs.

The evidence

Real code, real output, and annotations that tell you what each number means for a fraud or security team.

PRO TIP

If you ever want the formulas later, the reading list at the end points to the original papers. You will understand them far faster after this tutorial.

PART 1

Foundations of Anomaly Detection

Before touching any algorithm, we build a precise vocabulary and a feel for why rare-event detection is different from everyday prediction.

IN THIS PART

- What an anomaly is (and is not)
- Point, contextual and collective anomalies
- The base-rate problem
- Costs, labels and adversaries
- The detection lifecycle

FOUNDATIONS OF ANOMALY DETECTION

What is an anomaly?

An **anomaly** is an observation that does not fit the pattern formed by the great majority of other observations.

It is relative

"Unusual" only means something against a reference: this customer, this hour, this server, this week.

It is a signal, not a verdict

An anomaly says "look here". Whether it is fraud, an attack, a data error or a legitimately rare event is a separate question.

It is rare by definition

If a pattern became common, it would become part of normal behaviour — so detectors must keep re-learning what normal is.

ANALOGY

Think of a bank teller who has served the same customers for years. They do not memorise rules; they notice when something does not feel like the person in front of them. Anomaly detection teaches software that same instinct.

FOUNDATIONS OF ANOMALY DETECTION

Three families of anomaly

1 Point anomaly

A single event that is unusual on its own. A £4,500 purchase from a customer who never spends more than £60.

2 Contextual anomaly

Normal in one setting, abnormal in another. A £30 grocery shop at 2pm is ordinary; the same card used at 3am, 800km from home, is not.

3 Collective anomaly

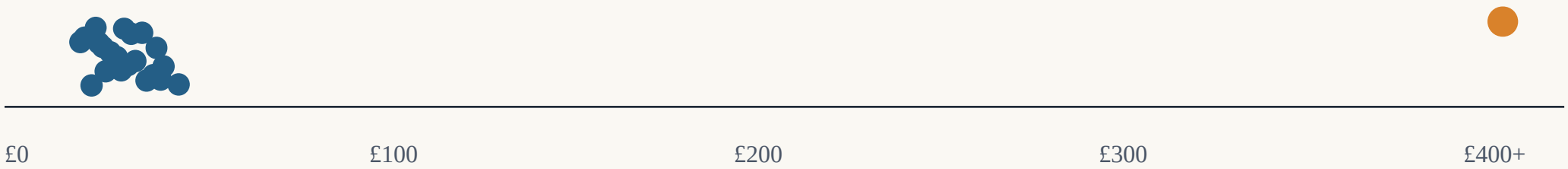
Each event looks fine, but the group does not. Twelve £1 online purchases in ten minutes — classic card testing.

KEY INSIGHT

Fraud and attacks often hide as contextual or collective anomalies precisely because criminals try to make each single event look normal.

Picture it: a point anomaly

One customer's last 23 card payments, plotted by amount (£) along a line.



WHAT THE OUTPUT SHOWS

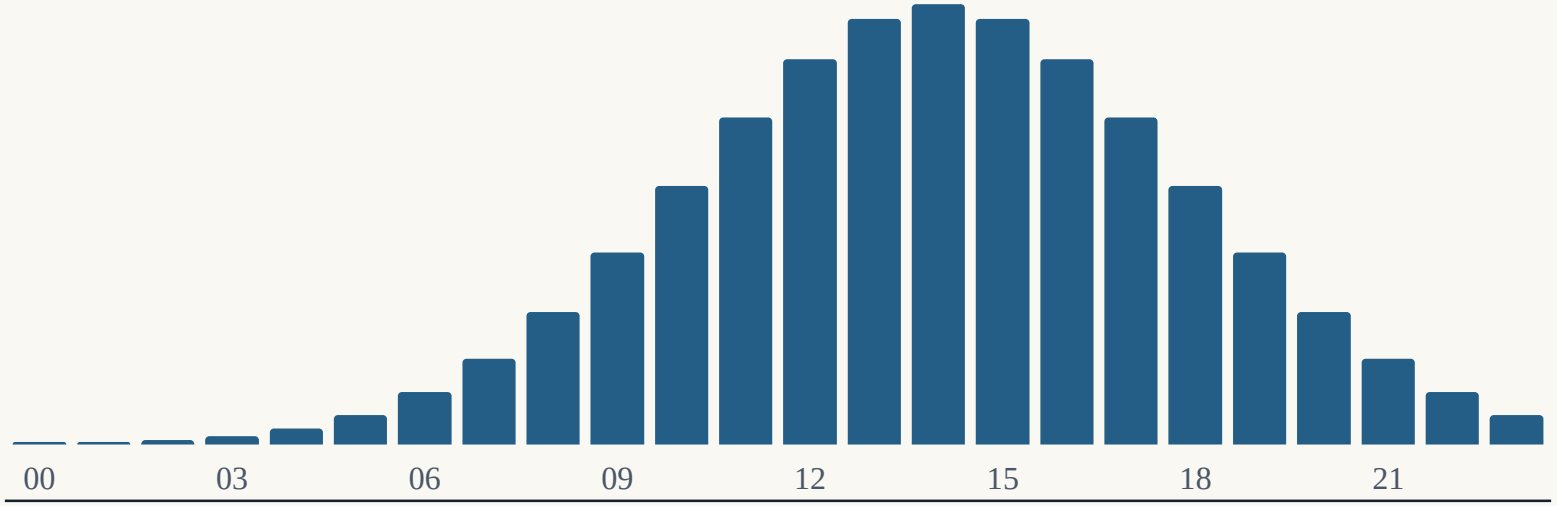
22 payments sit together between roughly £18 and £45. One payment of £410 sits far away on its own. Any reasonable method — a person, a rule, or a model — will flag it.

WATCH OUT

Being far away does not make it fraud. Perhaps it is a new sofa. The job of a detector is to rank this payment for attention, not to convict it.

Picture it: a contextual anomaly

Relative frequency of card activity by hour of day for a typical day-time customer.



KEY INSIGHT

The same £30 payment is ordinary at 14:00, where most of this customer's activity happens.

WHAT THE OUTPUT SHOWS

At 03:00 the bar is almost invisible: activity is rare, so a payment then is surprising *for this person*.

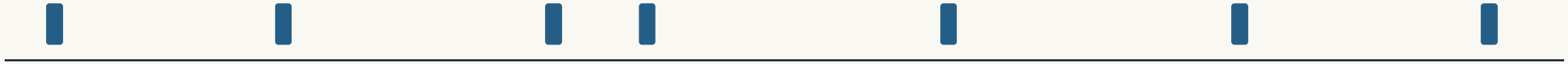
PRO TIP

Context features (hour, place, device, merchant) are what turn plain values into meaningful signals.

Picture it: a collective anomaly

Two days of payments on one card. Each mark is a payment of under £5.

Day 1 — a normal day: seven scattered small payments



Day 2 — eleven £1–£9 online payments within about 25 minutes



KEY INSIGHT

No single £2 payment is strange. The *cluster* is: a fraudster checking which stolen card numbers still work before a big purchase (card testing).

PRO TIP

Collective anomalies need features computed over windows of time — counts per hour, gaps between payments — which we build in Part 6.

Outliers, novelties and noise

Outlier detection

The training data itself is contaminated with a few anomalies. The method must find them without being told which ones they are.

Novelty detection

The training data is clean (only normal behaviour). New data is compared against it. Useful when you can certify a clean period.

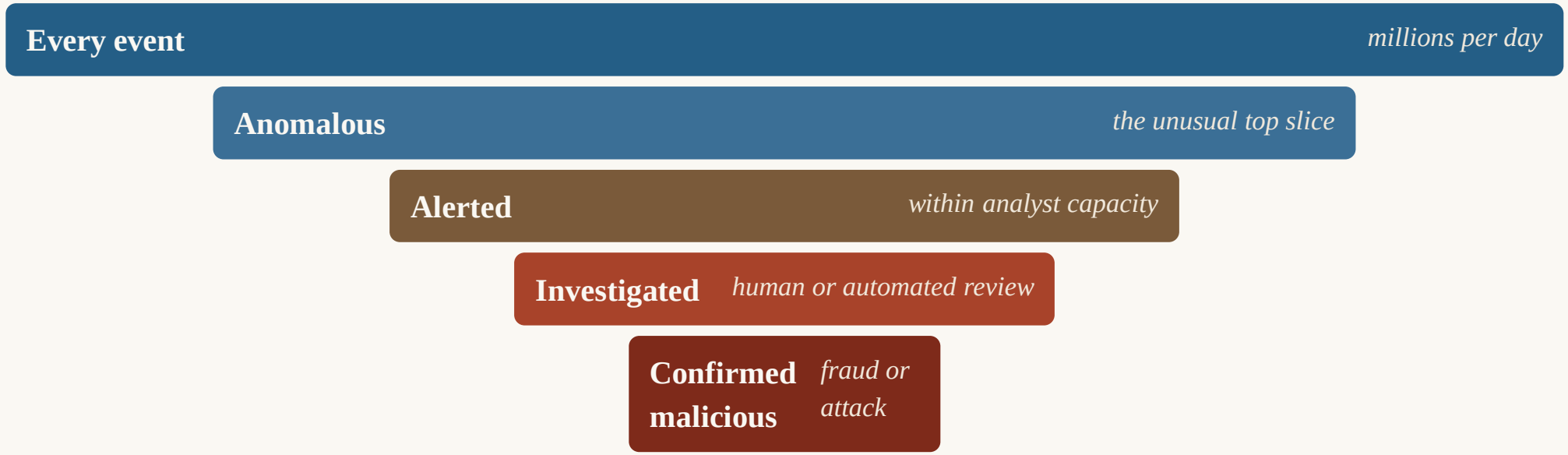
Noise

Random measurement error or messy data entry. It looks anomalous but carries no meaning. Good data cleaning removes much of it.

Training data contains anomalies?	Yes, a few	Assumed not
Typical scikit-learn setting	IsolationForest(), LOF()	LOF(novelty=True), One-Class SVM
Fraud/security example	Score last month’s transactions	Learn a server’s clean baseline, watch new traffic

Anomalous is not the same as malicious

The detection funnel: each stage narrows the stream.



KEY INSIGHT

A model’s job is to make the second and third layers as *rich in genuine threats* as possible, so scarce analyst time is spent wisely.

The base-rate problem

1 in 70

In our fraud data, about one transaction in seventy is fraudulent — and real portfolios are often far rarer than that (one in a thousand or less).

98.7%

The accuracy of a useless model that says ‘legitimate’ to everything on our test data. It catches zero fraud.

0

Frauds caught by that ‘accurate’ model. Accuracy is the wrong lens for rare events.

WATCH OUT

When positives are rare, even a very good detector produces many false alarms in absolute terms. Precision, recall and ‘alerts per day’ are the honest measures (Part 7).

Errors are not equally expensive

A missed fraud (false negative)

- Direct monetary loss
- Chargeback fees and scheme penalties
- Customer distress and lost trust
- In security: a breach that grows silently

A false alarm (false positive)

- Analyst time spent reviewing
- A declined card at a till: embarrassment and abandoned purchases
- Alert fatigue: real alerts get ignored
- In security: wasted incident response

KEY INSIGHT

Because the two errors cost different amounts, the ‘best’ threshold is a business decision. In Part 11 we choose it by minimising total cost — with real numbers.

How much do we know? The label spectrum

Unsupervised
no labels at all

Find whatever is unusual. Catches new attack types, but cannot tell ‘rare’ from ‘bad’.

Semi-supervised
clean normal data, or a few labels

Learn normal well, flag departures. Common in security baselining.

Supervised
many confirmed labels

Learn what bad looks like. Very accurate on known schemes, blind to brand-new ones.

KEY INSIGHT

Mature fraud and security programmes run all three side by side: supervised models for known patterns, unsupervised models as a safety net for the unknown.

Labels arrive late, and imperfectly

A typical card-fraud label timeline.

Day 0

Fraudulent payment authorised

Day 3–30

Customer notices statement

Day 30–60

Dispute / chargeback filed

Day 60–120

Investigation closes; label confirmed

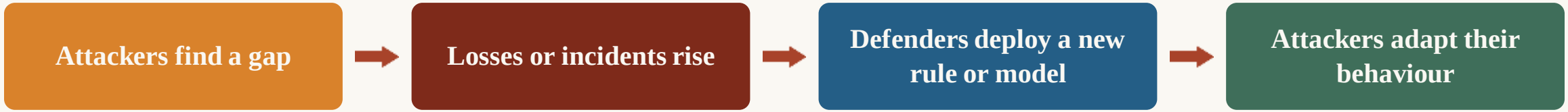
WATCH OUT

The newest data — the most relevant for spotting new schemes — is the data you have the *fewest* confirmed labels for.

KEY INSIGHT

Some fraud is never reported, and some disputes are ‘friendly fraud’ by genuine customers. Treat labels as noisy evidence, not perfect truth.

An adversarial game, not a static one



KEY INSIGHT

Unlike predicting the weather, the thing you are predicting *fights back*. In our data, card-testers switch from online retail to gambling merchants in the final month.

PRO TIP

Design for change: monitor drift, retrain on schedule, and keep an unsupervised layer that does not depend on yesterday's labels.

Concept drift: when ‘normal’ moves

Sudden drift

A step change: a new payment app launches, a pandemic shifts spending online overnight.

Gradual drift

Slow change: inflation lifts typical basket sizes by a few percent per quarter.

Recurring drift

Seasonal cycles: Black Friday, holidays, month-end payroll, quarter-end batch jobs on servers.

Adversarial drift

Attackers deliberately change tactics after detection — the hardest kind.

PRO TIP

Part 15 measures drift with the Population Stability Index and shows how a simulated price shock is caught.

Scores, thresholds and alerts



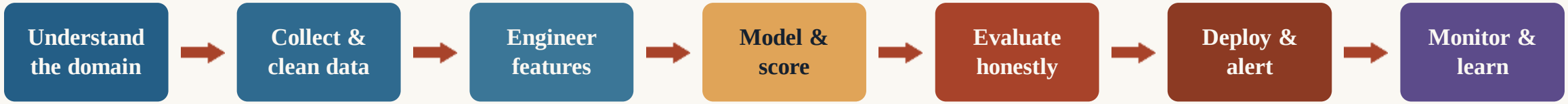
Why scores, not yes/no?

A score lets you rank events from most to least suspicious, so you can review the top of the list first and move the cut-off as capacity changes.

Who sets the threshold?

The business, using cost and capacity — not the algorithm. Most library defaults (like a ‘contamination’ setting) are just placeholders.

The anomaly-detection lifecycle



KEY INSIGHT

Most of the value is created *before* modelling: good features routinely beat clever algorithms. This tutorial spends as long on data and features as on models.

The parts of this tutorial follow this cycle from left to right.

Rules, statistics or machine learning?

Expert rules	Transparent, fast, encode policy	Brittle; easy for attackers to learn and avoid	Hard limits, regulatory checks
Statistical baselines	Simple, explainable per entity	One variable at a time; break on skewed data	Per-customer or per-host thresholds
Unsupervised ML	Finds unknown patterns, no labels	Flags ‘rare’ not ‘bad’; harder to tune	Novel schemes, new attacks, triage
Supervised ML	Most accurate on known patterns	Needs labels; blind to the brand new	Mature fraud models, phishing, malware

KEY INSIGHT

These are layers, not rivals. Strong programmes combine them, and use each to check the others.

Humans in the loop



Analysts supply labels

Every closed alert becomes a training example — but only for what was alerted. Unseen fraud stays unlabelled.

Explanations save time

A score of 0.97 tells an analyst little. ‘Spend is 11× this customer’s usual on a new device’ tells them where to look (Part 13).

Feedback can bias

If you only ever review what the model flags, the model only learns what it already knows. Sample some random cases too.

Vocabulary for the rest of the tutorial

True positive (TP)	An alert on a genuinely malicious event — a catch
False positive (FP)	An alert on an innocent event — a false alarm
False negative (FN)	A malicious event the detector missed
Precision	Of everything we alerted, what share was truly bad
Recall	Of everything truly bad, what share did we alert on
Anomaly score	A number where higher means ‘stranger’; used for ranking
Threshold	The score above which an event becomes an alert
Contamination	The assumed share of anomalies in the data (a model setting)

Check your understanding

Q1 **A £30 grocery payment at 3am, 800km from home. Which family of anomaly?**

Answer: Contextual: the value is ordinary, the context (time, place) is not.

Q2 **Why is 98.7% accuracy meaningless on our data?**

Answer: Because labelling everything ‘legitimate’ already scores 98.7% while catching no fraud at all.

Q3 **Name one reason labels are unreliable in fraud.**

Answer: Delay (chargebacks take weeks), under-reporting, or friendly fraud by genuine customers.

Q4 **Who should set the alert threshold?**

Answer: The business, from costs and analyst capacity — not a library default.

Key takeaways

01 An anomaly is unusual relative to a reference; it is a lead, not a verdict.

02 Point, contextual and collective anomalies need different features.

03 Rare positives make accuracy useless; think in precision, recall and alerts.

04 Errors have unequal costs, so thresholds are business choices.

05 Labels are late, noisy and incomplete; attackers adapt.

06 Rules, statistics and ML are complementary layers.

PART 2

The Fraud Domain

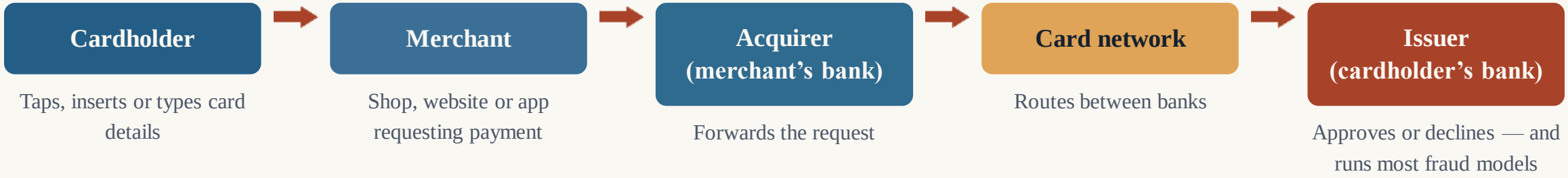
You cannot detect what you do not understand. This part maps the payment ecosystem and the behaviour of each fraud scheme we will later detect.

IN THIS PART

- How a card payment flows
- Eight fraud typologies
- The signals each scheme leaves
- Real-time versus batch detection
- False declines and customer experience

THE FRAUD DOMAIN

Who is involved in a card payment?



KEY INSIGHT

The issuer sees the cardholder's full history across every merchant, which is why customer-level behaviour features are so powerful for issuers.

THE FRAUD DOMAIN

A real-time decision in a fraction of a second

Approximate authorisation timeline for a card payment (illustrative).

0 ms	~50 ms	~100 ms	~150 ms	~300 ms
Card presented	Request reaches issuer	Fraud model scores the payment	Approve, decline or step-up check	Answer back at the till

PRO TIP

Real-time models must be *fast*: features have to be pre-computed or cheap (counts in the last hour, the customer's usual spend).

KEY INSIGHT

Slower, heavier models can still run after authorisation to catch fraud for review, card blocking or account recovery.

THE FRAUD DOMAIN

A map of fraud typologies

Card-not-present (CNP)

Stolen card details used online or by phone.

Card testing

Many tiny payments to check which stolen numbers work.

Account takeover (ATO)

Criminal logs in as the customer and spends.

Lost or stolen card

Physical card used in shops near the victim.

ATM cash-out

Cloned or stolen cards drained at cash machines.

Synthetic identity

A fake person built from real and invented details.

First-party fraud

The genuine customer lies: false disputes, bust-out.

Push-payment scams

Victim is tricked into sending money; mules move it on.

Our simulated data contains the first five; the last three need data beyond card transactions (identity, disputes, account transfers).

THE FRAUD DOMAIN

Scheme 1 — Card testing

How it works

Criminals buy lists of stolen card numbers. Before using them, they run tiny payments through a weak merchant to see which cards are still live.

How it appears in our data

316 payments in bursts of 4–11 within 40 minutes. In the final month, 60% switch to gambling merchants — an adaptation the models must survive.

SIGNALS IT LEAVES

- Bursts: many payments minutes apart
- Very small amounts (£0.50–£10)
- Online merchants, often a new device
- Followed later by a large purchase

THE FRAUD DOMAIN

Scheme 2 — Account takeover (ATO)

How it works

A criminal obtains login credentials (phishing, malware, password reuse), enters the account and spends — often on resaleable goods.

How it appears in our data

150 payments at 2–10× the customer's typical spend; 55% from a new device; electronics, online retail or gambling at varied hours.

SIGNALS IT LEAVES

- Spend far above the customer's usual level
- New device or changed contact details
- Unusual hours
- Electronics, gift cards, gambling

THE FRAUD DOMAIN

Scheme 3 — Foreign card-not-present

How it works

Card details stolen in a data breach are used on websites abroad, often for travel or goods that can be resold.

How it appears in our data

160 payments 150–9,000 km away; 80% flagged foreign; travel or online retail; moderately large amounts.

SIGNALS IT LEAVES

- Foreign merchant country
- Large distance from the cardholder’s home
- Travel or online retail
- Amounts unrelated to normal spend

THE FRAUD DOMAIN

Scheme 4 — ATM cash-out

How it works

Cloned or stolen cards are taken to cash machines, usually at night and away from the victim’s home, and drained in round amounts.

How it appears in our data

90 withdrawals of £100–£500 between 01:00 and 04:59, 20–300 km away. Legitimate ATM use is also round, so round amounts alone mislead.

SIGNALS IT LEAVES

- Cash withdrawals at night
- Round amounts (£100, £200…)
- Moderate distance from home
- Several cards at one ATM

THE FRAUD DOMAIN

Scheme 5 — Lost or stolen card

How it works

A thief uses a physically stolen card in local shops before the victim notices — near home, in-person, in everyday categories.

How it appears in our data

164 payments at grocery, fuel, restaurants or electronics near home, at 1.2–4× usual spend. Designed to be nearly invisible — watch how every model struggles with it.

SIGNALS IT LEAVES

- Local, card-present, everyday merchants
- Amounts only somewhat above normal
- A short spree over a few hours
- Very hard to separate from the real owner

THE FRAUD DOMAIN

Beyond our dataset: three schemes needing other data

Synthetic identity

A fabricated person (real ID number, invented name) builds credit slowly, then ‘busts out’. Detected with identity-graph and application data: shared phones, addresses, devices across applicants.

First-party fraud

The real customer disputes genuine purchases or maxes out credit with no intent to repay. Needs dispute history and repayment behaviour.

Push-payment scams and mules

Victims are manipulated into sending money; mule accounts receive and forward it fast. Needs account-to-account transfer graphs: fan-in, fan-out, rapid pass-through.

KEY INSIGHT

The same toolkit applies — anomaly scores, behavioural baselines, supervised models — but on different tables: applications, disputes, transfer networks.

THE FRAUD DOMAIN

Which signals reveal which scheme?

Amount vs customer's usual	much lower	much higher	higher	higher	a bit higher
Payments in last hour	very high	normal	normal	low–normal	slightly high
Night-time	sometimes	sometimes	sometimes	always	rarely
Distance from home	varied	low–medium	very high	medium	low
New device	often	often	sometimes	never	never
Merchant category	online / gambling	electronics etc.	travel / online	ATM	everyday

KEY INSIGHT

No single feature catches everything. That is why we need multi-variable models — and why 'lost/stolen', which barely moves any signal, stays hard.

The customer is their own baseline

The most powerful idea in fraud analytics: judge each payment against **that customer's** history, not against everyone.

Global view

£250 is unremarkable across all customers — plenty of people spend that.

Personal view

For a customer whose typical payment is £12, £250 is twenty times their norm: highly informative.

How we build it

Per-customer medians and typical spreads, computed only from *past* data, then ratios of each new payment against them (Part 6).

Where detection happens

Pre-authorisation (real time)

- Decides in ~100 ms whether to approve
- Simple, pre-computed features
- Wrong declines hurt customers instantly
- Typical: gradient-boosted trees + rules

Post-authorisation (batch / near real time)

- Reviews payments minutes to days later
- Heavier features: networks, sequences
- Feeds case queues, card blocks, recovery
- Typical: anomaly detectors, graph analytics

THE FRAUD DOMAIN

The hidden cost: false declines

For the customer

A declined card at checkout is embarrassing and frustrating; many customers switch to another card and may not come back.

For the merchant

Lost sales and abandoned baskets. Industry studies often estimate false-decline losses can rival direct fraud losses.

For the model designer

Optimise for the balance, not for fraud caught alone. Step-up checks (a one-time code) are a softer alternative to a hard decline.

PRO TIP

Strong customer authentication rules (such as the UK and EU requirements for two-factor checks on many online payments) shift where and how fraud happens — another reason to monitor drift.

Check your understanding

Q1 **Which party usually runs the main fraud model on a card payment, and why?**

Answer: The issuer: it sees the cardholder's full history across all merchants.

Q2 **What behaviour signature identifies card testing?**

Answer: Bursts of very small online payments, minutes apart, often from a new device.

Q3 **Why are round ATM amounts a weak signal on their own?**

Answer: Legitimate cash withdrawals are also round (£20, £50, £100), so roundness alone causes false alarms.

Q4 **Which of our five schemes is designed to be hardest?**

Answer: Lost/stolen: local, card-present, everyday merchants, only slightly higher amounts.

Key takeaways

01 The issuer sees the full customer history: personal baselines are the core signal.

02 Each fraud scheme leaves a different fingerprint across amount, time, velocity, location and device.

03 Some fraud barely changes any signal — expect every model to miss some of it.

04 Real-time decisions need fast features; batch review can afford heavier analysis.

05 False declines are a real cost; optimise the balance, not fraud caught alone.

06 Attackers adapt — our card-testers change merchants in the final month.

PART 3

The Threat-Intelligence Domain

Security operations face the same rare-event problem as fraud teams — with machines, accounts and networks instead of cards.

IN THIS PART

- What cyber threat intelligence is
- IOCs, TTPs and the Pyramid of Pain
- MITRE ATT&CK
- Security data sources
- Spraying, brute force, beaconing, DGA, exfiltration

THE THREAT-INTELLIGENCE DOMAIN

What is cyber threat intelligence (CTI)?

Threat intelligence is evidence-based knowledge about adversaries — who they are, how they operate and what they leave behind — used to make better defensive decisions.

Strategic

For leaders: which threat groups target our sector, and what is the business risk?

Operational

For defenders: details of an active campaign — timing, targets, infrastructure.

Tactical

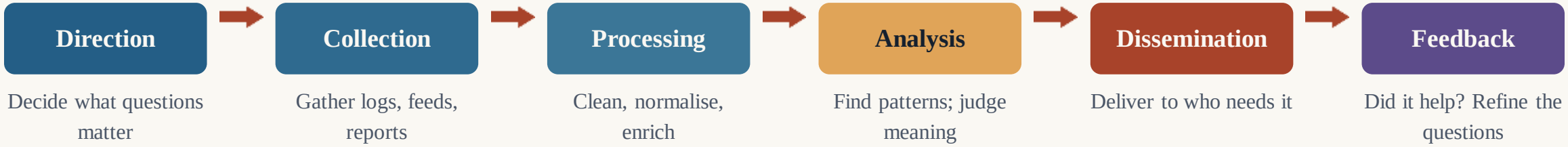
For tools and analysts: specific indicators and behaviours to detect and block.

KEY INSIGHT

Anomaly detection turns raw telemetry into *tactical intelligence*: it surfaces behaviours worth investigating, which analysts then enrich and share.

THE THREAT-INTELLIGENCE DOMAIN

The intelligence lifecycle



KEY INSIGHT

Machine learning lives mostly in *processing* and *analysis*: it reduces millions of events to a short, ranked list a human can reason about.

Indicators versus behaviours

Indicators of compromise (IOCs)

- Concrete artefacts: IP addresses, domains, file hashes
- Easy to share in feeds and block lists
- Easy for attackers to change in minutes
- Detected by exact matching

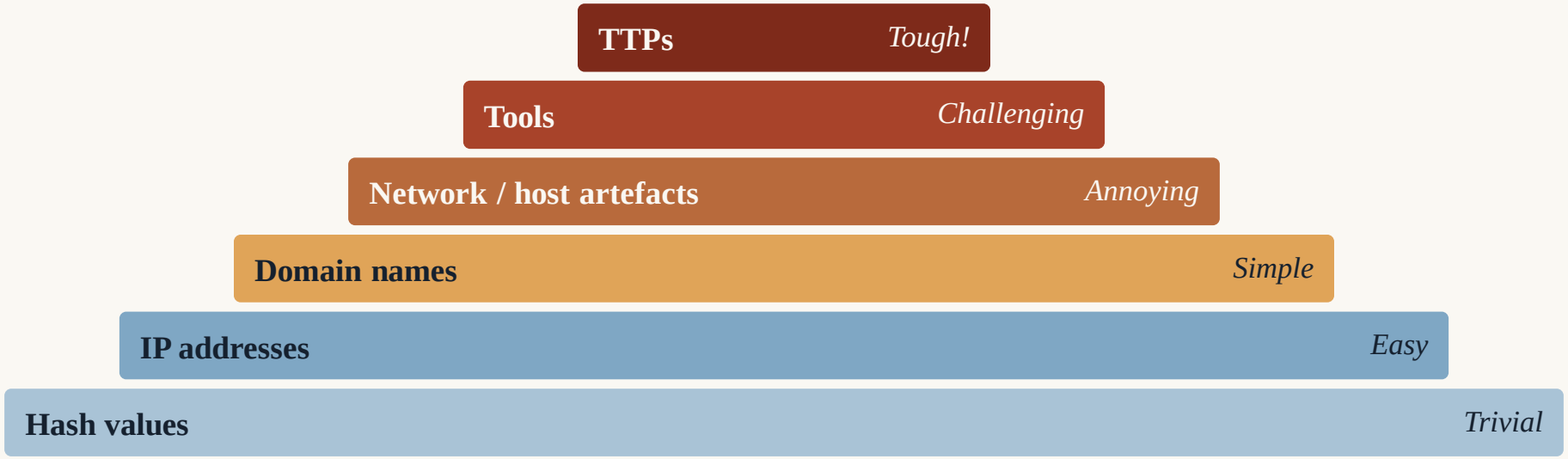
Tactics, techniques and procedures (TTPs)

- How the attacker operates: e.g. spraying one password across many accounts
- Hard and costly for attackers to change
- Cannot be matched exactly — they must be *recognised*
- Detected by behavioural analytics and anomaly detection

THE THREAT-INTELLIGENCE DOMAIN

The Pyramid of Pain

How much pain you cause an attacker when you detect each kind of evidence (after David J. Bianco's model).



KEY INSIGHT

Behavioural anomaly detection works at the top of the pyramid — where detection hurts attackers most.

THE THREAT-INTELLIGENCE DOMAIN

MITRE ATT&CK: a shared language for adversary behaviour

Tactic
the attacker's goal

→

Technique
how they achieve it

→

Sub-technique
a specific variant

→

Procedure
what one group actually did

Example chain

Credential Access (tactic) → Brute Force (T1110) → Password Spraying (T1110.003) → 'one common password tried once against 260 accounts in two hours' (our data).

Why it matters for ML

Mapping each detector to ATT&CK shows which behaviours you cover — and the gaps you do not. Part 14 maps every detector we build.

THE THREAT-INTELLIGENCE DOMAIN

The attack story hidden in our SOC data

Five ATT&CK tactics, in roughly the order an intrusion unfolds.

Initial Access	Credential Access	Discovery	Command & Control	Exfiltration
valid stolen account	spraying, brute force	port scanning	beaconing, DGA domains	bulk data out at night

KEY INSIGHT

Each stage leaves a *different* kind of anomaly, in a different data source. No single model sees the whole story — correlation across detectors does.

Security data sources

Authentication logs	user, time, source IP, country, success/failure	brute force, spraying, impossible travel
Network flow (NetFlow)	who talked to whom, bytes, ports, duration	scanning, exfiltration, beaconing
DNS logs	which domain names each host looked up	DGA domains, tunnelling, newly seen domains
Endpoint (EDR)	processes, files, registry, command lines	malware, living-off-the-land tools
Web proxy / email	URLs visited, attachments, senders	phishing, drive-by downloads
Cloud audit logs	API calls, role changes, storage access	privilege abuse, data theft

THE THREAT-INTELLIGENCE DOMAIN

UEBA: every user and host is its own baseline

User and Entity Behaviour Analytics applies the fraud idea — ‘compare each customer with themselves’ — to accounts, machines and services.

Baselines per entity

What does host-077 normally send out at 2am? What hours does user042 normally log in?

Peer groups

Compare a finance laptop with other finance laptops, not with database servers.

Deviation scores

How far is this hour from the entity’s own norm? We build exactly this in Part 14.

THE THREAT-INTELLIGENCE DOMAIN

Why anomaly detection matters in a SOC

1000s

of alerts per day reach a typical security team from signatures and rules

Most

are false positives or low value — the root cause of ‘alert fatigue’

Unknown

attacks have no signature yet; only behaviour can reveal them

KEY INSIGHT

The goal is not more alerts. It is *better-ranked* alerts, with context, so analysts look at the right five things first.

The detection-engineering lifecycle



PRO TIP

Treat detectors like code: version them, test them against known-bad and known-good data, and record which ATT&CK technique each one covers.

THE THREAT-INTELLIGENCE DOMAIN

Password spraying versus brute force

Brute force (T1110.001)

Many passwords, one account. An attacker hammers user042 with 450 guesses in under an hour until one works.

WHAT THE OUTPUT SHOWS

Signature: huge failure count for one user from one source.

Password spraying (T1110.003)

One password, many accounts. An attacker tries ‘Summer2026!’ once against 260 accounts — staying under lock-out limits.

WHAT THE OUTPUT SHOWS

Signature: one source failing against an unusually large number of *different* users.

THE THREAT-INTELLIGENCE DOMAIN

Command-and-control beaconing

Malware 'phones home' to its controller on a timer — and timers are unnaturally regular.

Human browsing to a website: irregular, bursty gaps



Malware beacon: a connection every ~5 minutes with tiny jitter



KEY INSIGHT

We will measure *regularity* — how much the gaps vary relative to their average. Human traffic varies a lot; beacons barely at all.

THE THREAT-INTELLIGENCE DOMAIN

Domain generation algorithms (DGA)

The attacker’s trick

Malware generates thousands of random-looking domain names each day and tries them in turn. The attacker registers just one, so blocking any single domain achieves little.

Look at the names

Legitimate: greencloudshop.io · paybooknews.com

Generated: x7k2q9zr4mfp1.xyz · qwhzkrtpvbn.info

KEY INSIGHT

Human-chosen names are pronounceable and built from words. Generated names have more digits, fewer vowels, long consonant runs and higher randomness.

PRO TIP

Those properties become features for a classifier — and also for an anomaly detector trained only on legitimate names (Part 14).

Data exfiltration

Exfiltration (ATT&CK tactic TA0010) is the theft of data out of the organisation — often the final, most damaging step.

Volume

Gigabytes leaving a host that normally sends megabytes.

Timing

Large transfers in the small hours, when nobody is watching.

Destination

A single, previously unseen external address.

KEY INSIGHT

A volume threshold across *all* hosts fails: a backup server legitimately sends gigabytes nightly. A per-host, per-hour baseline catches the laptop that suddenly behaves like a backup server.

Check your understanding

Q1 **Why do TTP-based detections cause attackers more pain than IP block lists?**

Answer: Changing an IP takes minutes; changing how you operate is slow and costly.

Q2 **Spraying or brute force: one source, 260 users, one failure each?**

Answer: Password spraying (T1110.003).

Q3 **What makes a C2 beacon stand out in connection logs?**

Answer: Its regularity: near-identical gaps between connections.

Q4 **Why do global volume thresholds fail for exfiltration?**

Answer: Different hosts have very different normal volumes; you need per-host, per-hour baselines.

Key takeaways

01 Threat intelligence turns telemetry into decisions; ML does the heavy filtering.

02 Behaviours (TTPs) are harder for attackers to change than indicators (IOCs).

03 MITRE ATT&CK gives every detector a shared name and exposes coverage gaps.

04 Different attack stages appear in different logs: auth, network, DNS.

05 Per-entity baselines (UEBA) beat global thresholds.

06 Regularity, randomness and volume-in-context are the key behavioural signals.

PART 4

Building the Fraud Dataset

We build 62,173 card transactions with complete ground truth. Every line of the simulation is shown, because understanding the data-generating process is the best training an analyst can get.

IN THIS PART

- Why simulate data
- Customers and everyday spending
- Realistic noise: travel, night owls, bursts
- Five fraud schemes, one function each
- Assembling one timeline

Why build synthetic data?

Ground truth

We know exactly which rows are fraud and which scheme produced them, so every metric is exact.

Privacy

Real card data is protected by law and payment-industry rules (PCI DSS). Synthetic data can be shared and taught freely.

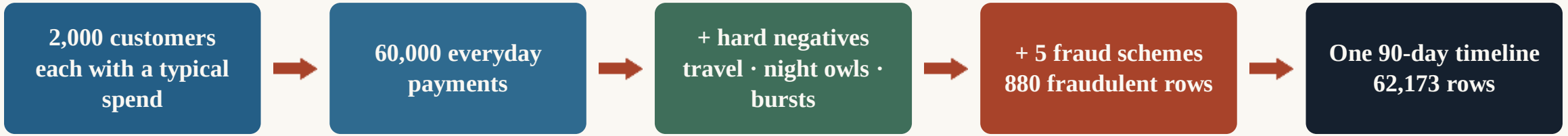
Control

We can add hard cases on purpose — legitimate night spending, legitimate bursts — and watch which models cope.

WATCH OUT

The trade-off: synthetic data is simpler than reality. Treat the *relative* behaviour of methods as the lesson, not the exact scores. Real projects should validate on real, labelled history.

The data blueprint



Hard negatives

Legitimate behaviour that *looks* suspicious. Without them, any model looks brilliant — which teaches nothing.

Time matters

Every row has a timestamp. The first 60 days train the models; the last 30 test them, as in a live deployment.

Attackers adapt

Card-testers change merchants in the final month, so the test period contains a pattern the training period never showed.

Step 1 — create the customers

PYTHON

```python
rng = np.random.default_rng(42)                                  # 1
N_CUST, N_TX, DAYS = 2_000, 60_000, 90

cust = pd.DataFrame({
    "customer_id": np.arange(N_CUST),
    "typical_spend": rng.lognormal(3.2, 0.5, N_CUST),            # 2
    "fav_cat": rng.integers(0, 6, N_CUST),
})
print(cust.head())
print("shape:", cust.shape)
```

OUTPUT

	customer_id	typical_spend	fav_cat
0	0	28.570	4
1	1	14.585	0
2	2	35.703	2
3	3	39.263	2
4	4	9.249	5

shape: (2000, 3)

1 A seeded random generator

Seed 42 makes every run identical, so your numbers will match these slides exactly.

2 Each customer has a typical spend

Drawn from a right-skewed (log-normal) distribution: most people spend modestly, a few spend a lot.

3 The result

Customer 4 typically spends about £9 per payment; customer 3 about £39. These personal levels drive everything that follows.

Step 2 — merchants and the daily rhythm

PYTHON

```
CATS = ["grocery", "fuel", "restaurant", "online_retail",
        "electronics", "travel", "atm", "gambling"]
CAT_P = [0.30, 0.14, 0.18, 0.20, 0.06, 0.04, 0.06, 0.02]

hours = np.arange(24)
hour_w = np.exp(-((hours - 14) ** 2) / 30)   # day-time peak
hour_p = hour_w / hour_w.sum()
print("Busiest hour:", hours[hour_p.argmax()])
print("P(2am) = %.4f    P(2pm) = %.4f" % (hour_p[2], hour_p[14]))
```

OUTPUT

```
Busiest hour: 14
P(2am) = 0.0009          P(2pm) = 0.1037
```

1 Eight merchant categories

With realistic shares: grocery is most common (30%), gambling rare (2%).

2 A bell-shaped day

Hour weights peak at 14:00 and fade towards night. This is the curve you saw on the ‘contextual anomaly’ slide.

3 Night is rare

A legitimate day-time customer has about a 1-in-1,000 chance of paying at 2am, versus about 10% at 2pm.

Step 3 — 60,000 everyday payments

PYTHON

```
cid = rng.integers(0, N_CUST, N_TX)
minute = (rng.integers(0, DAYS, N_TX) * 1440                                  1
          + rng.choice(24, N_TX, p=hour_p) * 60
          + rng.integers(0, 60, N_TX))
normal = pd.DataFrame({
    "customer_id": cid,
    "minute": minute,
    "amount": (cust.typical_spend.values[cid]                                 2
               * rng.lognormal(0, 0.6, N_TX)).round(2),
    "merchant_cat": rng.choice(CATS, N_TX, p=CAT_P),
    "distance_km": rng.exponential(8, N_TX).round(1),
    "new_device": rng.binomial(1, 0.05, N_TX),                                3
    "foreign": rng.binomial(1, 0.01, N_TX),
    "is_fraud": 0, "scheme": "legit"})
print(normal.shape)
```

OUTPUT

```
(60000, 9)                                                                    4
```

1 When

A random day, an hour drawn from the daily rhythm, and a random minute — stored as minutes since 1 January.

2 How much

The customer's typical spend multiplied by a random factor, so each person's payments vary around their own level.

3 Rare flags

5% of everyday payments come from a new device and 1% are foreign — legitimate noise models must tolerate.

4 60,000 rows, 9 columns

Every row is labelled is_fraud = 0 and scheme = 'legit'.

Step 4 — genuine travellers and real ATM habits

PYTHON

```
# Genuine customers sometimes travel: realistic noise
trav = rng.choice(N_TX, 900, replace=False)                    (1)
normal.loc[trav, "distance_km"] = rng.uniform(150, 3000, 900).round(1)
normal.loc[trav, "foreign"] = rng.binomial(1, 0.5, 900)
normal.loc[trav, "merchant_cat"] = rng.choice(
    ["travel", "restaurant", "fuel"], 900)
atm = (normal.merchant_cat == "atm").values                    (2)
normal.loc[atm, "amount"] = rng.choice([20., 50., 100., 200.],
                                       atm.sum())
print("Legit travel rows:", len(trav))
print("Legit foreign rows:", int(normal.foreign.sum()))
```

OUTPUT

```
Legit travel rows: 900
Legit foreign rows: 1015                                       (3)
```

1 900 legitimate trips

Payments far from home (150–3,000 km), half of them abroad, at travel, restaurants and fuel. These look like foreign fraud — on purpose.

2 Real cash habits

Legitimate ATM withdrawals become round (£20–£200). Now ‘round amount’ is no longer a giveaway for cash-out fraud.

3 1,015 genuine foreign payments

More than the foreign *fraud* we will add — so ‘foreign’ alone will raise many false alarms.

Step 5 — night-shift workers

PYTHON

```
# 8% of customers are night-shift workers: legit night spend
owls = rng.choice(N_CUST, 160, replace=False)                                  1
m = normal.customer_id.isin(owls).values & (rng.random(N_TX) < .6)
night_h = rng.choice([22, 23, 0, 1, 2, 3, 4], m.sum())                         2
normal.loc[m, "minute"] = ((normal.minute[m] // 1440) * 1440
                           + night_h * 60 + rng.integers(0, 60, m.sum()))
print("night-owl customers:", len(owls))
print("legit txns moved to night:", int(m.sum()))
```

OUTPUT

```
night-owl customers: 160
legit txns moved to night: 2931                                                3
```

1 160 night owls

8% of customers are given night-time habits: nurses, security staff, delivery drivers.

2 Moved to 22:00–04:59

60% of their payments are re-timed into the night, keeping the same day.

3 2,931 legitimate night payments

Night-time is now a *signal*, not proof. Detectors must combine it with other evidence.

Step 6 — legitimate bursts of tiny payments

PYTHON

```
# Legit bursts: app-store / in-game micro-purchases
rows = []
for v in rng.choice(N_CUST, 260, replace=False):                    (1)
    start = rng.integers(0, DAYS * 1440 - 60)
    for mm in np.sort(rng.integers(0, 40, rng.integers(3, 8))):
        rows.append(dict(customer_id=v, minute=start + mm,
            amount=round(rng.uniform(0.79, 9.99), 2),               (2)
            merchant_cat=rng.choice(["online_retail", "gambling"]),
            distance_km=round(rng.exponential(8), 1),
            new_device=int(rng.random() < .15), foreign=0,
            is_fraud=0, scheme="legit"))
normal = pd.concat([normal, pd.DataFrame(rows)], ignore_index=True)
print("legit burst txns added:", len(rows))
```

OUTPUT

```
legit burst txns added: 1293                                        (3)
```

1 260 customers buy in bursts

App-store or in-game purchases: 3–7 payments within 40 minutes.

2 Tiny amounts, online or gambling

£0.79–£9.99 — exactly the shape of card testing. These are the hardest negatives in the data.

3 1,293 burst payments added

Velocity features will fire on these too; the model must learn the subtler differences (new device, distance).

Why we deliberately made the problem harder

Without hard negatives

In an early version of this tutorial, fraud only ever happened at night, online, from new devices. Supervised models scored a perfect 1.000 — and taught nothing true about the real world.

With hard negatives

Night owls, travellers, bursty gamers and round ATM amounts overlap with fraud signals. Scores become realistic and the *differences between methods* become visible.

WATCH OUT

When a fraud model looks perfect, suspect the data before celebrating. Perfect separation usually means a leak or an unrealistically easy dataset.

Fraud scheme 1 — card testing — the code

PYTHON

```
def card_testing(n_victims=40):
    rows = []
    for v in rng.choice(N_CUST, n_victims, replace=False):
        start = rng.integers(0, DAYS * 1440 - 60)
        k = rng.integers(4, 12)            # burst size                    (1)
        late = start >= 60 * 1440          # attackers adapt later         (2)
        for m in np.sort(rng.integers(0, 40, k)):
            rows.append(dict(customer_id=v, minute=start + m,
                amount=round(rng.uniform(0.5, 9.99), 2),
                merchant_cat=("gambling" if late and
                              rng.random() < .6 else "online_retail"),
                distance_km=round(rng.uniform(5, 900), 1),
                new_device=int(rng.random() < .7),                          (3)
                foreign=int(rng.random() < .3),
                is_fraud=1, scheme="card_testing"))
    return pd.DataFrame(rows)
ct = card_testing(); print(ct.shape)
print(ct.groupby(ct.minute >= 60*1440).merchant_cat
      .value_counts().to_string())
```

1 Bursts of 4–11 payments

Within 40 minutes for each of 40 victim cards.

2 Attackers adapt

Bursts starting after day 60 move to gambling merchants 60% of the time — a drift the models never saw in training.

3 Mostly new devices

70% from a new device; 30% foreign; anywhere 5–900 km away.

Fraud scheme 1 — card testing — the output

PYTHON · LAST LINES OF THE CELL ABOVE

```
ct = card_testing(); print(ct.shape)
print(ct.groupby(ct.minute >= 60*1440).merchant_cat
      .value_counts().to_string())
```

OUTPUT

```
(316, 9)
minute  merchant_cat
False   online_retail    205
True    gambling          56
        online_retail     55
```

4

4

4 The drift, confirmed

Before day 60: 205 online-retail tests. After: 56 gambling and 55 online retail.

Fraud scheme 2 — account takeover

PYTHON

```
def account_takeover(n=150):
    v = rng.choice(N_CUST, n)
    night = rng.choice([0, 2, 4, 9, 13, 18, 21, 23], n) * 60          (1)
    return pd.DataFrame(dict(customer_id=v,
        minute=rng.integers(0, DAYS, n) * 1440 + night
               + rng.integers(0, 60, n),
        amount=(cust.typical_spend.values[v]
                * rng.uniform(2, 10, n)).round(2),                    (2)
        merchant_cat=rng.choice(["electronics",
                    "online_retail", "gambling"], n),
        distance_km=rng.exponential(30, n).round(1),
        new_device=rng.binomial(1, 0.55, n),                          (3)
        foreign=rng.binomial(1, 0.15, n),
        is_fraud=1, scheme="account_takeover"))
ato = account_takeover(); print(ato.shape)
```

OUTPUT

```
(150, 9)                                                              (4)
```

1 Any hour, many at night

Hours drawn from a mix of night and day slots: attackers do not keep office hours, but they do not always strike at night either.

2 2–10× the customer's spend

Scaled to *this* customer's typical spend, so it is only extreme relative to the person.

3 New device 55% of the time

Session hijacking can reuse the victim's own device, so this flag is far from certain.

4 150 takeover payments

Fraud schemes 3 and 4 — foreign CNP and ATM cash-out — the code

PYTHON

```
def cnp_foreign(n=160):
    v = rng.choice(N_CUST, n)
    return pd.DataFrame(dict(customer_id=v,
        minute=rng.integers(0, DAYS * 1440, n),
        amount=rng.lognormal(4.2, 0.8, n).round(2),
        merchant_cat=rng.choice(["online_retail", "travel"], n),
        distance_km=rng.uniform(150, 9000, n).round(1),
        new_device=rng.binomial(1, 0.4, n),
        foreign=rng.binomial(1, 0.8, n),
        is_fraud=1, scheme="cnp_foreign"))
def atm_cash_out(n=90):
    v = rng.choice(N_CUST, n)
    return pd.DataFrame(dict(customer_id=v,
        minute=rng.integers(0, DAYS, n) * 1440
            + rng.choice([1, 2, 3, 4], n) * 60,
        amount=rng.choice([100., 200., 250., 300., 500.], n),
        merchant_cat="atm",
        distance_km=rng.uniform(20, 300, n).round(1),
        new_device=0, foreign=rng.binomial(1, .3, n),
        is_fraud=1, scheme="atm_cash_out"))
cnp, cash = cnp_foreign(), atm_cash_out()
print(cnp.shape, cash.shape)
```

1

1 Foreign CNP: far away

150–9,000 km from home, 80% foreign, travel or online retail, at any time.

2 Cash-out: small hours

Withdrawals between 01:00 and 04:59 in round amounts of £100–£500.

Fraud schemes 3 and 4 — foreign CNP and ATM cash-out — the output

PYTHON · LAST LINES OF THE CELL ABOVE

```
        is_fraud=1, scheme="atm_cash_out"))
cnp, cash = cnp_foreign(), atm_cash_out()
print(cnp.shape, cash.shape)
```

OUTPUT

```
(160, 9) (90, 9)
```

3

3 **160 CNP and 90 cash-out rows**

Fraud scheme 5 — a lost or stolen card

PYTHON

```
def lost_stolen(n_cards=45):
    rows = []
    for v in rng.choice(N_CUST, n_cards, replace=False):
        start = rng.integers(0, DAYS * 1440 - 600)
        for m in np.sort(rng.integers(0, 480, rng.integers(2, 6))):
            rows.append(dict(customer_id=v, minute=start + m,
                amount=round(cust.typical_spend[v]
                             * rng.uniform(1.2, 4), 2),
                merchant_cat=rng.choice(["grocery", "fuel",
                             "restaurant", "electronics"]),
                distance_km=round(rng.exponential(12), 1),
                new_device=0, foreign=0,
                is_fraud=1, scheme="lost_stolen"))
    return pd.DataFrame(rows)
lost = lost_stolen(); print(lost.shape)
```

OUTPUT

```
(164, 9)
```

1 A short local spree

2–5 payments within eight hours of the theft.

2 Only 1.2–4× normal spend

Big enough to profit, small enough to look like an ordinary busy day.

3 Everyday merchants, near home, no device change

Card-present, so device and foreign flags never fire. This is the scheme every model will find hardest.

4 164 payments

Assembling one timeline

PYTHON

```
tx = pd.concat([normal, ct, ato, cnp, cash, lost],                    1
               ignore_index=True)
t0 = pd.Timestamp("2026-01-01")
tx["ts"] = t0 + pd.to_timedelta(tx.minute, unit="min")                2
tx = (tx.sort_values(["ts"]).reset_index(drop=True)
       .rename_axis("tx_id").reset_index())
tx["hour"] = tx.ts.dt.hour                                            3
tx["day"] = (tx.minute // 1440).astype(int)
print(tx.shape)
print(tx[["ts", "amount", "merchant_cat", "is_fraud"]].tail())
```

OUTPUT

```
(62173, 13)                                                           4
```

	ts	amount	merchant_cat	is_fraud
62168	2026-03-31 23:32:00	28.28	electronics	0
62169	2026-03-31 23:36:00	18.78	fuel	0
62170	2026-03-31 23:43:00	83.77	fuel	0
62171	2026-03-31 23:52:00	7.68	online_retail	0
62172	2026-03-31 23:57:00	24.47	grocery	0

1 Stack all six tables

Everyday payments plus the five fraud schemes.

2 Real timestamps

Minutes since 1 January 2026 become dates, then rows are sorted in time order and given an id.

3 Helper columns

Hour of day and day number (0–89) for features and for the train/test split.

4 62,173 rows, 13 columns

The last payments of 31 March. Nothing in a single row says ‘fraud’ — we must infer it.

The final schema

tx_id / customer_id	integer	transaction and customer identifiers
ts, hour, day	timestamp / int	when the payment happened
amount	float (£)	payment value
merchant_cat	text	one of eight merchant categories
distance_km	float	distance from the customer's home area
new_device, foreign	0 / 1	first time seen device; foreign merchant country
is_fraud	0 / 1	the label (only used to evaluate unsupervised models)
scheme	text	which scheme generated the row — for analysis only, never a feature

WATCH OUT

Two columns exist only because we simulated the data: is_fraud and scheme. Using either as a model input would be cheating ('target leakage').

Check your understanding

Q1 Why add legitimate night-shift workers?

Answer: So that ‘night-time’ is a signal rather than proof, as in real portfolios.

Q2 What makes lost/stolen fraud hard?

Answer: Local, card-present, everyday merchants, no device change and only slightly higher amounts.

Q3 What changes for card-testers after day 60?

Answer: 60% of bursts move to gambling merchants — a pattern unseen in training.

Q4 Why is ‘scheme’ never used as a feature?

Answer: It is derived from the label; using it would leak the answer.

Key takeaways

01 Synthetic data gives exact ground truth and safe sharing — at the cost of realism.

02 Hard negatives make results meaningful: travellers, night owls, bursts, round ATM cash.

03 Each fraud scheme is a small, readable function with its own fingerprint.

04 Scale fraud amounts to the individual customer, as real criminals effectively do.

05 Build drift into the test period to see how models cope with change.

06 Perfect scores usually mean an unrealistic dataset or a leak.

PART 5

Exploratory Data Analysis

Before any model, look. Exploratory data analysis (EDA) tells you which signals exist, which are noisy, and which mistakes to avoid.

IN THIS PART

- Data types and quality
- How rare is fraud?
- Amounts, merchants and hours
- Device, location and foreign signals
- What EDA tells the modeller

EXPLORATORY DATA ANALYSIS

The EDA mindset for anomaly work

1 Check the plumbing

Types, missing values, duplicates, time ranges. Broken data creates fake anomalies.

2 Measure the rarity

How many positives? That decides metrics, sampling and alert budgets.

3 Compare the groups

For each column: how does fraud differ from legitimate? Medians, rates and shapes, not just averages.

4 Look for context

Rates by hour, by merchant, by customer — anomalies are often contextual.

Data types and missing values

PYTHON

```
print(tx.dtypes.to_string())
print("missing values:", int(tx.isna().sum().sum()))
```

OUTPUT

```
tx_id                    int64
customer_id              int64
minute                   int64
amount                 float64   1
merchant_cat               str   2
distance_km            float64
new_device               int64
foreign                  int64
is_fraud                 int64
scheme                     str
ts           datetime64[us]
hour                     int32
day                      int64
missing values: 0                 3
```

1 Numbers are numbers

Amounts and distances are floats; flags and ids are integers — nothing read in as text by mistake.

2 Text columns

merchant_cat and scheme are strings; merchant category will need encoding before modelling.

3 No missing values

Real data rarely looks like this. Missing device or location data is itself often a fraud signal — never just drop it.

How rare is fraud?

PYTHON

```
counts = tx.is_fraud.value_counts()
print(counts.to_string())
print("Fraud rate: %.2f%%" % (100 * tx.is_fraud.mean()))
print("Legit per fraud: %d to 1" % (counts[0] // counts[1]))
```

OUTPUT

```
is_fraud
0       61293
1         880
Fraud rate: 1.42%
Legit per fraud: 69 to 1
```

1. **880 frauds among 62,173 payments**
2. **1.42%**

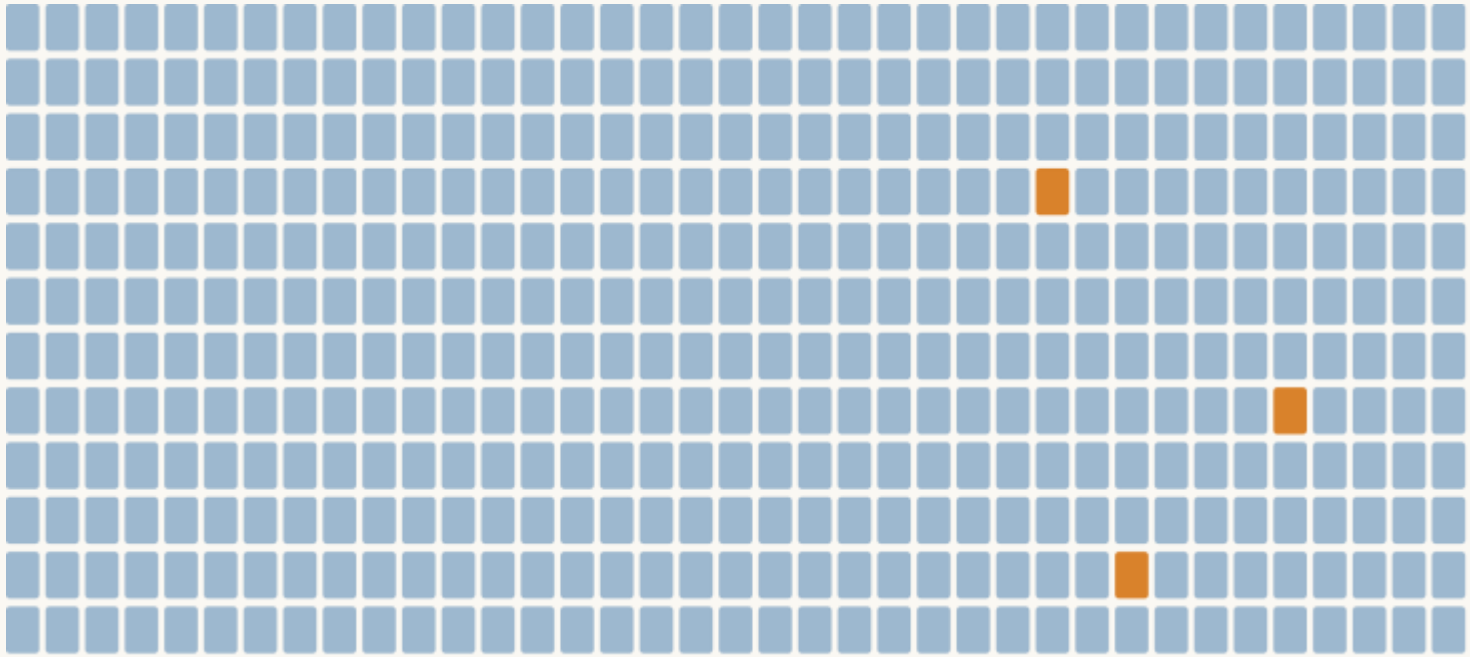
   Higher than most real portfolios (often well below 0.5%), chosen so the tutorial has enough fraud to analyse.
3. **69 legitimate payments per fraud**

   A detector that alerts at random would be wrong 69 times for every catch.

EXPLORATORY DATA ANALYSIS

Seeing the imbalance

Each square below is roughly 140 transactions.



441

squares of legitimate payments

~6

squares' worth of fraud spread among them (three shown in orange)

KEY INSIGHT

Finding fraud is finding a few orange squares in a sea of blue — that is why we need rankings and alert budgets.

Rows per scheme

PYTHON

```
print(tx.scheme.value_counts().to_string())
```

OUTPUT

```
scheme
legit          61293
card_testing     316   (1)
lost_stolen      164
cnp_foreign      160
account_takeover 150
atm_cash_out      90   (2)
```

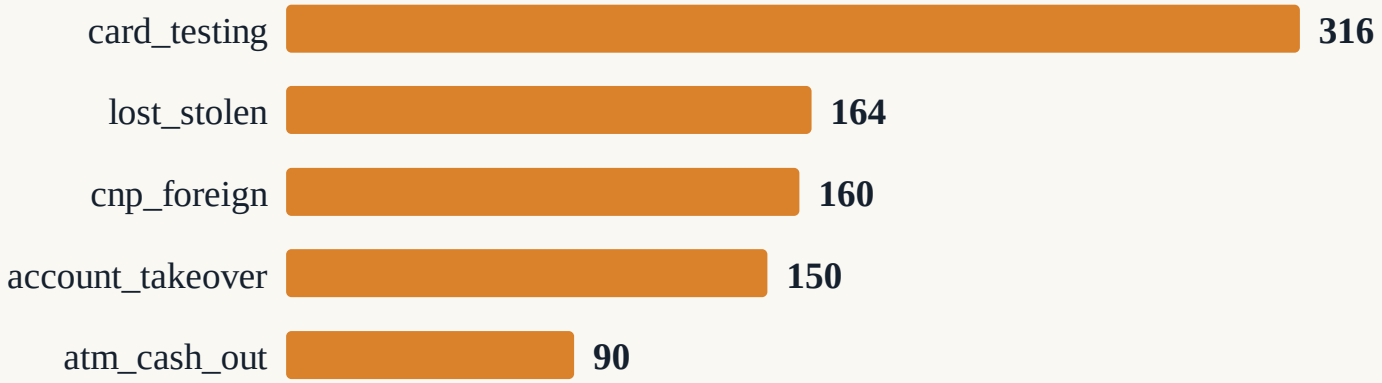
1 Card testing dominates by count

316 rows, because each attack is a burst of many small payments.

2 The smallest group

Only 90 ATM cash-out rows. Small groups are easy to under-weight; per-scheme reporting keeps them visible.

Fraud rows by scheme



KEY INSIGHT

Counting *rows* over-states card testing; counting *incidents* (cards attacked) would rank it lower. Decide which unit matters for your business before reporting.

PRO TIP

Losses differ too: card testing is cheap per row; takeover and cash-out cost far more each.

Are fraudulent payments larger?

PYTHON

```
print(tx.groupby("is_fraud").amount.describe()
      [["mean", "50%", "max"]].round(2))
```

3

OUTPUT

is_fraud	mean	50%	max	
0	35.29	24.20	738.71	1
1	86.57	47.52	546.88	2

1 Fraud is larger on average

Mean £86.57 versus £35.29.

2 But not always

The typical (median) fraud is £47.52 — and card testing is mostly under £10. ‘Large amount’ only catches some schemes.

3 Tip: compare medians

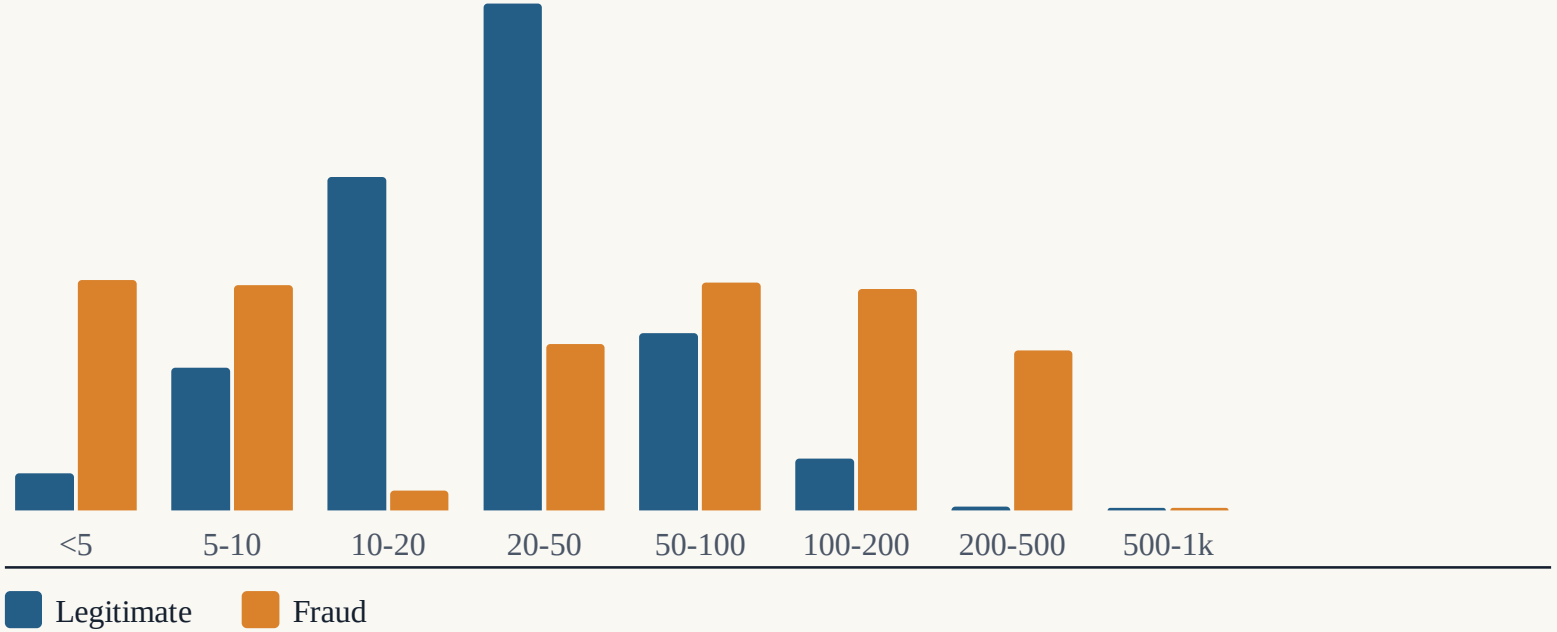
Averages are pulled about by a few big values; medians describe the typical case.

EXPLORATORY DATA ANALYSIS

Amount distribution: legitimate versus fraud

Each colour sums to 100%, so the two shapes can be compared despite the imbalance.

Share of each class in the amount band (%)



WHAT THE OUTPUT SHOWS

Fraud is *bimodal*: a spike under £5 (card testing) and a long tail above £100 (takeover, cash-out, CNP).

KEY INSIGHT

Legitimate spending clusters in the £10–£50 bands.

WATCH OUT

A single ‘amount too high’ rule would miss the left spike entirely.

Fraud rate by merchant category

PYTHON

```
cat = (tx.groupby("merchant_cat")
        .agg(n=("is_fraud", "size"), frauds=("is_fraud", "sum"),
             rate=("is_fraud", "mean"))
        .sort_values("rate", ascending=False))
cat["rate"] = (cat.rate * 100).round(2)
print(cat)
```

1

OUTPUT

merchant_cat	n	frauds	rate	
gambling	1906	95	4.98	2
online_retail	12982	417	3.21	
atm	3621	90	2.49	
electronics	3625	83	2.29	
travel	2764	60	2.17	
fuel	8624	48	0.56	
restaurant	10796	44	0.41	
grocery	17855	43	0.24	3

1 Group, count, average

For each category: rows, frauds and the fraud rate, sorted from riskiest to safest.

2 Gambling is riskiest

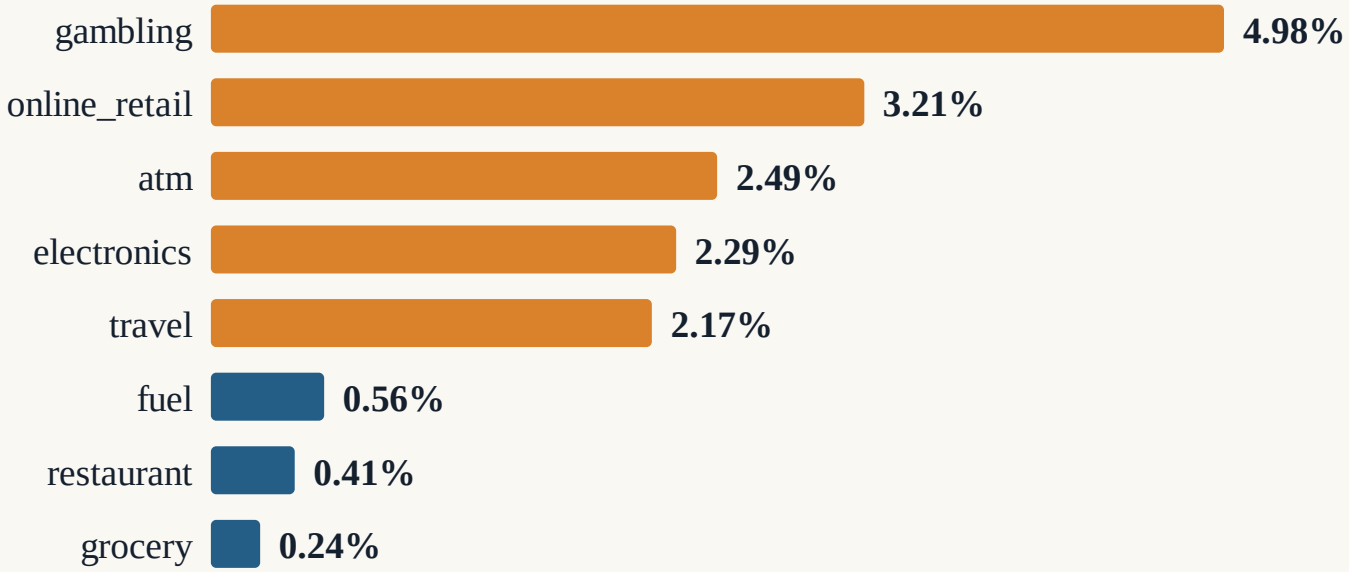
Nearly 5% — boosted by the card-testers who moved there in the last month.

3 Grocery is safest

0.24%, but not zero: lost/stolen cards are used in everyday shops.

EXPLORATORY DATA ANALYSIS

Merchant risk at a glance



WHAT THE OUTPUT SHOWS

Orange bars mark categories above 2%. The top five categories are where four of the five schemes operate.

KEY INSIGHT

A category's risk is an average. The model must still decide *which* gambling payment is risky — most are not.

Fraud rate by hour of day

PYTHON

```
by_hour = tx.groupby("hour").is_fraud.mean().mul(100).round(2)
print("Fraud rate by hour (%):")
print(by_hour.iloc[[0, 2, 4, 8, 12, 16, 20, 23]].to_string())
```

OUTPUT

```
Fraud rate by hour (%):
hour
0        8.96
2       13.84   1
4        8.15
8        1.26
12       0.76   2
16       0.74
20       1.33
23       7.04
```

1 Night is high-risk

Around 9–14% of payments between midnight and 4am are fraud…

2 …day-time is not

…versus under 1% at noon and 4pm.

3 Remember the night owls

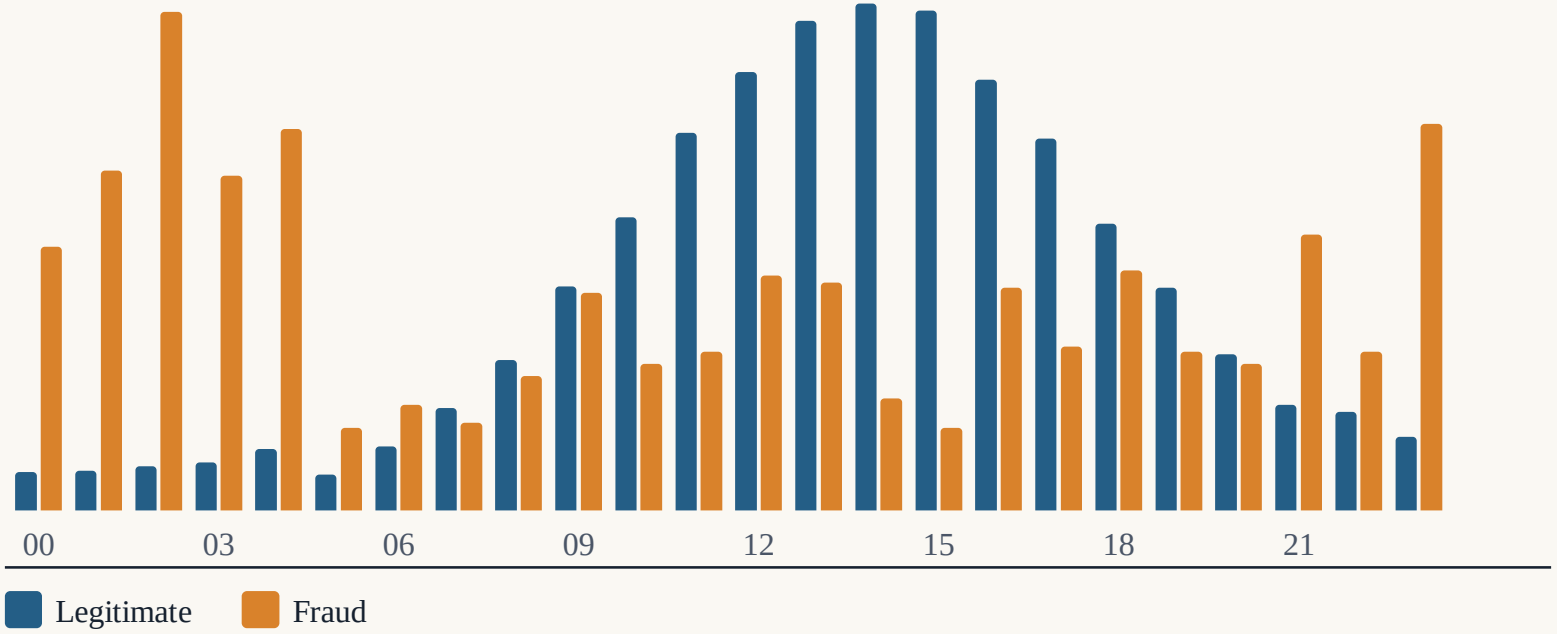
Even at 2am most payments are legitimate: night is a risk *multiplier*, not a verdict.

EXPLORATORY DATA ANALYSIS

When does each class happen?

Each colour sums to 100% across the 24 hours.

Share of each class by hour (%)



WHAT THE OUTPUT SHOWS

Legitimate activity follows the daytime bell, plus a small night-owl shoulder.

KEY INSIGHT

Fraud is spread far more evenly across the clock, with heavy night peaks from cash-out and takeover.

Device, foreign and distance signals

PYTHON

```
print(tx.groupby("is_fraud")[["new_device", "foreign"]]
      .mean().mul(100).round(1))
print()
print(tx.groupby("is_fraud").distance_km.median())
```

OUTPUT

```
new_device     foreign
is_fraud
0                   5.2         1.7                    1
1                  38.6        30.2                    2

is_fraud
0          5.6
1        191.7                                         3
Name: distance_km, dtype: float64
```

1 New device: 38.6% vs 5.2%

Seven times more common in fraud, but most fraud still uses a known device.

2 Foreign: 30.2% vs 1.7%

Strong, but 1,015 legitimate foreign payments mean many false alarms if used alone.

3 Distance: median 192 km vs 6 km

The single strongest separator in this data — as later models will confirm.

EXPLORATORY DATA ANALYSIS

What EDA told us

Fraud is 1.42% of rows (69:1)	Use precision, recall and alert budgets; never accuracy
Fraud amounts are bimodal	Need amount relative to the customer, not raw amount alone
Risk concentrates in five categories	Encode merchant category; expect interactions with device and time
Night raises risk but night owls exist	Combine time with personal history
Distance, device, foreign are strong but noisy	Multi-variable models will beat any single rule
Card testing comes in bursts	Build velocity features over time windows

Key takeaways

01 Check types, ranges and missing values first; broken data fakes anomalies.

02 Quantify rarity early: it drives every later decision.

03 Compare distributions (medians, shares), not just averages.

04 Fraud is bimodal in amount, so raw thresholds fail.

05 Contextual signals — hour, merchant, distance — carry the most information.

06 Every EDA finding should become a feature or a modelling choice.

PART 6

Feature Engineering

Features are how we describe each payment to a model. Good features encode the analyst's intuition: 'is this normal *for this customer, at this moment*?'

IN THIS PART

- Time since previous payment
- Velocity over sliding windows
- A time-based train/test split
- Customer baselines and spend ratios
- Logs, flags, encoding and scaling

FEATURE ENGINEERING

What is a feature?

A **feature** is one measurable property of an event that a model can use. Raw columns are features; *engineered* features combine raw columns with context and history.

Raw

amount = £120

Contextual

payment at 03:00 → is_night = 1

Historical

£120 is 11× this customer's usual spend

Behavioural

4th payment on this card in the last hour

KEY INSIGHT

In fraud and security, the historical and behavioural features almost always matter more than the raw ones.

FEATURE ENGINEERING

Six families of fraud features

1 Amount

log amount, ratio to the customer’s median, round-number flag

2 Time

hour of day, night flag, minutes since the previous payment

3 Velocity

payments in the last hour and last 24 hours

4 Location

distance from home, foreign merchant flag

5 Device

new-device flag (in practice: device age, device sharing)

6 Merchant

merchant category as one-hot columns (in practice: merchant risk scores)

Time since the customer’s previous payment

PYTHON

```
tx = tx.sort_values(["customer_id", "ts"]).reset_index(drop=True)   1
gap = tx.groupby("customer_id").ts.diff()                           2
tx["mins_since_prev"] = (gap.dt.total_seconds() / 60).fillna(1e4)   3
print(tx.mins_since_prev.describe().round(1).to_string())
```

OUTPUT

```
count    62173.0
mean      4214.5
std       4189.1
min          0.0
25%       1250.0
50%       2896.0   4
75%       5934.0
max      39243.0
```

1 Sort by customer, then time

So each row sits directly after the same customer’s previous payment.

2 Difference within each customer

groupby(...).diff() subtracts the previous timestamp — never mixing customers.

3 First payment ever?

There is no previous one, so we fill with a large gap (10,000 minutes ≈ one week).

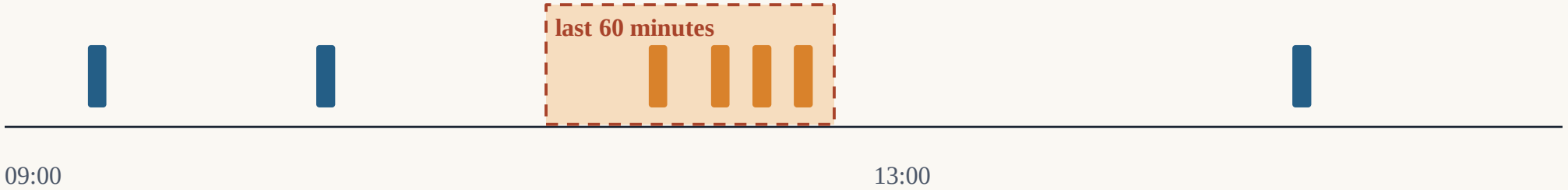
4 Typical gap: about two days

Median 2,896 minutes. Zero-minute gaps exist: bursts, both legitimate and fraudulent.

FEATURE ENGINEERING

Velocity: counting payments in a sliding window

At the moment of the payment at the right edge of the window, how many of this customer's payments fall inside it?



WHAT THE OUTPUT SHOWS

Here four payments fall in the hour ending at the current one, so tx_1h = 4. Normal customers rarely exceed 1 or 2.

WATCH OUT

The window only looks *backwards*. Counting future payments would be impossible in real time and would leak information.

Building velocity features

PYTHON

```
def rolling_count(window):
    s = (tx.set_index("ts").groupby("customer_id")
          .amount.rolling(window).count())                    1
    return s.values          # aligned: sorted by customer, ts  2
tx["tx_1h"] = rolling_count("60min")
tx["tx_24h"] = rolling_count("24h")
print(tx.groupby("is_fraud")[["tx_1h", "tx_24h"]].mean().round(2))
```

OUTPUT

```
           tx_1h    tx_24h
is_fraud
0           1.07      1.39
1           2.39      2.85                                   3
```

1 Time-based rolling windows

pandas counts rows within the last ‘60min’ or ‘24h’ for each customer, up to and including the current payment.

2 Alignment matters

Because the table is sorted by customer then time, the rolling result lines up row-for-row.

3 Fraud has higher velocity

On average 2.39 payments in the hour for fraud versus 1.07 for legitimate — driven by card-testing bursts.

FEATURE ENGINEERING

Split by time, never at random

Models are always used on the future, so we must evaluate them on a later period than they learned from.



Why not a random split?

Random splits let the model learn from payments *after* the ones it is tested on — for example from the same card-testing burst — which inflates scores.

Bonus: drift is tested

Our test month contains the card-testers' move to gambling. Only a time split exposes whether models survive that change.

Rule for all statistics

Anything learned from data — medians, scalers, encoders, models — is fitted on the training period only.

The time split in code

PYTHON

```
TRAIN_END = 60                    # days 0-59 train, 60-89 test
is_train = tx.day < TRAIN_END
print("train rows:", int(is_train.sum()),
      " fraud:", int(tx.is_fraud[is_train].sum()))
print("test  rows:", int((~is_train).sum()),
      " fraud:", int(tx.is_fraud[~is_train].sum()))
```

OUTPUT

```
train rows: 41319       fraud: 612
test       rows: 20854       fraud: 268
```

1 One boundary: day 60

Everything before trains; everything from day 60 onwards tests.

2 41,319 training rows, 612 fraud

3 20,854 test rows, 268 fraud

The test fraud rate (1.29%) is a little lower than training (1.48%) — real periods differ too.

FEATURE ENGINEERING

Customer baselines

To ask ‘is this unusual *for this customer*?’, we first need each customer’s normal level.

Median, not mean

The median spend ignores the odd big purchase, so one sofa does not redefine ‘normal’.

Learned on training days only

Using the test month to compute medians would let future fraud contaminate the baseline.

A fallback for newcomers

Customers with no history get the overall median — new customers are a classic blind spot.

FEATURE ENGINEERING

Spend relative to the customer’s own normal

PYTHON

```
# Customer "normal spend" learned from TRAIN window only
med = tx[is_train].groupby("customer_id").amount.median()                1
tx["cust_median"] = tx.customer_id.map(med).fillna(                      2
    tx.amount[is_train].median())
tx["amount_ratio"] = (tx.amount / tx.cust_median).round(3)               3
print(tx.groupby("is_fraud").amount_ratio.median())
```

OUTPUT

```
is_fraud
0       1.000
1       1.992                                                            4
Name: amount_ratio, dtype: float64
```

1 Medians from training days only

One median per customer.

2 Fallback

Customers absent from training take the overall training median.

3 The ratio

amount ÷ the customer’s median: 1.0 means typical, 5.0 means five times their usual.

4 Fraud is about 2× typical

The median legitimate payment has a ratio of exactly 1.0, as designed; the median fraud is 1.99.

FEATURE ENGINEERING

Logs, night and round-number flags

PYTHON

```
tx["log_amount"] = np.log1p(tx.amount)                                  1
tx["log_dist"] = np.log1p(tx.distance_km)
tx["log_gap"] = np.log1p(tx.mins_since_prev)
tx["is_night"] = tx.hour.isin([0, 1, 2, 3, 4, 5]).astype(int)           2
tx["is_round"] = (tx.amount % 50 == 0).astype(int)                      3
print(tx.groupby("is_fraud")[["is_night", "is_round"]]
        .mean().round(3))
```

OUTPUT

is_fraud	is_night	is_round	
0	0.052	0.043	
1	0.368	0.103	4

1 Log transforms

log1p compresses huge values (see next slide) so £5,000 is not treated as 1,000 times more extreme than £5.

2 Night flag

00:00–05:59.

3 Round-number flag

Amount divisible by £50 — a cash-out habit, but also common for real ATM use.

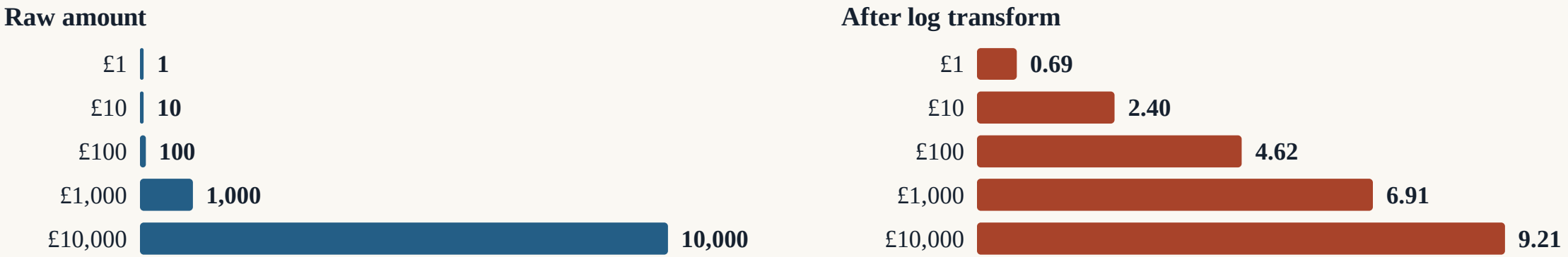
4 Fraud: 37% at night, 10% round

Versus 5% and 4% for legitimate payments.

FEATURE ENGINEERING

Why take logarithms?

Raw amounts are wildly skewed; the log scale turns ‘times bigger’ into ‘steps bigger’.



KEY INSIGHT

On the raw scale, £1 to £100 is invisible next to £10,000. On the log scale each tenfold jump is an equal step — distance-based models (LOF, SVM, k-means) behave far better.

FEATURE ENGINEERING

Encoding categories

Models need numbers. A merchant category like ‘gambling’ must be turned into numeric columns.

One-hot encoding (used here)

One 0/1 column per category. Simple and safe for 8 categories.

Label numbering (avoid)

grocery = 1, fuel = 2, … invents a false order: is fuel ‘between’ grocery and restaurant?

Target / risk encoding (advanced)

Replace each category with its historical fraud rate — powerful, but must be computed out-of-fold on past data to avoid leakage.

One-hot encoding the merchant category

PYTHON

```
dummies = pd.get_dummies(tx.merchant_cat, prefix="cat",
                         dtype=int)
tx = pd.concat([tx, dummies], axis=1)
print("\n".join(dummies.columns))
```

1

OUTPUT

```
cat_atm
cat_electronics
cat_fuel
cat_gambling
cat_grocery
cat_online_retail
cat_restaurant
cat_travel
```

2

1 get_dummies

Creates one integer column per category, named with a ‘cat_’ prefix.

2 Eight new columns

Exactly one of them is 1 on every row.

FEATURE ENGINEERING

The final feature matrix

PYTHON

```
NUM = ["log_amount", "amount_ratio", "hour", "is_night",                    1
       "log_dist", "new_device", "foreign", "tx_1h",
       "tx_24h", "log_gap", "is_round"]
FEATS = NUM + list(dummies.columns)                                         2
tr, te = tx[tx.day < TRAIN_END], tx[tx.day >= TRAIN_END]
X_tr, y_tr = tr[FEATS].values, tr.is_fraud.values                           3
X_te, y_te = te[FEATS].values, te.is_fraud.values
print("X_tr", X_tr.shape, " X_te", X_te.shape)
print(len(FEATS), "features")
```

OUTPUT

```
X_tr (41319, 19)        X_te (20854, 19)                                    4
19 features
```

1 11 numeric features

Amount, ratio, time, location, device, velocity and round-number signals.

2 + 8 merchant columns = 19

3 X and y

X holds the features; y holds the labels — used for training supervised models and for scoring every model.

4 41,319 × 19 and 20,854 × 19

Rows by features for the training and test periods.

FEATURE ENGINEERING

Scaling: putting features on comparable rulers

Why scale?

Distance-based methods add up differences across features. Without scaling, ‘hour’ (0–23) would swamp ‘new_device’ (0–1) purely because of its units.

Standard scaling

Centre on the mean; divide by the standard spread. Sensitive to extreme values — which anomaly data is full of.

Robust scaling (used here)

Centre on the median; divide by the interquartile range (the middle 50%). Extreme values barely affect it, so anomalies stay extreme.

PRO TIP

Tree-based models (Isolation Forest, random forests, boosting) do not need scaling — but it does no harm, and LOF, SVMs and neural nets require it.

Robust scaling in code

PYTHON

```
from sklearn.preprocessing import StandardScaler, RobustScaler
scaler = RobustScaler().fit(X_tr)
Xs_tr, Xs_te = scaler.transform(X_tr), scaler.transform(X_te)
print("median after scaling:",
      np.round(np.median(Xs_tr[:, :4], axis=0), 2))
print("raw log_amount range :",
      X_tr[:, 0].min().round(2), "to", X_tr[:, 0].max().round(2))
```

OUTPUT

```
median after scaling: [0. 0. 0. 0.]
raw log_amount range : 0.41 to 6.61
```

1 Fit on training data only

The scaler learns medians and spreads from the past, then applies them to both periods.

2 Medians become zero

After scaling, the typical value of each feature sits at 0.

3 Raw log-amount spans 0.41–6.61

Robust scaling keeps that full spread — anomalies remain far from zero.

FEATURE ENGINEERING

Feature catalogue

log_amount	Amount	size of payment, compressed	takeover, CNP, cash-out
amount_ratio	Amount	size relative to the customer’s median	takeover, lost/stolen
hour, is_night	Time	time-of-day context	cash-out, takeover
log_gap	Time	minutes since previous payment	card testing
tx_1h, tx_24h	Velocity	recent payment counts	card testing, lost/stolen
log_dist, foreign	Location	distance from home; abroad	CNP, cash-out
new_device	Device	first time this device is seen	card testing, takeover
is_round	Amount	divisible by £50	cash-out
cat_* (8 columns)	Merchant	merchant category	all schemes

Key takeaways

01 Engineered features encode an analyst’s questions as numbers.

02 Velocity over backward-looking windows captures collective anomalies.

03 Compare each payment with the customer’s own history (ratios to medians).

04 Split by time; fit every statistic on the past only.

05 Log-transform skewed values; one-hot encode categories.

06 Robust scaling keeps anomalies extreme and helps distance-based models.

PART 7

How to Measure a Detector

We set the scoreboard before the game. Every model in the rest of this tutorial is judged with the same honest, imbalance-aware metrics.

IN THIS PART

- The confusion matrix
- Precision and recall in words
- ROC-AUC and PR-AUC
- Precision and recall at k
- Our reusable evaluation helper

HOW TO MEASURE A DETECTOR

The confusion matrix

Every alerting decision lands in one of four boxes.

	Model says: legitimate	Model says: fraud (alert)
Truly legitimate	**True negative** Correctly left alone. The vast majority.	**False positive** A false alarm: analyst time, a declined customer.
Truly fraud	**False negative** Missed fraud: money lost.	**True positive** A catch.

HOW TO MEASURE A DETECTOR

Precision and recall in plain words

Precision

Of the payments we alerted on, how many were really fraud?
High precision = analysts are not wasting time. Precision of 0.25 means one real fraud in every four alerts.

Recall

Of all the fraud that happened, how much did we alert on?
High recall = little fraud slips through. Recall of 0.80 means we caught four frauds in five.

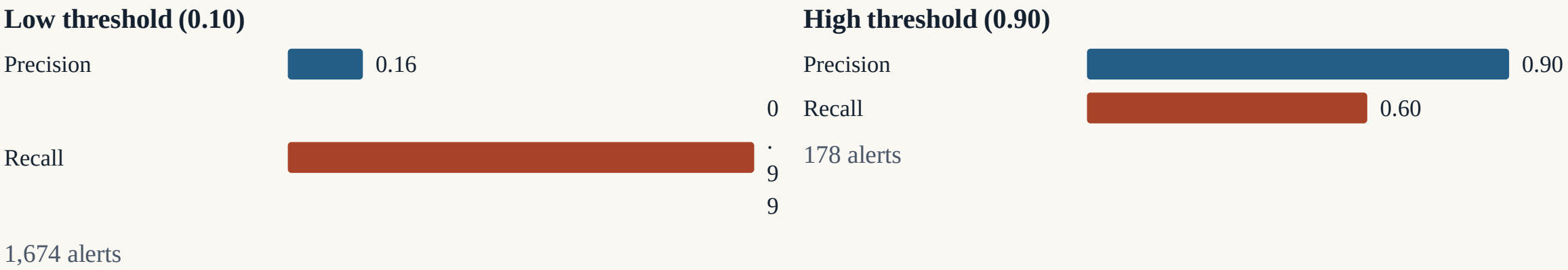
KEY INSIGHT

They pull against each other: alert on more payments and recall rises while precision falls. Choosing the balance is choosing the threshold.

HOW TO MEASURE A DETECTOR

The trade-off, from our own gradient-boosting model

Real numbers from Part 10: as the threshold rises, precision rises and recall falls.



KEY INSIGHT

Same model, two very different operating points. Which one is ‘better’ depends entirely on costs and capacity — Part 11 settles it with money.

HOW TO MEASURE A DETECTOR

Area-under-curve scores

Rather than judging one threshold, these summarise performance across **all** thresholds — useful for comparing models before you choose a cut-off.

ROC-AUC

The chance that a randomly picked fraud is ranked above a randomly picked legitimate payment. 0.5 = coin toss, 1.0 = perfect ranking.

PR-AUC (average precision)

How high precision stays as recall grows. A random model scores the fraud rate (≈ 0.013 here), so the scale is far more honest for rare events.

Why prefer PR-AUC here

With 69 legitimate per fraud, ROC-AUC can look superb (0.95+) while the alert queue is still mostly false alarms. PR-AUC exposes that.

HOW TO MEASURE A DETECTOR

Precision and recall at k: the analyst's view

Analysts do not review a threshold; they review the **top of a ranked list**. So we ask: if we look at the top k = 300 test payments, what happens?

Precision@300

What share of the 300 highest-scored payments are fraud?

Recall@300

What share of all 268 test frauds are inside those 300?

Why 300?

Roughly 10 reviews per day over the 30-day test month — a plausible small fraud team's capacity.

KEY INSIGHT

This is the metric that translates directly into 'frauds caught per analyst shift'.

A reusable evaluation helper

PYTHON

```
from sklearn.metrics import (roc_auc_score,                                  1
                             average_precision_score)
class Res(dict):                    # pretty, two-line printout
    def __str__(s):
        return ("%s\n  ROC-AUC %.3f  PR-AUC %.3f  P@300 %.3f"
                "  R@300 %.3f" % (s["model"], s["roc_auc"],
                s["pr_auc"], s["prec_at_k"], s["recall_at_k"]))
def evaluate(name, scores, y=None, k=300):
    y = y_te if y is None else y
    top = np.argsort(-scores)[:k]                                            2
    res = dict(model=name,                                                   3
               roc_auc=roc_auc_score(y, scores),
               pr_auc=average_precision_score(y, scores),
               prec_at_k=y[top].mean(),
               recall_at_k=y[top].sum() / y.sum())
    return Res({k_: (v if isinstance(v, str) else
                 round(float(v), 3)) for k_, v in res.items()})
print(evaluate("random", np.random.default_rng(0).random(len(y_te))))
```

OUTPUT

```
random
ROC-AUC 0.487   PR-AUC 0.013   P@300 0.013   R@300 0.015                    4
```

1 Two library metrics

roc_auc_score and average_precision_score (PR-AUC) from scikit-learn.

2 Top-k by score

Sort scores from highest to lowest and keep the first 300 positions.

3 One tidy result

Every model returns the same four numbers, printed on two lines.

4 The random baseline

ROC-AUC ≈ 0.5, PR-AUC ≈ 0.013 (the fraud rate), P@300 ≈ 0.013. Anything worth using must beat these comfortably.

Which metric for which question?

Does this model rank fraud above legitimate at all?	ROC-AUC
How good is the ranking where it matters, for a rare class?	PR-AUC (average precision)
How useful is the top of the alert queue?	Precision@k
How much fraud does our review capacity catch?	Recall@k
At our chosen threshold, how clean are alerts?	Precision
At our chosen threshold, how much fraud do we miss?	Recall (and £ value missed)
What does it cost overall?	Total cost = reviews × cost + missed fraud value

Key takeaways

01 Four outcomes: true/false positives and negatives; costs differ.

02 Precision = cleanliness of alerts; recall = share of fraud caught.

03 ROC-AUC flatters rare-event models; prefer PR-AUC.

04 Precision@k and recall@k mirror how analysts actually work.

05 A random model's PR-AUC equals the fraud rate — always compare with it.

06 Fix the scoreboard before comparing models.

PART 8

Statistical Baselines

Simple statistics are fast, transparent and surprisingly strong. Every machine-learning model must prove it beats them.

IN THIS PART

- Per-customer z-scores
- The interquartile-range fence
- Median absolute deviation — and its trap
- Expert rules
- A baseline scoreboard

STATISTICAL BASELINES

Why start simple?

A yardstick

Without a baseline you cannot tell whether a complex model adds anything worth its cost.

Explainability

‘Five times larger than usual for this customer’ is instantly understood by analysts and auditors.

Speed and robustness

Statistics compute in milliseconds and rarely break silently.

KEY INSIGHT

In practice, many production systems are a blend: statistical guards plus machine-learning scores.

STATISTICAL BASELINES

The z-score idea, in words

The question

How many ‘typical spreads’ away from this customer’s usual level is this payment?

Reading it

0 means perfectly typical. 1 or 2 is ordinary variation. 3 or more is rare — a classic alert line.

ANALOGY

A customer who spends £20 ± £5 and suddenly pays £45 is five spreads above normal. A customer who spends £20 ± £30 paying £45 is under one spread — not remarkable.

WATCH OUT

The method assumes roughly bell-shaped data and judges one variable at a time. Card testing (tiny amounts) sits *below* normal and will be missed by an ‘unusually high’ rule.

Per-customer z-scores on log amount

PYTHON

```
# Per-customer z-score on log amount (statistics from train)
g = tr.groupby("customer_id").log_amount                          1
mu, sd = g.mean(), g.std().fillna(0.5).clip(lower=0.1)            2
z = ((te.log_amount - te.customer_id.map(mu).fillna(3.3))
     / te.customer_id.map(sd).fillna(0.6))
z = z.values
print(evaluate("z-score amount", z))
print("Rule z > 3 flags", int((z > 3).sum()), "transactions")
```

OUTPUT

```
z-score amount
ROC-AUC 0.552   PR-AUC 0.057    P@300 0.133     R@300 0.149          3
Rule z > 3 flags 288 transactions                                     4
```

1 Personal statistics

Each customer's average and spread of log amount, from the training period.

2 Guard rails

Customers with a single payment get a default spread; spreads are never allowed below 0.1.

3 Barely better than random

PR-AUC 0.057 versus a random 0.013, and ROC-AUC only 0.552.

4 288 payments above z = 3

What the z-score rule catches — and misses

```python
flag = z > 3
tp = int((flag & (y_te == 1)).sum())
print("caught frauds :", tp, "of", int(y_te.sum()))
print("precision     : %.3f" % (tp / flag.sum()))
print(te[flag & (y_te == 1)].scheme.value_counts().to_string())
```

OUTPUT

```
caught frauds : 39 of 268                    1
precision     : 0.135                        2
scheme
atm_cash_out          19                     3
account_takeover      14
cnp_foreign            6
```

1 39 of 268 frauds caught

Recall of about 15%.

2 Precision 0.135

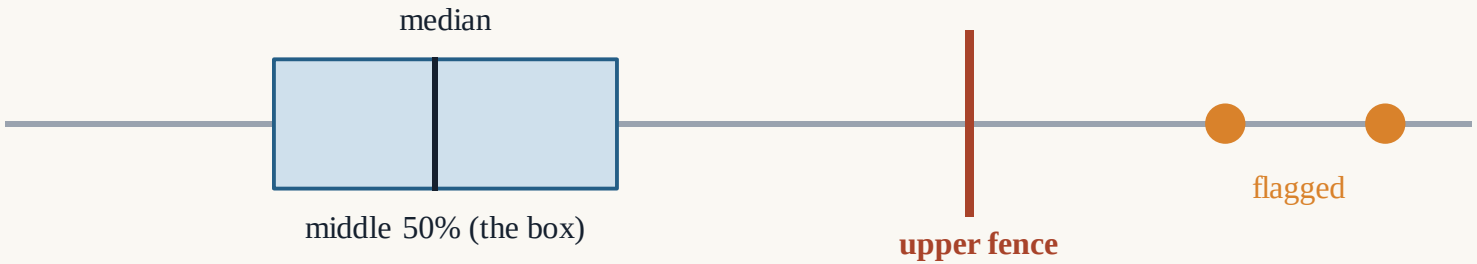
Roughly one fraud in every seven alerts.

3 Only ‘big amount’ schemes

Cash-out, takeover and CNP. Card testing and lost/stolen cards never appear: one-variable rules see one kind of anomaly.

The interquartile-range (IQR) fence

The rule behind every box plot.



KEY INSIGHT

Take the middle half of the data (the box). Measure its width. Anything more than one-and-a-half box-widths above the top of the box is flagged.

PRO TIP

It uses medians and quartiles, so a few extreme values cannot drag the fence upwards — a big improvement on averages for skewed data.

The IQR fence on spend ratio

PYTHON

```
q1, q3 = np.percentile(tr.amount_ratio, [25, 75])
fence = q3 + 1.5 * (q3 - q1)
flag = te.amount_ratio.values > fence
print("upper fence (amount_ratio): %.2f" % fence)
print("flagged:", int(flag.sum()),
      " precision: %.3f" % y_te[flag].mean(),
      " recall: %.3f" % (y_te[flag].sum() / y_te.sum()))
```

OUTPUT

```
upper fence (amount_ratio): 2.79
flagged: 1646        precision: 0.063        recall: 0.384
```

1 Quartiles from training

The 25th and 75th percentiles of amount_ratio.

2 Fence at 2.79×

Anything above 2.79 times the customer’s usual spend is flagged.

3 1,646 alerts, precision 0.063

Recall 38%, but 15 false alarms for every catch — far too noisy to operate.

STATISTICAL BASELINES

Median absolute deviation (MAD)

The idea

Like a z-score, but built from medians: the typical distance of values from their median. Very resistant to outliers.

When it shines

Skewed, spiky data — server traffic, per-host byte counts. We use it for network baselines in Part 14.

WATCH OUT

The trap: if more than half the values are identical, the typical distance from the median is zero. Dividing by zero makes every non-median value infinitely extreme.

KEY INSIGHT

Velocity features are exactly like that: most customers make one payment in any hour. Watch what happens next.

Robust z-scores — and the zero-spread trap

PYTHON

```
def robust_z(x, ref):
    m = np.median(ref)
    mad = np.median(np.abs(ref - m)) * 1.4826
    return (x - m) / mad
rz = robust_z(te.tx_1h.values, tr.tx_1h.values + 1e-9)
print("MAD of tx_1h in train:",
      np.median(np.abs(tr.tx_1h - tr.tx_1h.median())))
print("unique robust-z values:", np.unique(np.round(rz, 1))[:4])
print(evaluate("velocity (tx_1h)",
               te.tx_1h.values + 0.01 * te.log_amount.values))
```

OUTPUT

```
MAD of tx_1h in train: 0.0
unique robust-z values: [-inf          inf]
velocity (tx_1h)
ROC-AUC 0.857   PR-AUC 0.236    P@300 0.260     R@300 0.291
```

1 The robust z-score

Distance from the median divided by the scaled median absolute deviation.

2 MAD = 0

Over half the training rows have tx_1h equal to the median (1), so the spread is zero…

3 …and every value becomes ±infinity

The statistic is useless here. Always check spreads before dividing by them.

4 Plain velocity still works

Ranking by raw tx_1h (with a tiny amount tie-breaker) gives PR-AUC 0.236 — four times the z-score.

Expert rules: the analyst’s first line of defence

PYTHON

```
rules = ((te.tx_1h >= 4)                                          1
        | ((te.new_device == 1) & (te.amount_ratio > 4))          2
        | ((te.foreign == 1) & (te.log_dist > 6))                 3
        | ((te.merchant_cat == "atm") & (te.is_night == 1)))
f = rules.values
print("alerts:", int(f.sum()),
      "| precision %.3f" % y_te[f].mean(),
      "| recall %.3f" % (y_te[f].sum() / y_te.sum()))
```

OUTPUT

```
alerts: 535 | precision 0.284 | recall 0.567                    4
```

1 Velocity rule

Four or more payments in an hour.

2 Takeover rule

New device and more than four times usual spend.

3 Geography and cash rules

Foreign and very far away; ATM at night.

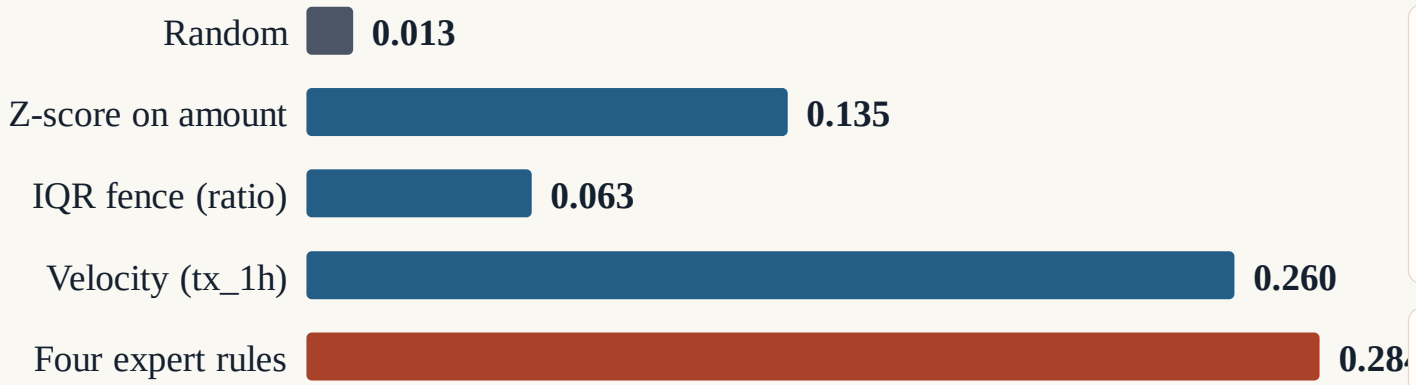
4 Recall 57% at precision 0.28

535 alerts. Four sensible rules beat every single-statistic baseline — domain knowledge is powerful.

STATISTICAL BASELINES

Baseline scoreboard

Precision of each baseline's alerts on the test month (higher is better).



WHAT THE OUTPUT SHOWS

Bars show precision of the alerts (for ranked methods, precision@300). The rules win, but still miss 43% of fraud.

KEY INSIGHT

This is the bar machine learning must clear. Part 9 starts with Isolation Forest.

Key takeaways

01 Statistical baselines are fast, explainable and essential yardsticks.

02 Z-scores assume bell shapes and see one variable at a time.

03 IQR and MAD resist outliers but can be noisy or degenerate.

04 Check for zero spreads before dividing — MAD collapses on count data.

05 A few expert rules beat any single statistic: domain knowledge matters.

06 Single-variable methods miss multi-variable schemes like card testing.

PART 9

Unsupervised Machine Learning

Eight algorithms that learn what ‘normal’ looks like without ever seeing a fraud label. Labels are used only afterwards, to grade them.

IN THIS PART

- Isolation Forest — in depth
- Local Outlier Factor
- One-Class SVM and Elliptic Envelope
- Clustering: k-means, DBSCAN, Gaussian mixtures
- PCA and autoencoder reconstruction
- Head-to-head comparison and ensembles

How every unsupervised detector works

Different algorithms, one shared recipe:

1. Learn normal

Fit on the training period. The model sees features only — no is_fraud column.

2. Score new events

Each test payment gets an anomaly score: higher means less like the training data.

3. Rank and alert

Sort by score; review the top k or alert above a threshold.

4. Grade (tutorial only)

Because we know the truth, we measure ROC-AUC, PR-AUC and precision/recall@300.

KEY INSIGHT

The four algorithm families differ in *how they define normal*: isolation (how easy to separate), density (how crowded the neighbourhood), boundary (inside or outside a learned shape), and reconstruction (how well it can be compressed and rebuilt).

Isolation Forest

Anomalies are few and different — so they are easy to isolate. Liu, Ting and Zhou, 2008.

UNSUPERVISED MACHINE LEARNING

The intuition: a game of twenty questions

ANALOGY

Imagine guessing a payment by asking random yes/no questions such as ‘is the amount above £83?’ or ‘is the distance above 40 km?’.

Ordinary payments

Sit in a crowd. You need many questions to single one out from its look-alikes.

Anomalous payments

Sit alone. One or two lucky questions isolate them completely.

Few questions

= short path = **high anomaly score**

Many questions

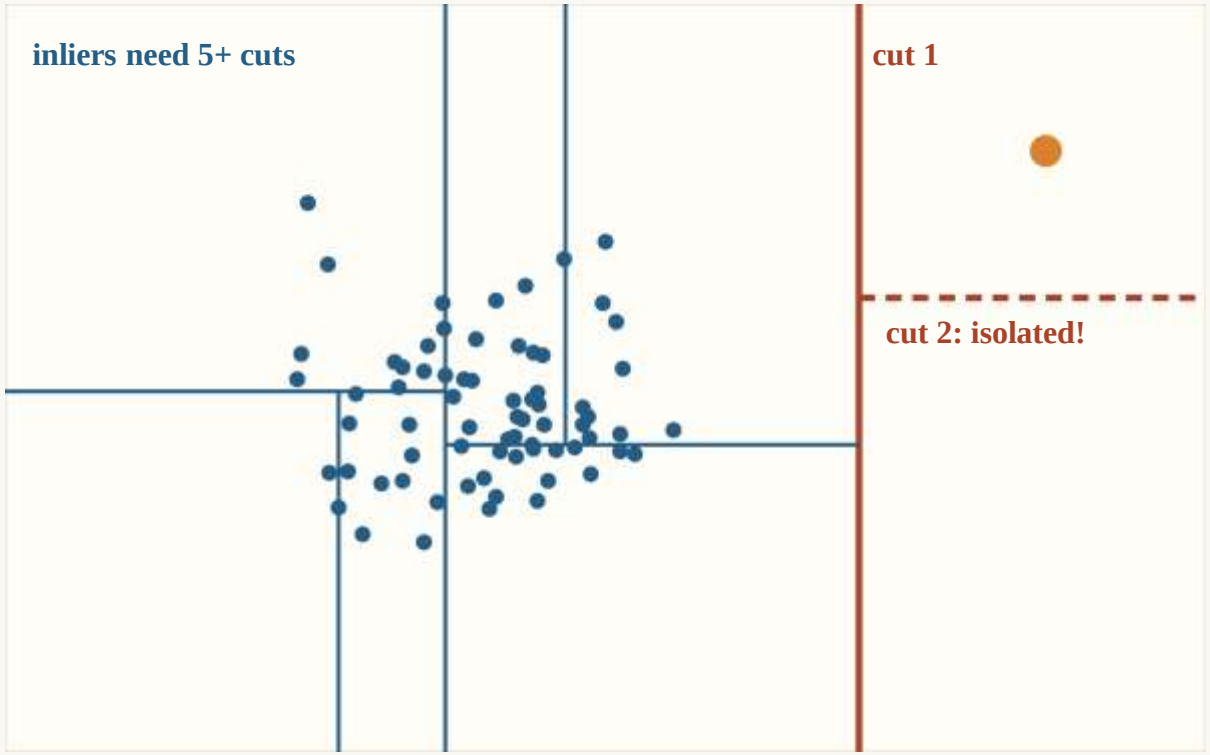
= long path = **normal**

KEY INSIGHT

Isolation Forest never measures distances or densities. It only counts how quickly random cuts can isolate a point — which is why it is fast and copes with many features.

Isolation, drawn

Two features, 70 normal payments and one outlier; each line is one random question.



KEY INSIGHT

Each line is a random cut on a random feature, at a random value between that feature's minimum and maximum.

WHAT THE OUTPUT SHOWS

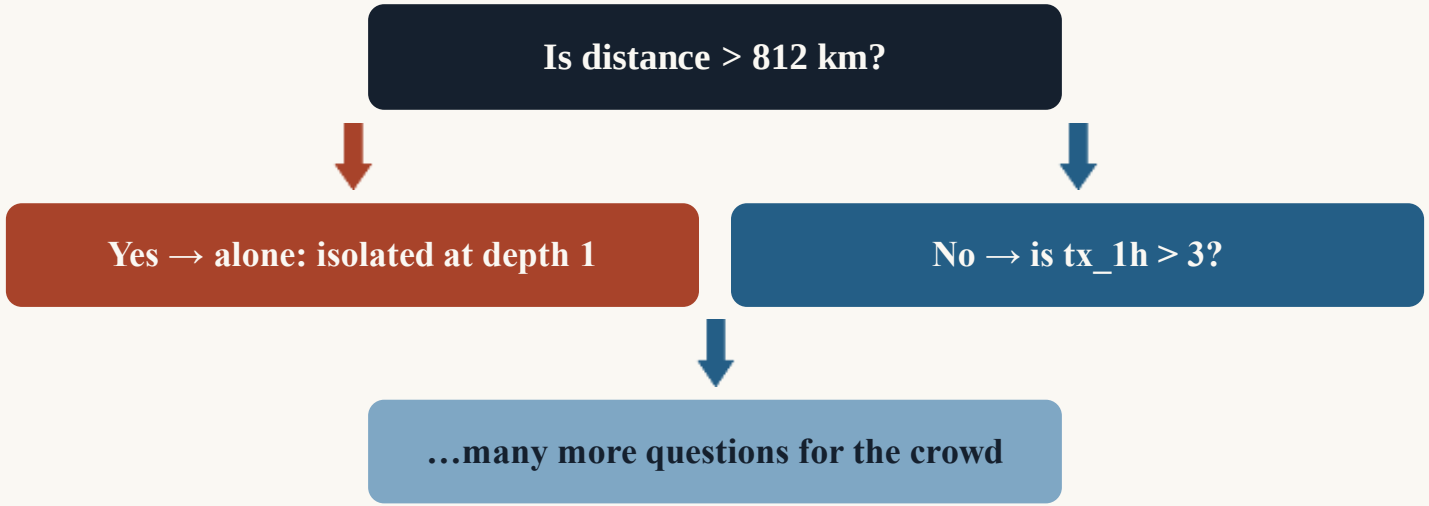
The orange outlier is alone in its region after **two cuts**. A point inside the blue crowd still shares its region after five or more.

PRO TIP

Repeat with many random trees and average: the outlier is *consistently* isolated early.

One isolation tree

A tree is just the sequence of random questions, recorded.



KEY INSIGHT

The **path length** — how many questions until a point stands alone — is averaged over hundreds of trees and converted into a score between 0 and 1. Scores near 1 are strong anomalies; around 0.5 and below are normal.

Isolation Forest settings that matter

n_estimators	number of random trees	100–300; more trees = steadier scores, slower
max_samples	rows each tree sees (sub-sample)	256 is the classic default; larger can help when anomalies cluster
contamination	share of points labelled −1 by predict()	only sets a threshold — does NOT change the scores
max_features	features each tree may cut on	1.0 (all) by default; lower adds diversity
random_state	seed for the random cuts	fix it for reproducible results

PRO TIP

score_samples() returns the raw score (lower = more abnormal in scikit-learn). We negate it so that **higher = stranger** throughout this tutorial.

Isolation Forest on the fraud data

PYTHON

```
from sklearn.ensemble import IsolationForest
import time
t = time.time()
iso = IsolationForest(n_estimators=300, max_samples=256,
                      contamination="auto", random_state=0)
iso.fit(Xs_tr)                    # NO labels used
s_iso = -iso.score_samples(Xs_te)    # higher = stranger
print("fit+score %.1fs" % (time.time() - t))
print(evaluate("IsolationForest", s_iso))
```

OUTPUT

```
fit+score 0.9s
IsolationForest
ROC-AUC 0.961   PR-AUC 0.495    P@300 0.457     R@300 0.511
```

1 300 trees, 256 rows each

contamination='auto' leaves the scores untouched.

2 No labels

fit() receives only the scaled feature matrix from the training period.

3 Flip the sign

So that higher scores mean more anomalous.

4 PR-AUC 0.495

Nearly 40× the random baseline and double the best statistical method — with zero labels.

5 P@300 = 0.457

Almost half of the top 300 alerts are fraud; they contain 51% of all test fraud.

What do the scores look like?

PYTHON

```
print(pd.Series(s_iso).describe().round(3).to_string())
print("mean score legit: %.3f" % s_iso[y_te == 0].mean())
print("mean score fraud: %.3f" % s_iso[y_te == 1].mean())
```

OUTPUT

```
count    20854.000
mean         0.420
std          0.058
min          0.354
25%          0.379
50%          0.398
75%          0.443
max          0.743   (1)
mean score legit: 0.417
mean score fraud: 0.598   (2)
```

1 A narrow range

Scores run from 0.354 to 0.743. Most payments sit near 0.4 — clearly ‘normal’.

2 Fraud averages 0.598

Versus 0.417 for legitimate payments: a clear shift, but the groups overlap.

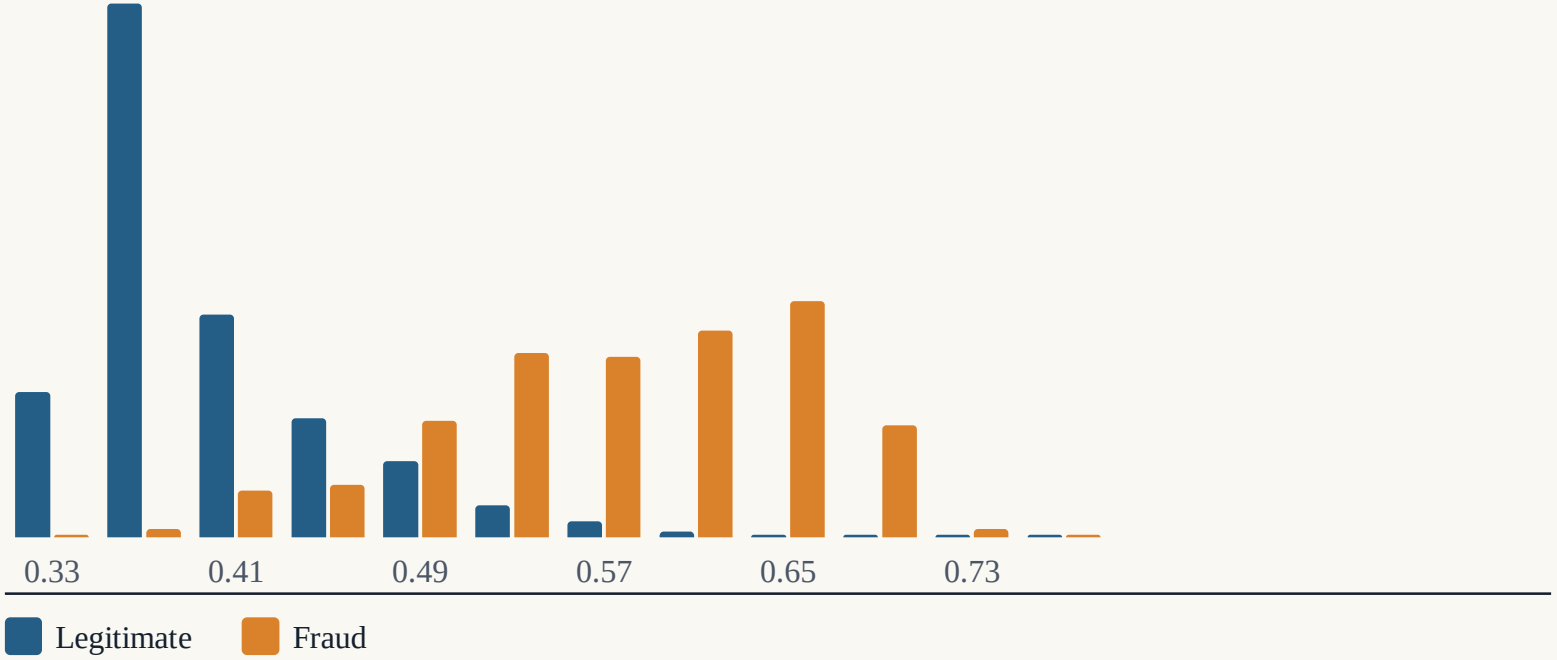
3 Scores rank; they are not probabilities

0.6 does not mean a 60% chance of fraud. Use them to order and threshold.

Score distributions: legitimate versus fraud

Each colour is normalised separately so both shapes are visible.

Density of payments at each score



WHAT THE OUTPUT SHOWS

Legitimate scores pile up around 0.37–0.41.

KEY INSIGHT

Fraud spreads out to the right, peaking around 0.57–0.69.

WATCH OUT

The overlap zone (0.45–0.55) is where analysts will see both — that is where precision is lost.

The ten most anomalous test payments

PYTHON

```
te_out = te.assign(score=s_iso).sort_values("score",
                                            ascending=False)
cols = ["amount", "merchant_cat", "tx_1h", "new_device",
        "is_fraud"]
print(te_out[cols].head(10).to_string(index=False))
```

OUTPUT

amount	merchant_cat	tx_1h	new_device	is_fraud
0.87	gambling	8.0	1	1
4.93	gambling	10.0	1	1
6.18	online_retail	8.0	1	1
1.05	gambling	4.0	1	1
1.26	gambling	6.0	1	1
5.64	gambling	10.0	1	1
0.55	gambling	8.0	1	1
3.62	online_retail	6.0	1	1
2.97	gambling	6.0	1	1
3.96	gambling	9.0	1	1

1 Sort by score

Highest first — exactly what an analyst's queue would show.

2 All ten are fraud

Tiny amounts, 4–10 payments in the hour, new devices, mostly gambling.

3 The drifted card-testers

Gambling card-testing never appeared in training, yet Isolation Forest ranks it first — the strength of unsupervised methods on *new* patterns.

Who is in the top 300 — and who is missing?

PYTHON

```
top = te_out.head(300)
print("Caught in top-300 alerts, by scheme:")
print(top.scheme.value_counts().to_string())
print("\nAll test frauds by scheme:")
print(te[te.is_fraud == 1].scheme.value_counts().to_string())
```

OUTPUT

```
Caught in top-300 alerts, by scheme:
scheme
legit            163
card_testing      88
atm_cash_out      32
cnp_foreign       11
account_takeover   6

All test frauds by scheme:
scheme
card_testing     111
cnp_foreign       47
lost_stolen       40
account_takeover  38
atm_cash_out      32
```

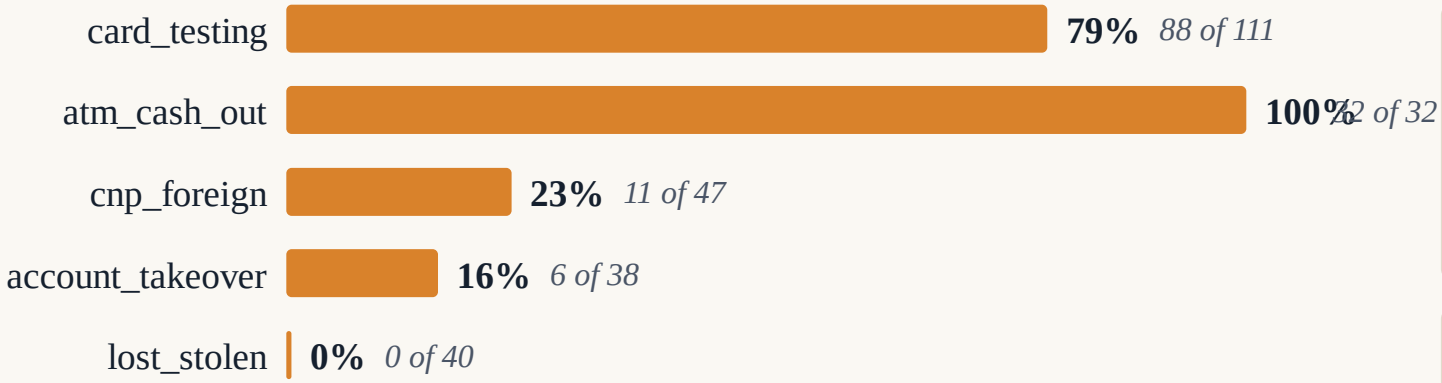
1. **163 legitimate**
   False alarms — mostly bursty gamers and travellers.

2. **Card testing: 88 of 111**
   Velocity plus new devices make bursts stand out.

3. **Cash-out: all 32 caught**
   Night, ATM, round, far away — highly unusual combinations.

4. **Lost/stolen: zero of 40**
   Absent from the list entirely. A card thief who behaves like the owner is *not anomalous* — no unsupervised method can see it.

Recall by scheme inside the top 300



WHAT THE OUTPUT SHOWS

Isolation Forest is excellent on loud, multi-signal schemes and weak on quiet ones.

KEY INSIGHT

This per-scheme view is far more informative than one overall number. Always break results down by segment.

PRO TIP

Quiet schemes need *labels* — supervised models in Part 10 learn the faint patterns of takeover and lost cards.

The contamination setting is just a threshold

PYTHON

```
for c in [0.005, 0.01, 0.02, 0.05]:
    m = IsolationForest(n_estimators=200, contamination=c,
                        random_state=0).fit(Xs_tr)
    pred = m.predict(Xs_te) == -1        # -1 = anomaly
    print("contam %.3f -> alerts %5d  prec %.3f  rec %.3f" % (
        c, pred.sum(), y_te[pred].mean(),
        y_te[pred].sum() / y_te.sum()))
```

1

OUTPUT

```
contam 0.005 -> alerts   118   prec 0.686   rec 0.302   2
contam 0.010 -> alerts   222   prec 0.541   rec 0.448   4
contam 0.020 -> alerts   402   prec 0.376   rec 0.563
contam 0.050 -> alerts  1056   prec 0.195   rec 0.769   3
```

1 Four contamination values

Each refits a model and uses predict(): −1 means ‘anomaly’.

2 0.5% → 118 alerts, precision 0.686

Clean alerts, but only 30% of fraud caught.

3 5% → 1,056 alerts, recall 0.769

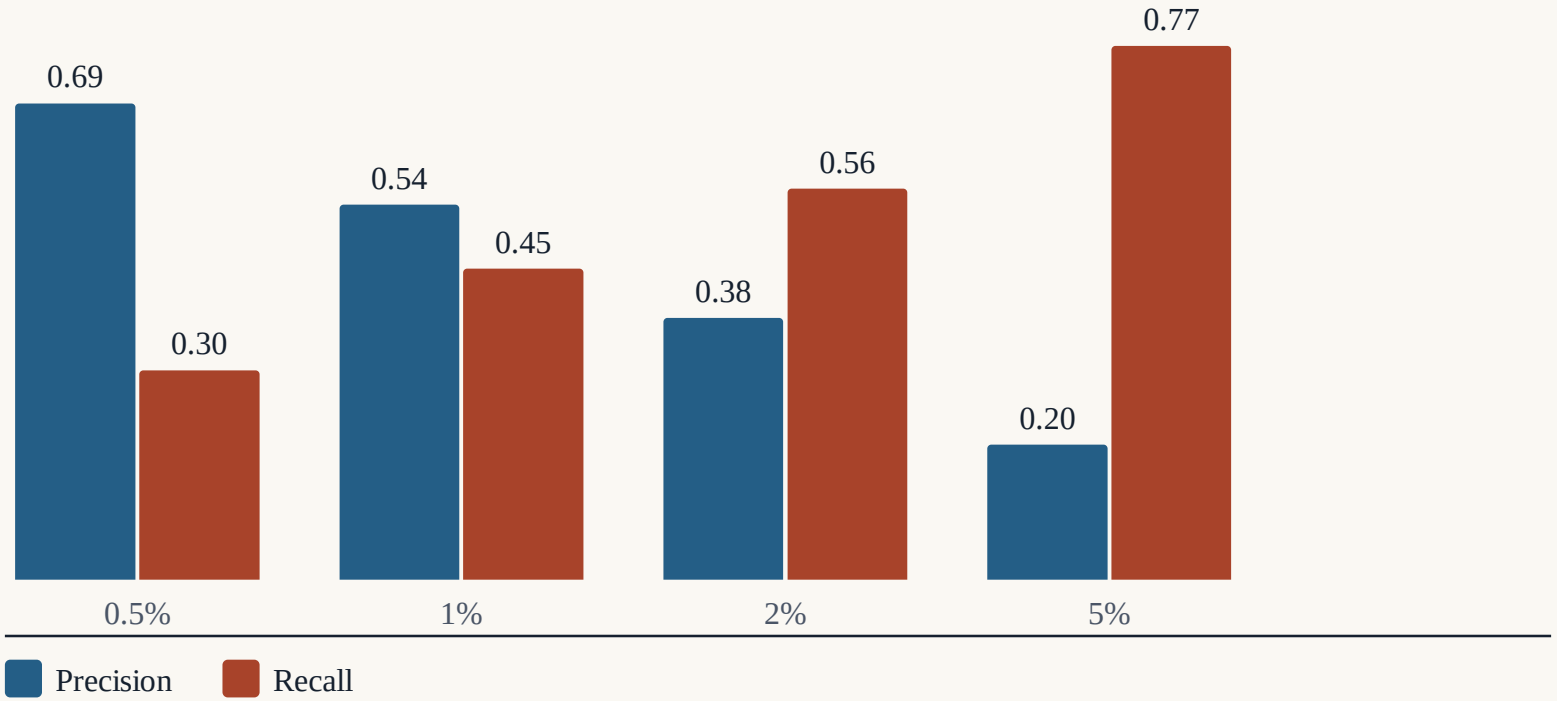
More fraud caught, but four in five alerts are false.

4 The scores never changed

Only the cut-off moved. Choose it from capacity and cost, not by guessing the fraud rate.

Precision falls as recall rises

Contamination setting → alerts flagged



WHAT THE OUTPUT SHOWS

Moving left to right we alert on more payments: recall climbs from 0.30 to 0.77 while precision drops from 0.69 to 0.20.

KEY INSIGHT

There is no free lunch: every detector traces a curve like this. The business chooses the point on it.

How many trees are enough?

PYTHON

```
for n in [10, 50, 100, 300]:
    m = IsolationForest(n_estimators=n, random_state=0)
    s = -m.fit(Xs_tr).score_samples(Xs_te)
    print("trees %3d  ROC-AUC %.3f  PR-AUC %.3f" % (
        n, roc_auc_score(y_te, s),
        average_precision_score(y_te, s)))
```

OUTPUT

```
trees  10  ROC-AUC 0.943  PR-AUC 0.313   1
trees  50  ROC-AUC 0.958  PR-AUC 0.449
trees 100  ROC-AUC 0.961  PR-AUC 0.477
trees 300  ROC-AUC 0.961  PR-AUC 0.495   2
```

1 Ten trees is noisy

PR-AUC 0.313: too few random cuts to average out luck.

2 Diminishing returns

100 trees gives 0.477; 300 gives 0.495. Beyond a few hundred, extra trees add time, not accuracy.

Sub-sample size per tree

PYTHON

```
for ms in [64, 256, 1024, 4096]:
    m = IsolationForest(n_estimators=200, max_samples=ms,
                        random_state=0)
    s = -m.fit(Xs_tr).score_samples(Xs_te)
    print("max_samples %4d  PR-AUC %.3f" % (
        ms, average_precision_score(y_te, s)))
```

3

OUTPUT

```
max_samples   64   PR-AUC 0.413
max_samples  256   PR-AUC 0.497
max_samples 1024   PR-AUC 0.529
max_samples 4096   PR-AUC 0.567
```

1 2

1 Bigger samples help here

PR-AUC rises from 0.413 at 64 rows to 0.567 at 4,096.

2 Why?

Card-testing bursts form small *clusters* of anomalies. Small samples cannot see that clusters are rare; larger samples can. This ‘masking’ effect is a known Isolation Forest weakness.

3 Treat 256 as a starting point

Tune max_samples on a validation period.

Do the merchant columns help Isolation Forest?

PYTHON

```
num_idx = [FEATS.index(c) for c in NUM]
m = IsolationForest(n_estimators=300, random_state=0)
s = -m.fit(Xs_tr[:, num_idx]).score_samples(Xs_te[:, num_idx])
print(evaluate("IForest numeric only", s))
```

OUTPUT

```
IForest numeric only
ROC-AUC 0.964   PR-AUC 0.515    P@300 0.477    R@300 0.534
```

1 Numeric features only

Drop the eight one-hot merchant columns.

2 Slightly better without them

PR-AUC 0.515 versus 0.495. Sparse 0/1 columns give the random cuts many unhelpful places to split.

3 Lesson

Feature selection matters even for unsupervised models. Test, do not assume.

Isolation Forest: strengths and weaknesses

Strengths

- Fast: linear in data size; trains in under a second here
- No distance calculations: copes with many features
- No scaling needed
- Strong default choice for tabular anomaly detection

Weaknesses

- Scores rank but are not probabilities
- Cuts are axis-parallel: diagonal patterns are harder
- Clusters of anomalies can mask each other
- Blind to anomalies that look normal (lost/stolen cards)

PRO TIP

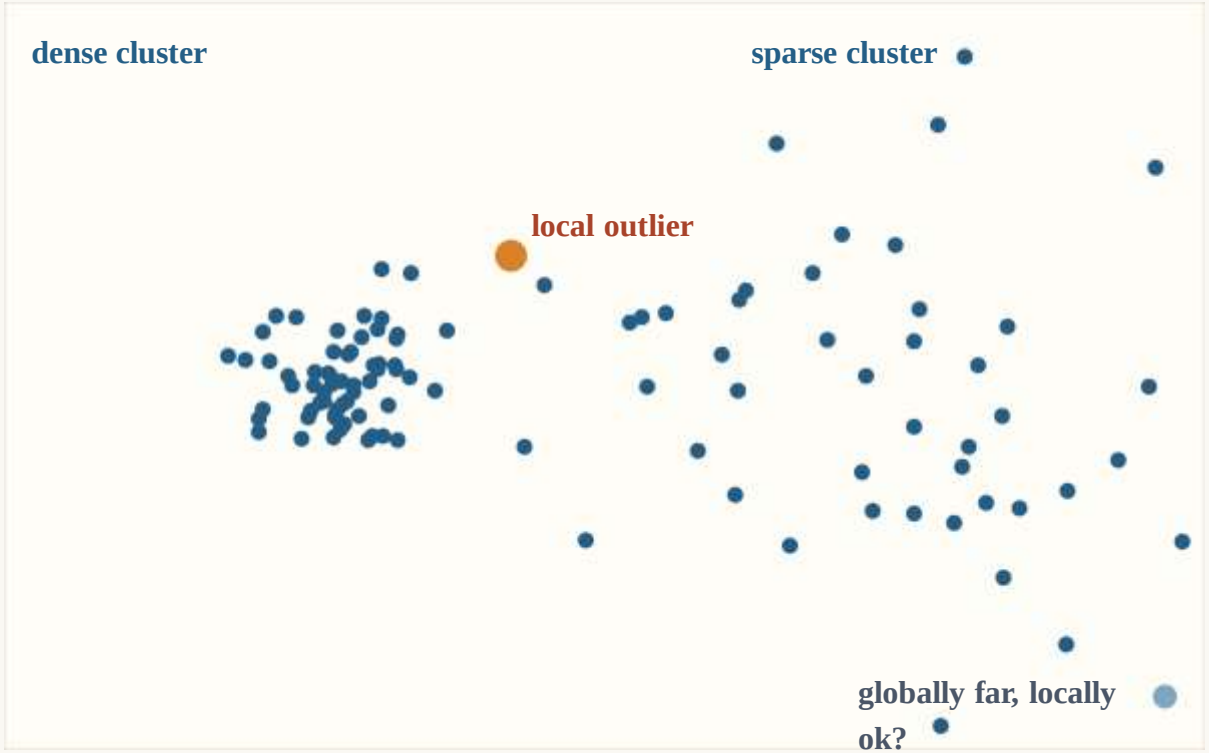
Extended Isolation Forest (random diagonal cuts) addresses the axis-parallel weakness; it lives in separate libraries.

Local Outlier Factor

A point is anomalous if its neighbourhood is much emptier than its neighbours' neighbourhoods. Breunig et al., 2000.

Density relative to the neighbourhood

Two legitimate behaviour groups of different tightness.



WHAT THE OUTPUT SHOWS

The orange point is close to the dense cluster in absolute terms — but its neighbourhood is far emptier than that cluster's. LOF flags it.

KEY INSIGHT

Points in the sparse cluster are far apart, yet so are their neighbours: they look normal *locally*.

PRO TIP

This is the 'local' in LOF: each point is judged against its own surroundings, not one global yardstick.

UNSUPERVISED MACHINE LEARNING

LOF in four steps, in words

1 Find neighbours

For each point, find its k nearest neighbours (k = n_neighbors).

2 Measure reach

How far must you travel to reach those neighbours?

3 Local density

Short reach = crowded = high density; long reach = sparse.

4 Compare

Divide the neighbours' typical density by the point's own. Around 1 = normal; well above 1 = outlier.

PRO TIP

novelty=True lets a fitted LOF score *new* points (our test month). The default (novelty=False) only scores the training data itself.

LOF on the fraud data

PYTHON

```
from sklearn.neighbors import LocalOutlierFactor
idx = np.random.default_rng(1).choice(len(Xs_tr), 15000,
                                      replace=False)
t = time.time()
lof = LocalOutlierFactor(n_neighbors=35, novelty=True)
lof.fit(Xs_tr[idx])
s_lof = -lof.score_samples(Xs_te)
print("fit+score %.1fs" % (time.time() - t))
print(evaluate("LOF (k=35)", s_lof))
```

OUTPUT

```
fit+score 1.2s
LOF (k=35)
ROC-AUC 0.876   PR-AUC 0.082    P@300 0.107    R@300 0.119
```

1 A 15,000-row sample

LOF compares every point with its neighbours, which becomes slow on large data; a sample keeps it practical.

2 k = 35, novelty mode

3 PR-AUC only 0.082

Far below Isolation Forest (0.495) — barely six times random.

4 ROC-AUC 0.876 looks fine

Another example of ROC-AUC flattering a weak detector on rare events.

The neighbourhood size k changes everything

PYTHON

```
for k in [5, 20, 50, 100]:
    m = LocalOutlierFactor(n_neighbors=k, novelty=True)
    s = -m.fit(Xs_tr[idx]).score_samples(Xs_te)
    print("k=%3d  ROC-AUC %.3f  PR-AUC %.3f" % (
        k, roc_auc_score(y_te, s),
        average_precision_score(y_te, s)))
```

OUTPUT

```
k=  5   ROC-AUC 0.732   PR-AUC 0.034   (1)
k= 20   ROC-AUC 0.834   PR-AUC 0.060
k= 50   ROC-AUC 0.888   PR-AUC 0.105
k=100   ROC-AUC 0.883   PR-AUC 0.147   (2)
```

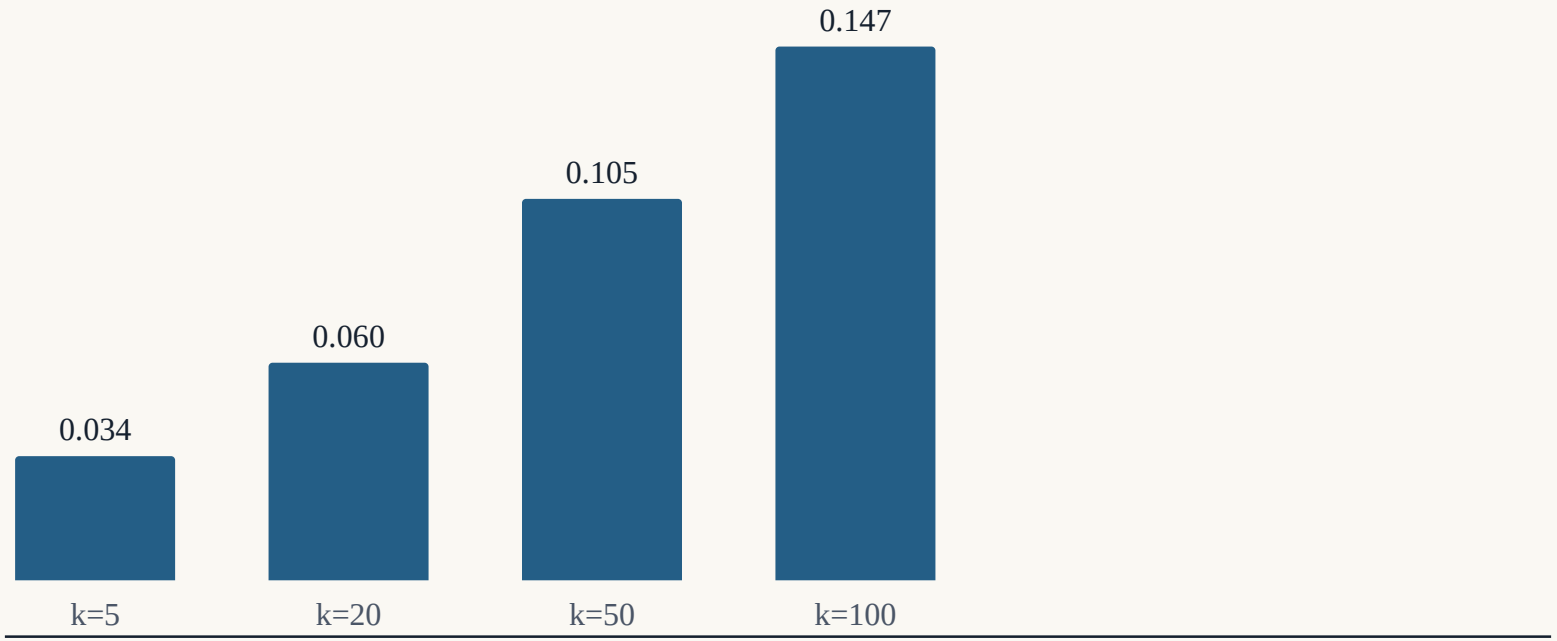
1 Small k is myopic

With 5 neighbours, a tight card-testing burst forms its *own* dense neighbourhood and looks normal.

2 Larger k sees the bigger picture

PR-AUC rises steadily to 0.147 at k = 100: the burst is now compared with the wider population.

LOF: PR-AUC by neighbourhood size



KEY INSIGHT

Clusters of anomalies need k larger than the cluster size, or they protect each other.

PRO TIP

Rule of thumb: start with k between 20 and 50 and validate; increase it when anomalies arrive in groups.

A surprising twist: LOF without scaling

PYTHON

```
m = LocalOutlierFactor(n_neighbors=35, novelty=True)
s = -m.fit(X_tr[idx]).score_samples(X_te)     # raw units!
print(evaluate("LOF unscaled", s))
```

1

OUTPUT

```
LOF unscaled
ROC-AUC 0.905   PR-AUC 0.203    P@300 0.310     R@300 0.347
```

2

1 Raw units

Same LOF, but on unscaled features.

2 PR-AUC jumps to 0.203

In raw units, hour (0–23) and the velocity counts dominate the distances — and those happen to be strong fraud signals here.

3 Not a recommendation

It is luck, not design. Scaling is still correct practice; the real lesson is that distance-based methods are extremely sensitive to which features dominate.

LOF: strengths and weaknesses

Strengths

- Finds *local* outliers in data with clusters of different densities
- Intuitive neighbour-based explanation
- Works well in low dimensions

Weaknesses

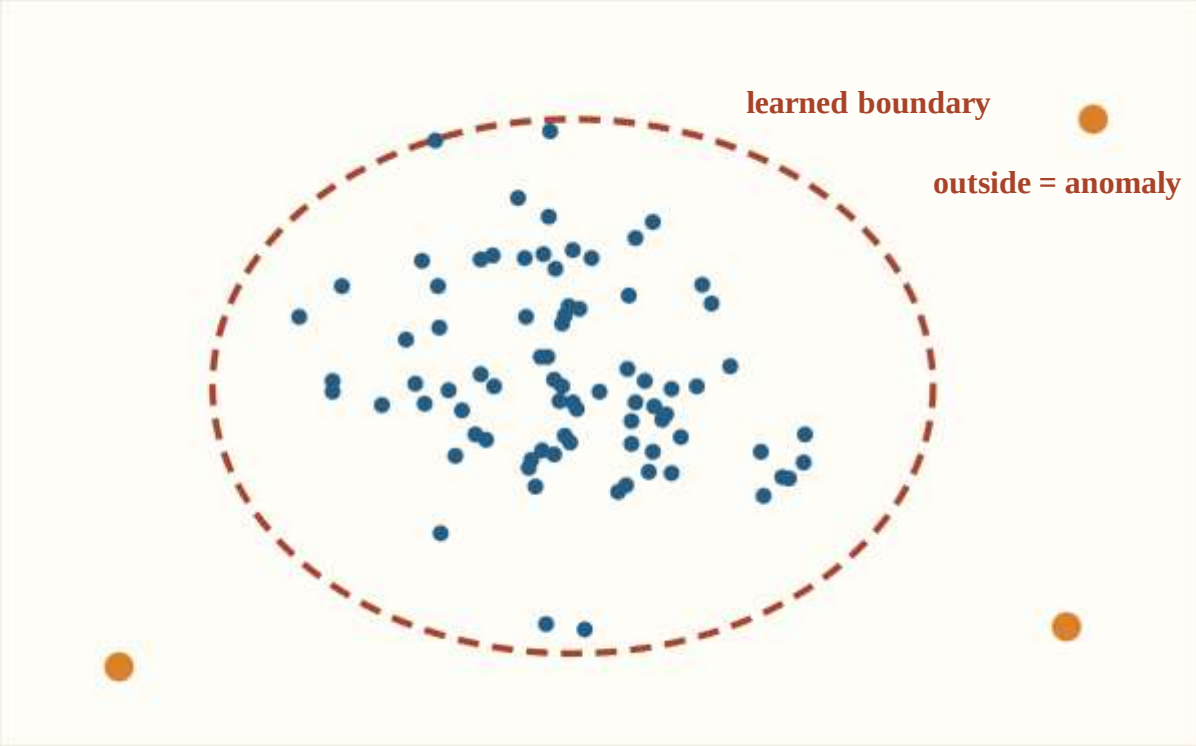
- Slow on large data (neighbour searches)
- Very sensitive to k and to feature scaling
- Distances lose meaning with many features
- Struggles with clustered anomalies

One-Class SVM

Learn a flexible boundary that wraps around normal data; whatever falls outside is anomalous. Schölkopf et al., 2001.

The intuition: draw a fence around normal

Shown in two dimensions; the real boundary lives in all 19 feature dimensions.



KEY INSIGHT

The algorithm finds the tightest smooth boundary that contains almost all training points.

PRO TIP

nu sets how many training points may fall outside (roughly an upper bound on that share).

PRO TIP

gamma sets how wiggly the boundary is: small = smooth, large = tightly hugging each cluster.

The kernel trick, without equations

A straight fence cannot wrap a curved shape. The trick is to judge closeness with a **similarity function** (the RBF kernel) instead of straight lines.

Similarity

Two payments are ‘similar’ when their features are close; similarity fades smoothly with distance.

Curved boundaries for free

Working with similarities lets the boundary bend around any shape of normal data.

Cost

Similarities between many pairs are expensive, so One-Class SVM is trained on a sample (5,000 rows here).

One-Class SVM on the fraud data

PYTHON

```
from sklearn.svm import OneClassSVM
sub = Xs_tr[idx[:5000]]                                          (1)
t = time.time()
oc = OneClassSVM(kernel="rbf", nu=0.01, gamma="scale")           (2)
oc.fit(sub)
s_oc = -oc.decision_function(Xs_te)
print("fit+score %.1fs" % (time.time() - t))
print(evaluate("One-Class SVM", s_oc))
```

OUTPUT

```
fit+score 0.1s
One-Class SVM
ROC-AUC 0.938   PR-AUC 0.267    P@300 0.353     R@300 0.396      (3)
```

1 5,000-row training sample

Training time grows quickly with rows.

2 RBF kernel, nu = 1%

gamma='scale' picks a sensible default from the data's spread.

3 PR-AUC 0.267

Better than LOF, well behind Isolation Forest. Fast here because of the sample.

Changing nu moves the fence

PYTHON

```
for nu in [0.001, 0.01, 0.05, 0.2]:
    m = OneClassSVM(nu=nu, gamma="scale").fit(sub)
    pred = m.predict(Xs_te) == -1
    print("nu %.3f  alerts %5d  prec %.3f  rec %.3f" % (
        nu, pred.sum(), y_te[pred].mean(),
        y_te[pred].sum() / y_te.sum()))
```

OUTPUT

```
nu 0.001    alerts   678   prec 0.229   rec 0.578   (1)
nu 0.010    alerts   679   prec 0.228   rec 0.578
nu 0.050    alerts  1296   prec 0.184   rec 0.892
nu 0.200    alerts  4245   prec 0.062   rec 0.978   (2)
```

1 nu 0.001 and 0.01 behave alike

About 680 alerts: the boundary is already as tight as the sample allows.

2 nu 0.2 → 4,245 alerts

Recall 98%, precision 6%: a fence drawn too tight around the crowd.

gamma: smooth versus wiggly boundaries

PYTHON

```
for gm in [0.005, 0.05, 0.5, 5.0]:
    m = OneClassSVM(nu=0.01, gamma=gm).fit(sub)
    s = -m.decision_function(Xs_te)
    print("gamma %5.3f  PR-AUC %.3f" % (
        gm, average_precision_score(y_te, s)))
```

OUTPUT

```
gamma 0.005    PR-AUC 0.122   1
gamma 0.050    PR-AUC 0.241
gamma 0.500    PR-AUC 0.387   2
gamma 5.000    PR-AUC 0.380   3
```

1 Too smooth

gamma 0.005 → PR-AUC 0.122: the fence is a loose bubble.

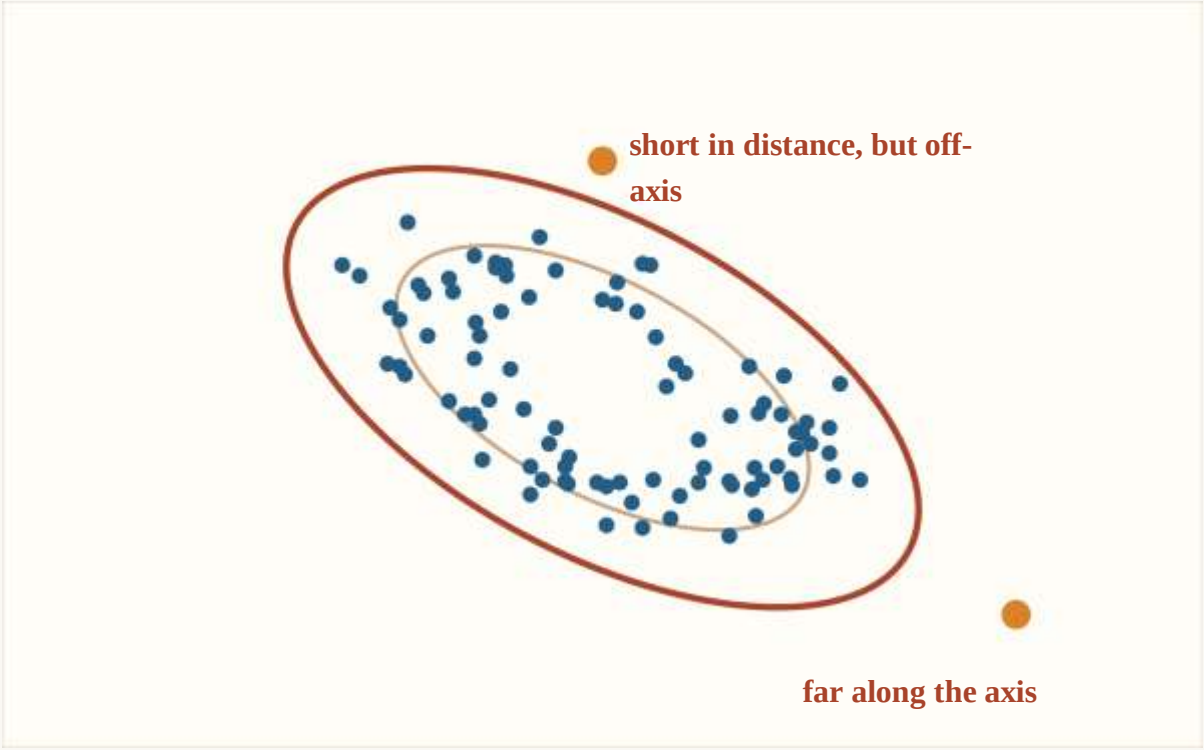
2 Better fit

gamma 0.5 → 0.387, a big jump from the default. Hyperparameters matter enormously for SVMs.

3 Validate on a later period

Tuning on the test month (as this demo does) overstates results; use a separate validation window in practice.

Elliptic Envelope: one tilted oval



KEY INSIGHT

Assumes normal data forms one bell-shaped cloud. It learns the cloud's centre, stretch and tilt — robustly, ignoring a few extreme points.

WHAT THE OUTPUT SHOWS

A point *across* the tilt can be anomalous even when close to the centre; distance is measured relative to the cloud's shape.

WATCH OUT

Breaks down when normal data has several groups or heavy skew.

Elliptic Envelope on six continuous features

PYTHON

```
from sklearn.covariance import EllipticEnvelope
cont = [FEATS.index(c) for c in ["log_amount", "amount_ratio",
        "log_dist", "tx_1h", "tx_24h", "log_gap"]]
ee = EllipticEnvelope(contamination=0.01, random_state=0)
ee.fit(Xs_tr[:, cont])
s_ee = -ee.score_samples(Xs_te[:, cont])
print(evaluate("Elliptic Envelope", s_ee))
```

OUTPUT

```
Elliptic Envelope
ROC-AUC 0.955   PR-AUC 0.182    P@300 0.223     R@300 0.250
```

1 Continuous features only

0/1 flags and one-hot columns break the single-oval assumption, so we use six continuous features.

2 Robust fit

Estimates centre and shape while ignoring the most extreme 1%.

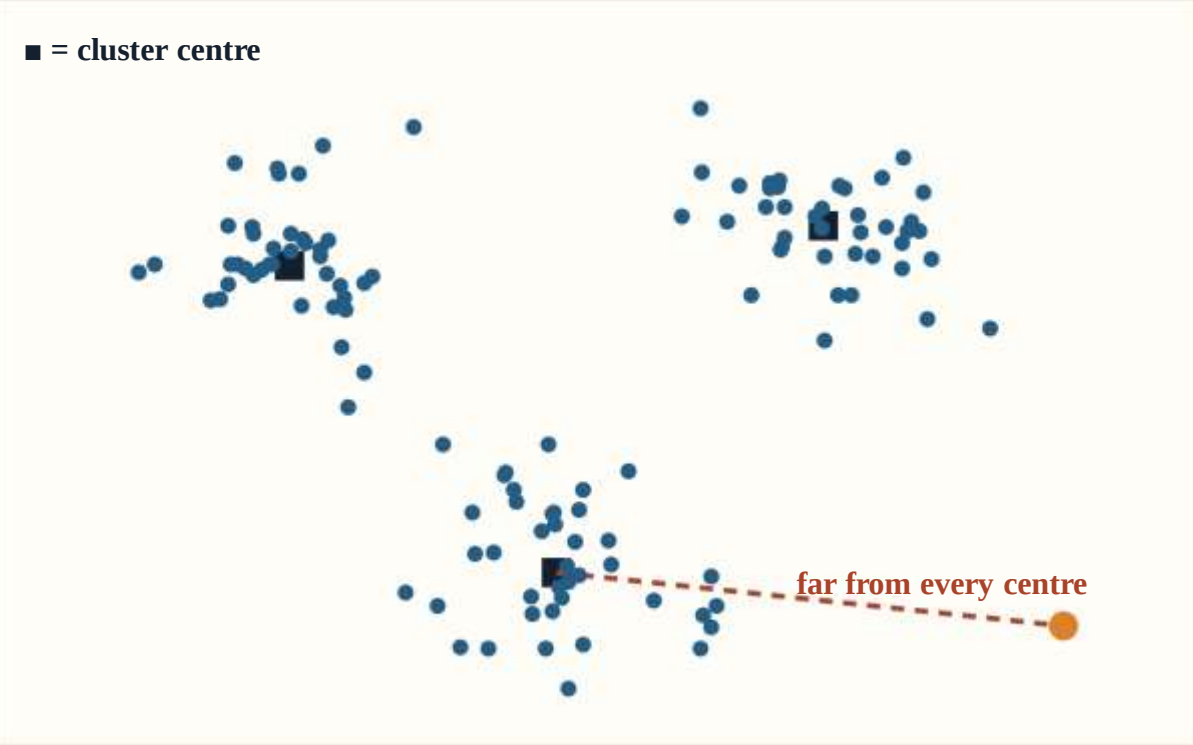
3 PR-AUC 0.182

Spending behaviour is skewed and multi-group, not one oval — so the method underperforms.

Clustering-based detection

Group normal behaviour into clusters; anomalies belong to none of them comfortably.

k-means: distance to the nearest centre



KEY INSIGHT

k-means places k centres so that each point is close to one of them.

WHAT THE OUTPUT SHOWS

Anomaly score = distance to the nearest centre. Points that fit no cluster are far from all of them.

WATCH OUT

Tiny clusters can capture groups of anomalies — inspect cluster sizes.

k-means distance on the fraud data

PYTHON

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=12, n_init=5, random_state=0)
km.fit(Xs_tr)
s_km = km.transform(Xs_te).min(axis=1)   # dist to nearest centre
print(evaluate("K-Means distance", s_km))
print("cluster sizes:", np.bincount(km.labels_))
```

OUTPUT

```
K-Means distance
ROC-AUC 0.969  PR-AUC 0.380  P@300 0.453  R@300 0.507
cluster sizes: [2732 1025 7555 1371  344  500 7787 4265 7846   76 7137  681]
```

1 12 clusters

Roughly one per broad behaviour pattern.

2 Distance to nearest centre

transform() gives distances to all 12 centres; we keep the smallest.

3 PR-AUC 0.380

Second-best unsupervised method so far — simple methods can be strong.

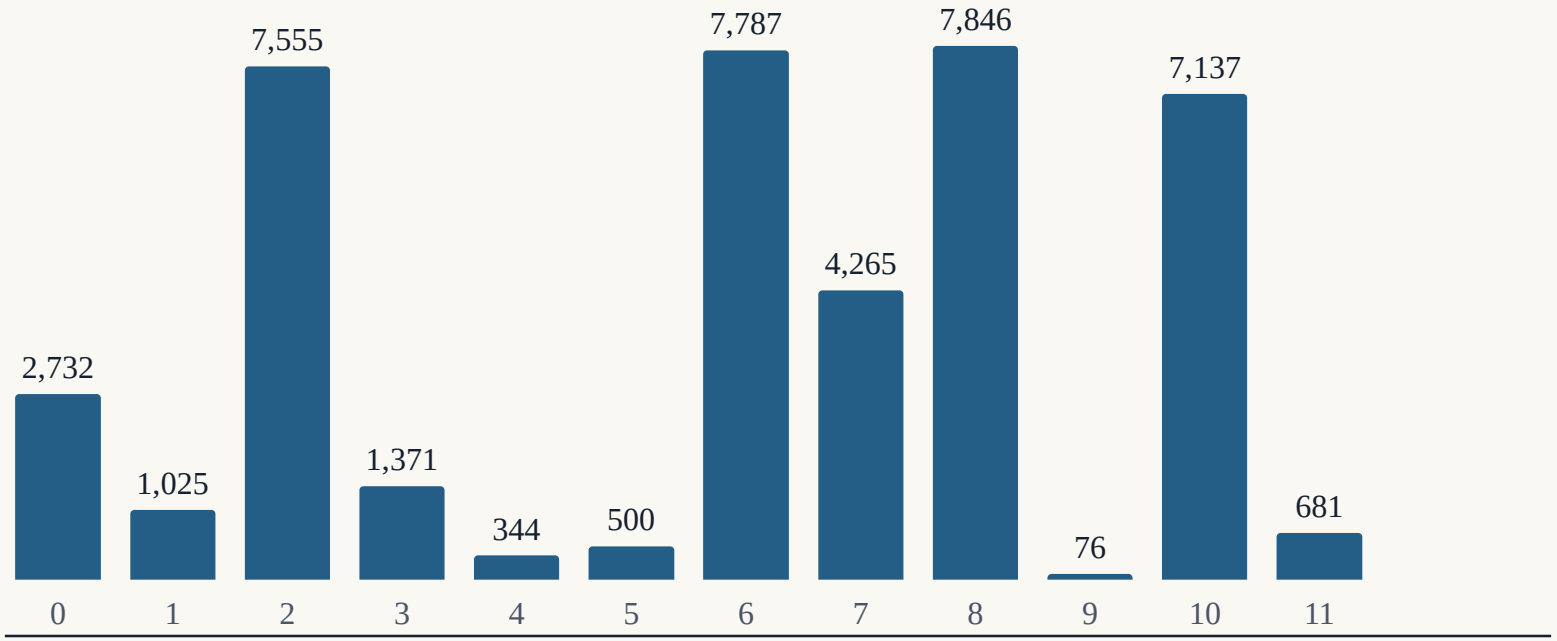
4 A cluster of 76

A tiny cluster: worth checking whether it is a pocket of anomalies being treated as ‘normal’.

UNSUPERVISED MACHINE LEARNING

Cluster sizes from the k-means model

Training rows per cluster

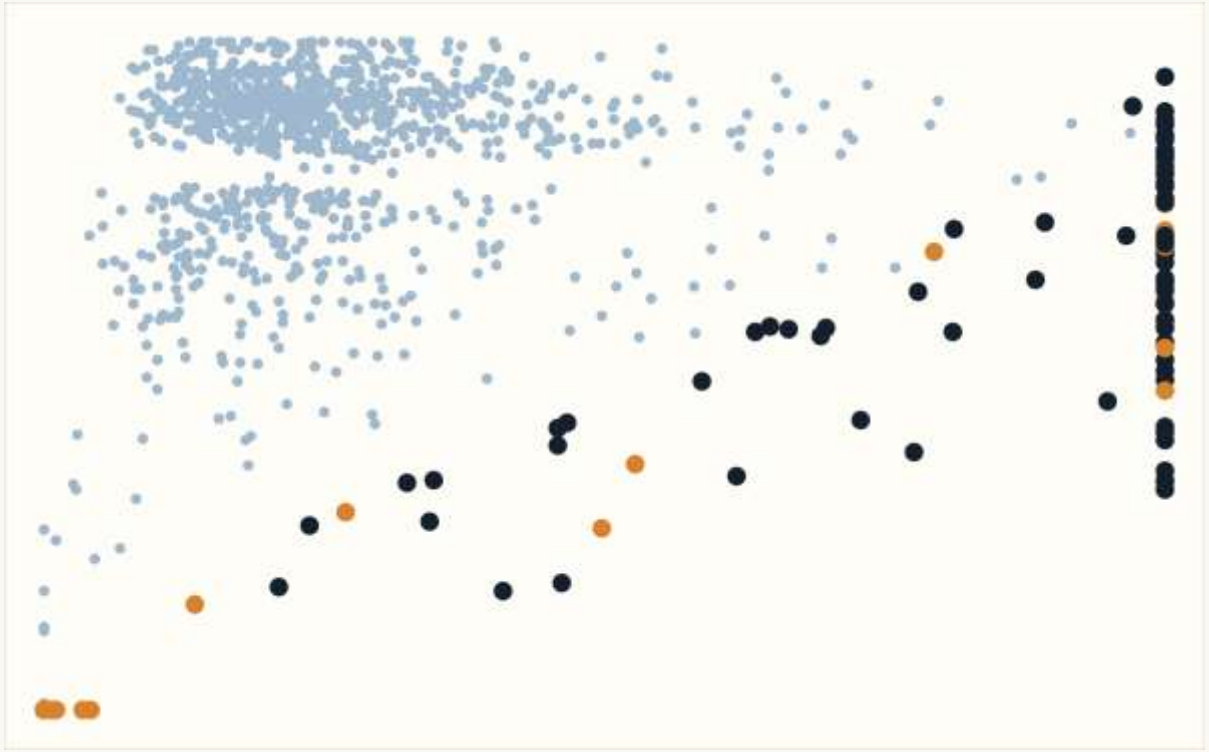


KEY INSIGHT

Three clusters hold 500 rows or fewer. In fraud work, small clusters deserve a look: they may be niche legitimate behaviour — or organised attacks.

DBSCAN: what does not belong to any dense region?

Every dot is a real test payment from our data.



KEY INSIGHT

DBSCAN grows clusters from points with at least min_samples neighbours within distance eps. Points reachable from no cluster are **noise**.

WHAT THE OUTPUT SHOWS

Real output, plotted on the first two principal components of 4,000 test payments.

DBSCAN on a 2-D projection

PYTHON

```
from sklearn.decomposition import PCA
from sklearn.cluster import DBSCAN
samp = te.sample(4000, random_state=3)
Z = PCA(n_components=2, random_state=0).fit(Xs_tr) \
      .transform(scaler.transform(samp[FEATS].values))
db = DBSCAN(eps=0.6, min_samples=15).fit(Z)
noise = db.labels_ == -1
print("clusters:", len(set(db.labels_)) - 1,
     "| noise points:", int(noise.sum()))
print("fraud rate in noise : %.3f" % samp.is_fraud[noise].mean())
print("fraud rate overall  : %.3f" % samp.is_fraud.mean())
```

OUTPUT

```
clusters: 1 | noise points: 104
fraud rate in noise : 0.202
fraud rate overall          : 0.015
```

1 Project to two dimensions

PCA squeezes 19 features into 2 so density is meaningful and the result can be plotted.

2 eps and min_samples

The neighbourhood radius and the crowd size needed to form a cluster.

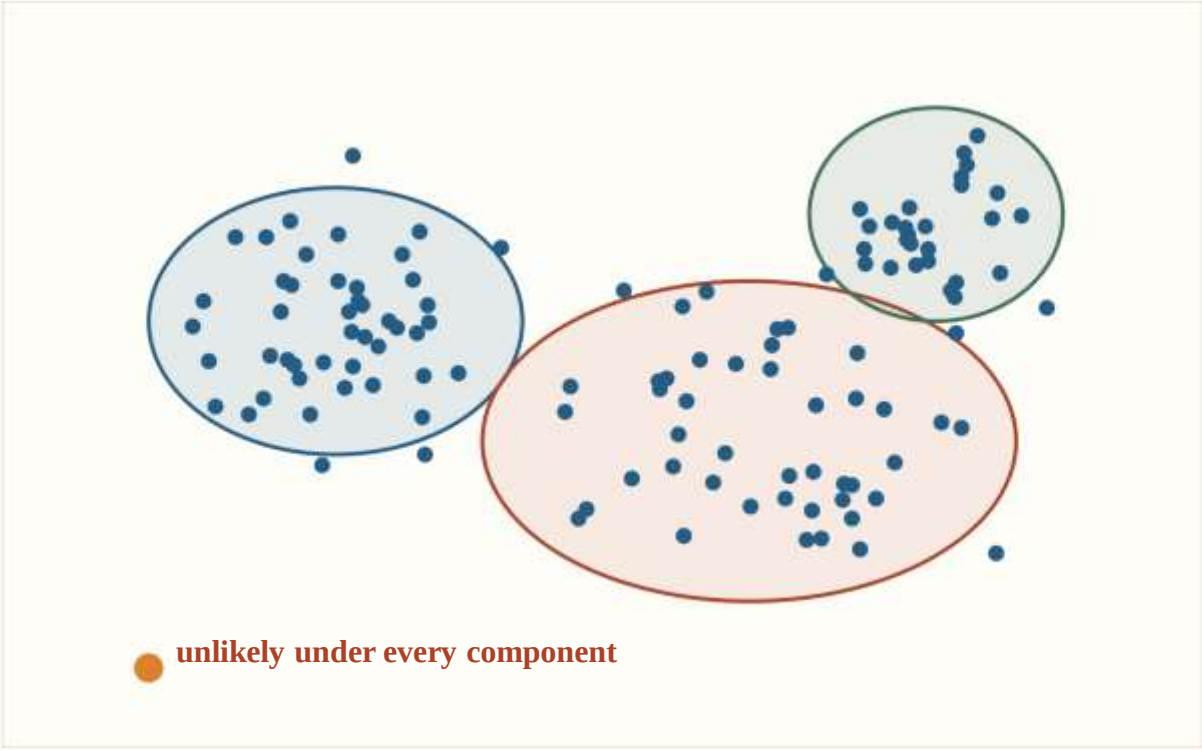
3 104 noise points

One big cluster and 104 points that fit nowhere.

4 20% of noise is fraud

Thirteen times the base rate of 1.5% — a useful but blunt filter.

Gaussian mixture models: overlapping soft clusters



KEY INSIGHT

A mixture describes normal data as several overlapping bell-shaped clouds of different sizes and tilts.

WHAT THE OUTPUT SHOWS

Each point gets a **likelihood**: how plausible it is under the whole mixture. Low likelihood = anomaly.

PRO TIP

Unlike k-means, membership is soft: a point can belong 70% to one cloud and 30% to another.

A Gaussian mixture on the fraud data

PYTHON

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=8, covariance_type="full",
                      random_state=0, reg_covar=1e-3)
gmm.fit(Xs_tr[:, cont])
s_gmm = -gmm.score_samples(Xs_te[:, cont])  # low likelihood
print(evaluate("Gaussian Mixture", s_gmm))
print("BIC: %.0f" % gmm.bic(Xs_tr[:, cont]))
```

OUTPUT

```
Gaussian Mixture
ROC-AUC 0.958   PR-AUC 0.211    P@300 0.277    R@300 0.310
BIC: 5125
```

1 Eight full-shape components

reg_covar adds a little stability so components cannot collapse onto a few identical points.

2 Negative log-likelihood

We flip the sign so that *less likely* means a *higher* score.

3 PR-AUC 0.211

Reasonable, but the bell-shape assumption struggles with skewed counts.

How many components? Ask the BIC

PYTHON

```
for k in [1, 2, 4, 8, 12]:
    g_ = GaussianMixture(k, random_state=0, reg_covar=1e-3)
    g_.fit(Xs_tr[:, cont])
    print("components %2d  BIC %9.0f" % (k,
          g_.bic(Xs_tr[:, cont])))
```

1

OUTPUT

```
components  1   BIC    574200
components  2   BIC    301487
components  4   BIC    210343
components  8   BIC      5125   3
components 12   BIC      -356   2
```

1 BIC: fit penalised by complexity

The Bayesian Information Criterion rewards fitting the data and punishes extra parameters. Lower is better.

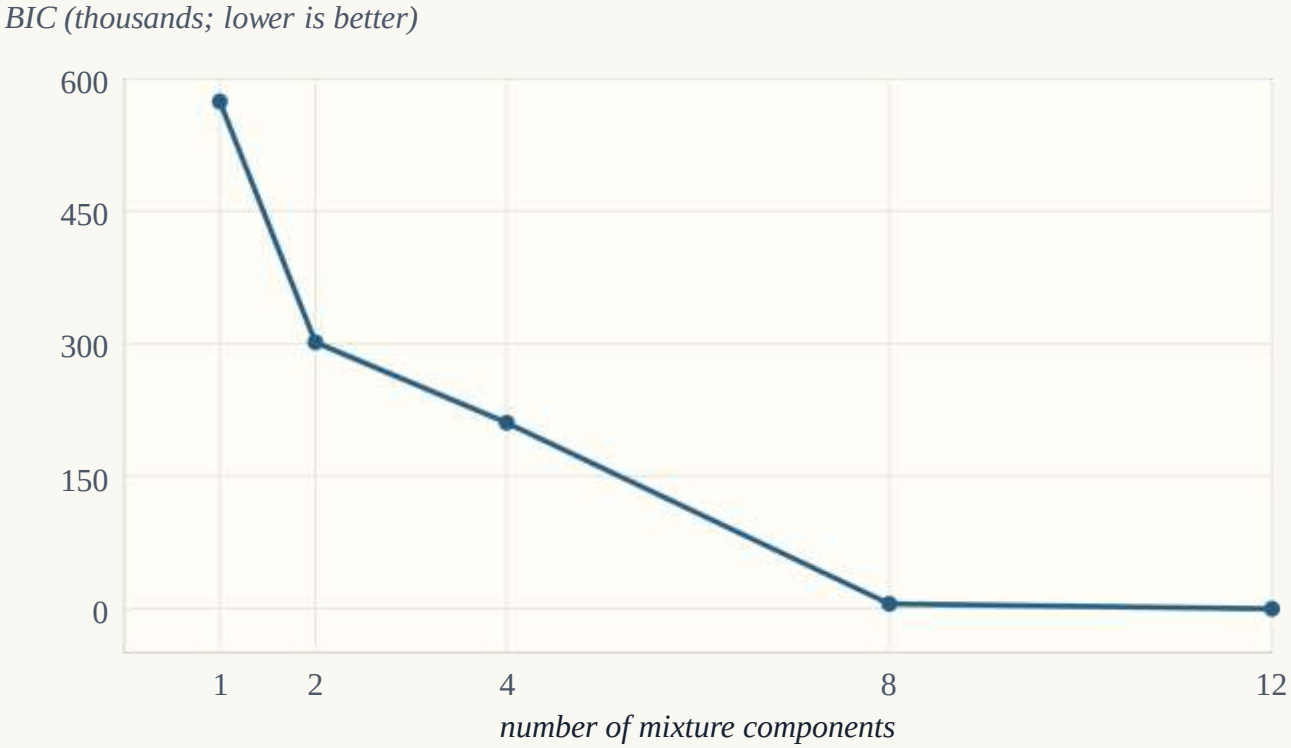
2 Still falling at 12

The data has many sub-populations (night owls, travellers, bursts…), so more components keep paying off.

3 Lower BIC ≠ better detection

BIC measures how well normal data is described, not how well fraud is found. Validate both.

BIC by number of components



WHAT THE OUTPUT SHOWS

The steep early drop shows one or two clouds cannot describe spending behaviour.

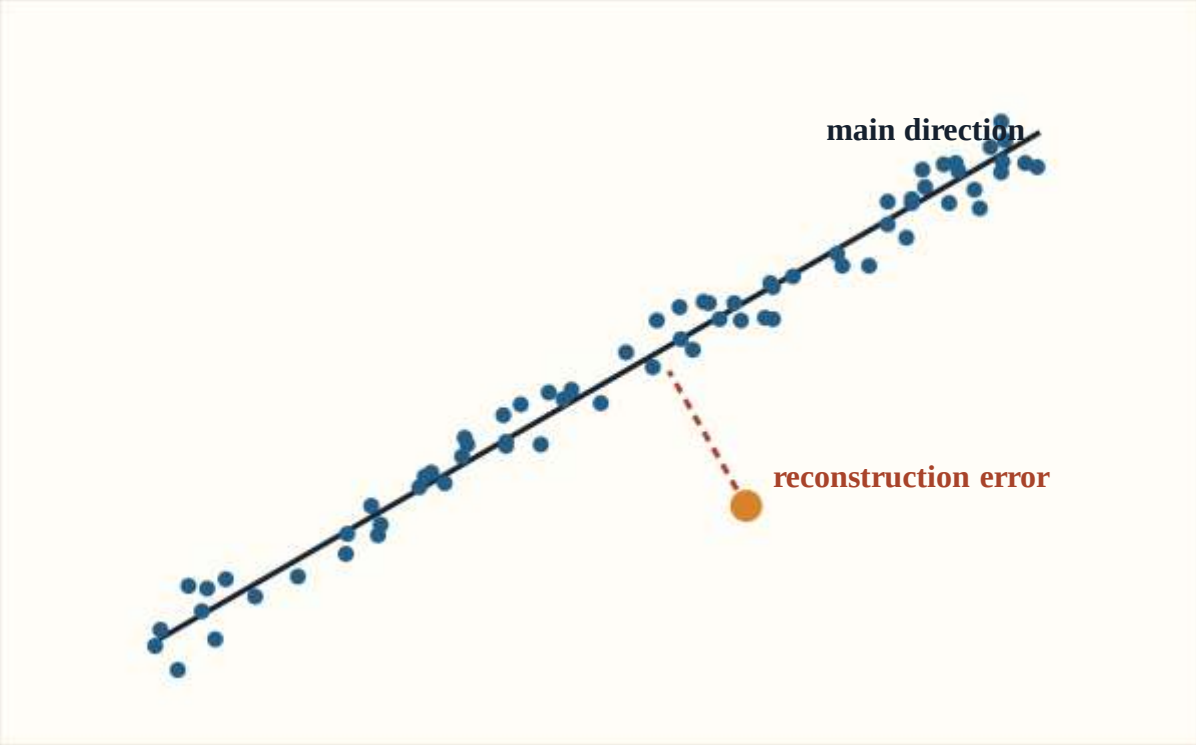
KEY INSIGHT

The curve flattens after eight: extra components add little description.

Reconstruction-based detection

Compress the data, rebuild it, and measure what was lost. Normal data rebuilds well; anomalies do not.

PCA: keep the main directions, measure the leftovers



KEY INSIGHT

Principal component analysis finds the directions along which normal data varies most.

WHAT THE OUTPUT SHOWS

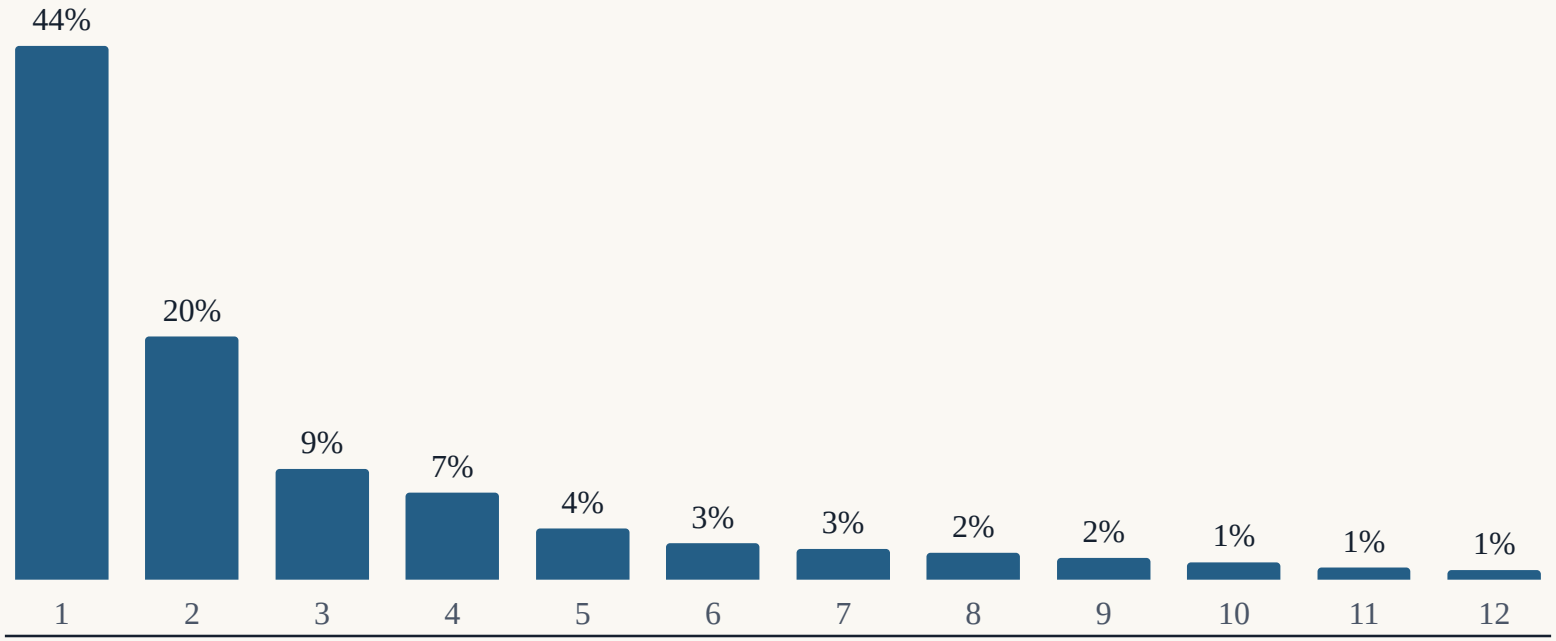
Squash each point onto those directions and rebuild it. Normal points barely move; the outlier’s rebuilt version lands far from where it really was.

PRO TIP

That gap — the **reconstruction error** — is the anomaly score.

How much information do the components hold?

Variance explained by each component (%)



WHAT THE OUTPUT SHOWS

The first component captures 44% of the variation; the first six together hold 88%.

KEY INSIGHT

A ‘scree plot’ like this guides how many components to keep.

PCA reconstruction error

PYTHON

```
pca = PCA(n_components=6, random_state=0).fit(Xs_tr)                    1
print("variance kept by 6 comps: %.3f"
      % pca.explained_variance_ratio_.sum())
recon = pca.inverse_transform(pca.transform(Xs_te))                     2
s_pca = ((Xs_te - recon) ** 2).sum(axis=1)                              3
print(evaluate("PCA reconstruction", s_pca))
```

OUTPUT

```
variance kept by 6 comps: 0.880
PCA reconstruction
ROC-AUC 0.874   PR-AUC 0.214    P@300 0.260     R@300 0.291             4
```

1 Keep six components

2 Compress, then rebuild

transform() squashes; inverse_transform() rebuilds in the original 19 dimensions.

3 Score = squared leftovers

Summed across features.

4 PR-AUC 0.214

Respectable, below Isolation Forest.

Fewer components can detect better

PYTHON

```
for k in [2, 4, 6, 10, 14]:
    p = PCA(n_components=k, random_state=0).fit(Xs_tr)
    r = p.inverse_transform(p.transform(Xs_te))
    s = ((Xs_te - r) ** 2).sum(axis=1)
    print("k=%2d  var %.2f  PR-AUC %.3f" % (k,
          p.explained_variance_ratio_.sum(),
          average_precision_score(y_te, s)))
```

OUTPUT

```
k= 2    var 0.64    PR-AUC 0.340   1
k= 4    var 0.81    PR-AUC 0.201
k= 6    var 0.88    PR-AUC 0.214
k=10    var 0.96    PR-AUC 0.083   3
k=14    var 0.99    PR-AUC 0.107   2
```

1 Two components: PR-AUC 0.340

The best setting tested.

2 Fourteen components: 0.107

Keeping 99% of the variation lets the model rebuild anomalies too — so they stop standing out.

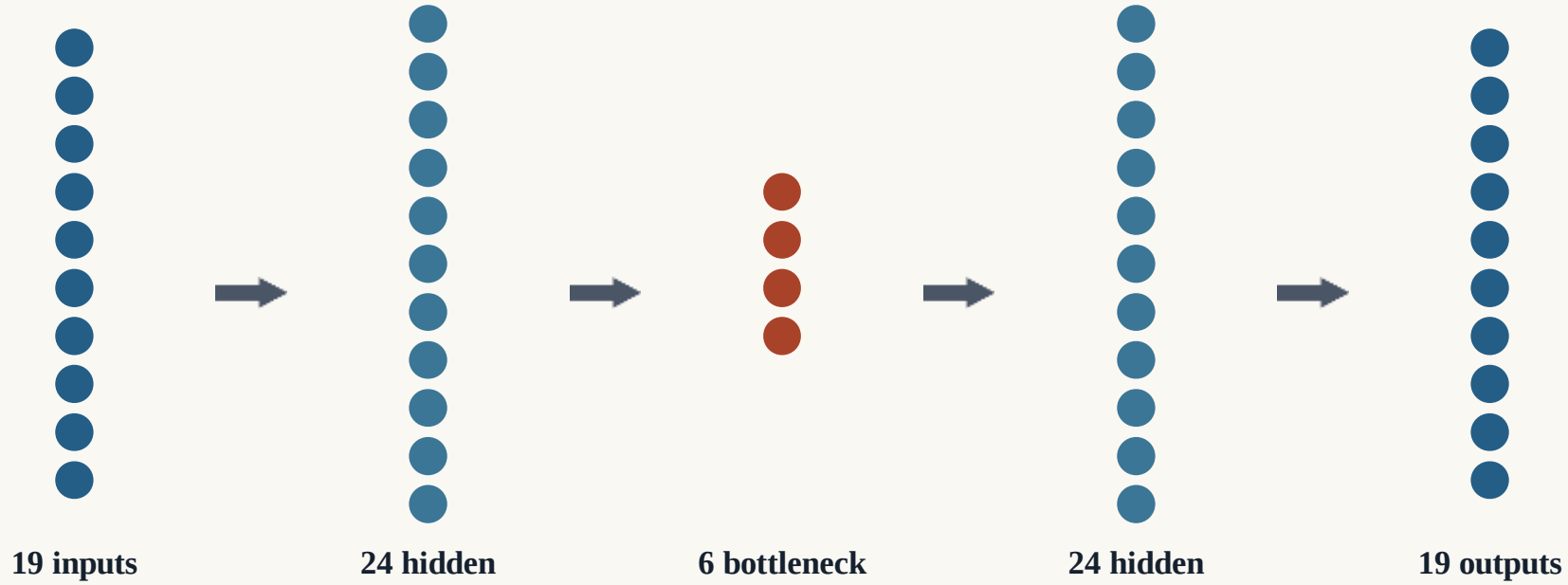
3 The counter-intuitive rule

For detection, a model that compresses *harder* often works better. Do not choose k by variance alone.

UNSUPERVISED MACHINE LEARNING

Autoencoders: neural compression

A neural network trained to reproduce its own input through a narrow bottleneck.



KEY INSIGHT

To squeeze 19 features through 6 numbers and rebuild them, the network must learn the *regularities* of normal payments. Unusual payments violate those regularities, so they are rebuilt badly — and their reconstruction error is high.

An autoencoder with scikit-learn

PYTHON

```
from sklearn.neural_network import MLPRegressor
mm = StandardScaler().fit(X_tr)                                    1
A_tr, A_te = mm.transform(X_tr), mm.transform(X_te)
t = time.time()
ae = MLPRegressor(hidden_layer_sizes=(24, 6, 24),                  2
                  activation="relu", max_iter=60,
                  random_state=0, early_stopping=True)
ae.fit(A_tr, A_tr)                  # input == target              3
s_ae = ((A_te - ae.predict(A_te)) ** 2).mean(axis=1)               4
print("train %.1fs, epochs %d" % (time.time() - t, ae.n_iter_))
print(evaluate("Autoencoder (MLP)", s_ae))
```

OUTPUT

```
train 4.7s, epochs 60
Autoencoder (MLP)
ROC-AUC 0.953   PR-AUC 0.316    P@300 0.363    R@300 0.407          5
```

1 Standard scaling

Neural networks train best on features centred around zero with similar spreads.

2 24 → 6 → 24 hidden units

The 6-unit middle layer is the bottleneck.

3 Input equals target

The network learns to copy its input — the whole trick.

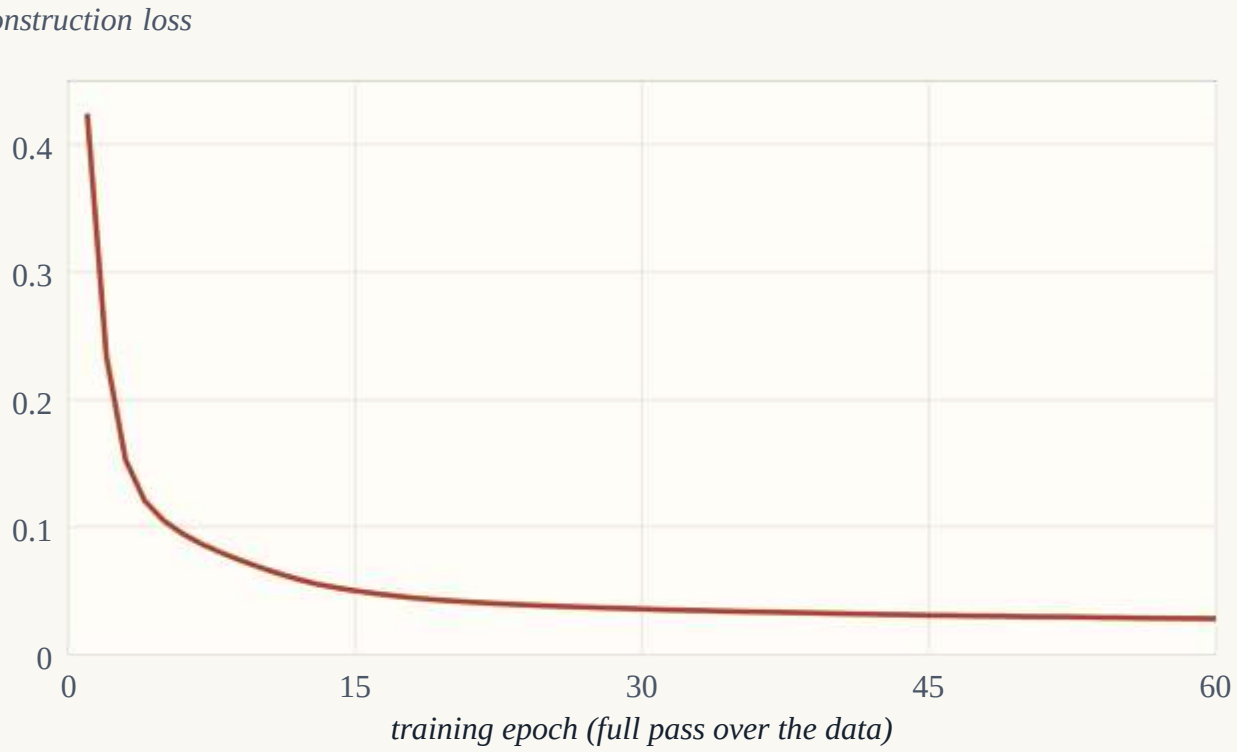
4 Score = average squared error

Per payment, across all 19 features.

5 PR-AUC 0.316

Third-best unsupervised method, behind Isolation Forest and k-means.

Training curve: the network learns normal



WHAT THE OUTPUT SHOWS

The loss drops fast in the first ten epochs, then flattens near 0.03: the network has captured most regular structure.

WATCH OUT

If loss kept falling to near zero, the network might learn to copy *anything* — including anomalies. The bottleneck prevents that.

Which schemes does the autoencoder find strange?

PYTHON

```
err = pd.Series(s_ae).groupby(te.scheme.values).mean()
print("Mean reconstruction error by scheme:")
print(err.sort_values(ascending=False).round(3).to_string())
```

OUTPUT

```
Mean reconstruction error by scheme:
card_testing      1.077
account_takeover  0.637
atm_cash_out      0.506
cnp_foreign       0.465
lost_stolen       0.203
legit             0.053
```

1 **Card testing is rebuilt worst**

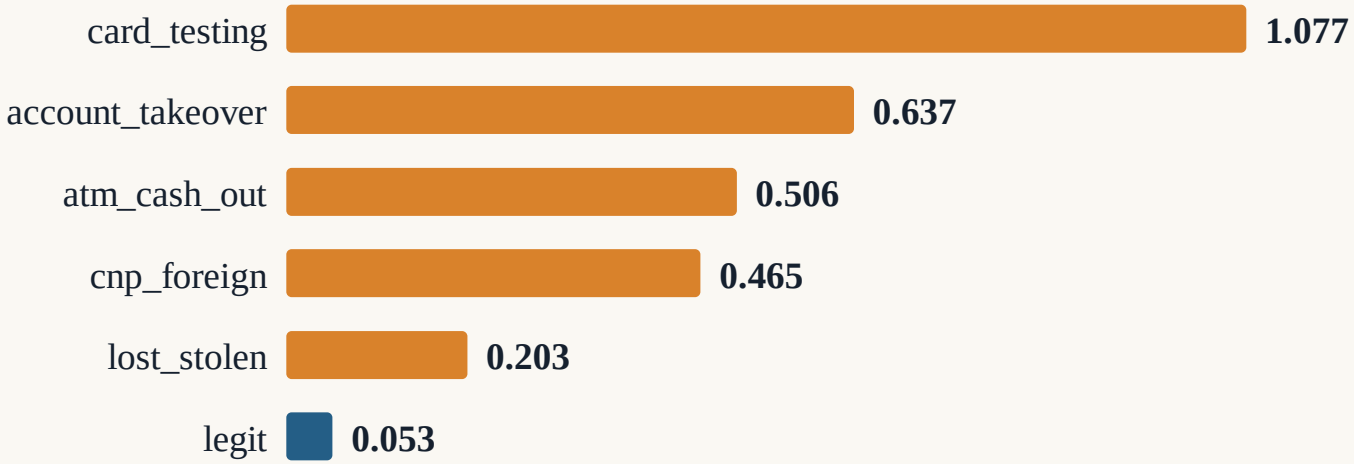
Error 1.08 — twenty times the legitimate average.

2 **Lost/stolen is rebuilt almost perfectly**

Error 0.20, close to normal payments. Once again, a fraud that copies normal behaviour is invisible to unsupervised learning.

3 **Legitimate baseline: 0.053**

Reconstruction error by scheme



WHAT THE OUTPUT SHOWS

Every fraud scheme is rebuilt worse than legitimate behaviour — but by very different margins.

PRO TIP

Grouping errors by known categories is a powerful diagnostic for any unsupervised model.

Reconstruction methods: strengths and weaknesses

PCA

- Fast, deterministic, explainable per feature
- Linear: misses curved patterns
- Choose components for detection, not variance

Autoencoder

- Captures non-linear patterns
- Scales to images, sequences, logs (with the right architecture)
- Needs scaling, tuning and more data; less explainable
- Per-feature errors give a natural explanation

Head-to-head: every unsupervised method

PYTHON

```
cmp = pd.DataFrame(RES).drop_duplicates("model")
cmp = cmp.sort_values("pr_auc", ascending=False)
print(cmp[["model", "roc_auc", "pr_auc", "prec_at_k"]]
      .to_string(index=False))
```

OUTPUT

model	roc_auc	pr_auc	prec_at_k	
IsolationForest	0.961	0.495	0.457	1
K-Means distance	0.969	0.380	0.453	2
Autoencoder (MLP)	0.953	0.316	0.363	
One-Class SVM	0.938	0.267	0.353	
velocity (tx_1h)	0.857	0.236	0.260	
PCA reconstruction	0.874	0.214	0.260	
Gaussian Mixture	0.958	0.211	0.277	
Elliptic Envelope	0.955	0.182	0.223	
LOF (k=35)	0.876	0.082	0.107	3
z-score amount	0.552	0.057	0.133	

4

1 Isolation Forest wins

Best PR-AUC (0.495) and top precision@300.

2 Simple k-means is second

3 LOF last among ML methods

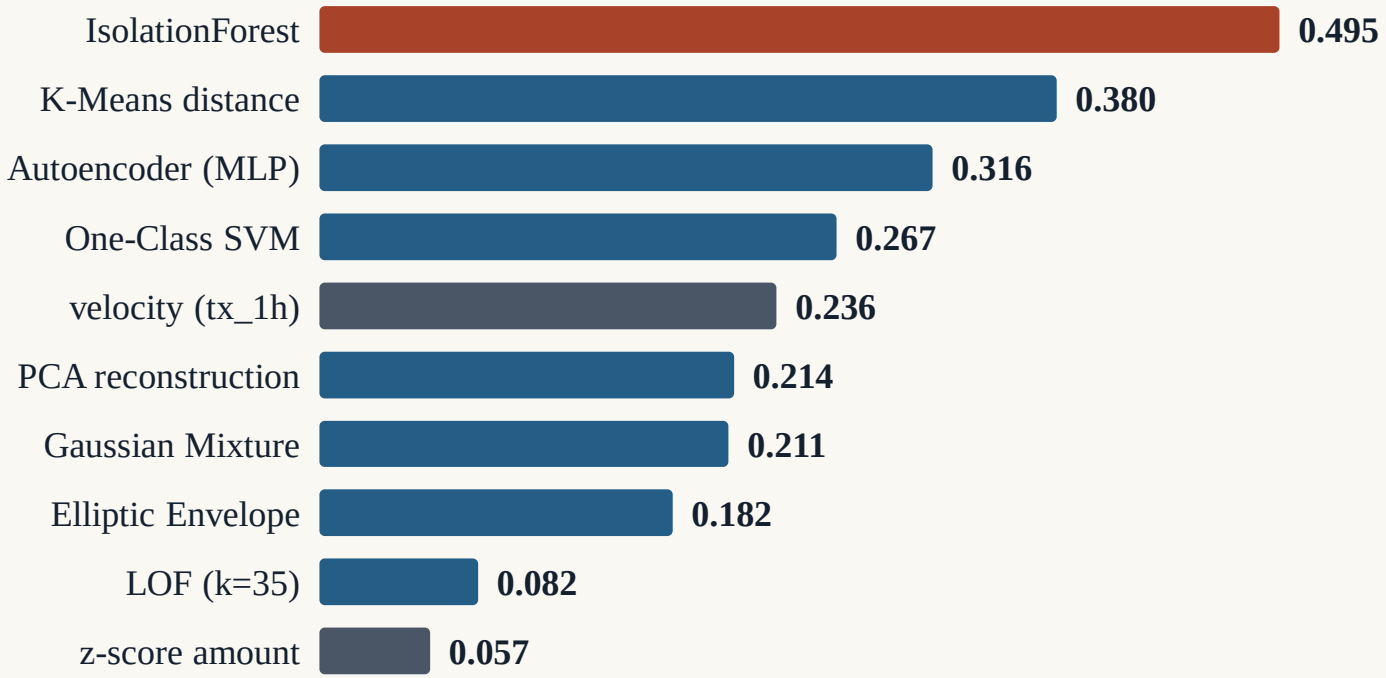
Despite a ROC-AUC of 0.876.

4 ROC-AUC barely separates them

Most sit between 0.94 and 0.97. PR-AUC reveals a 6× spread.

Unsupervised leaderboard (PR-AUC)

Test month; higher is better.



PRO TIP

Grey bars are the statistical baselines from Part 8, for reference.

KEY INSIGHT

Random scores 0.013. Every ML method beats random; only three beat a simple velocity count convincingly.

Choosing an unsupervised method

Tabular data, many features, large volume	Isolation Forest	fast, robust, few settings
Several normal groups of different tightness	LOF (tuned k)	judges each point locally
Small, clean training set; smooth normal region	One-Class SVM	flexible boundary
One compact, roughly bell-shaped normal group	Elliptic Envelope	simple and robust
Want interpretable behaviour segments too	k-means / GMM	clusters double as segments
Images, sequences, text-like logs	Autoencoder	learns complex structure
Need per-feature explanations, fast	PCA reconstruction	errors break down by feature

Ensembles: combining detectors

Different methods make different mistakes, so combining them *can* be more robust.

Rank averaging

Convert each model's scores to ranks (0 to 1) so scales are comparable, then average.

Maximum rank

Alert if *any* detector is very confident — favours recall.

Weighted voting

Give stronger detectors more weight, chosen on a validation period.

Does a simple ensemble help?

PYTHON

```
from scipy.stats import rankdata
parts = [s_iso, s_lof, s_ae, s_pca]                                  1
rank_avg = np.mean([rankdata(p) / len(p) for p in parts], axis=0)
print(evaluate("Rank-average ensemble", rank_avg))
print(evaluate("IsolationForest alone", s_iso))
```

OUTPUT

```
Rank-average ensemble
ROC-AUC 0.957   PR-AUC 0.326   P@300 0.383   R@300 0.429              2
IsolationForest alone
ROC-AUC 0.961   PR-AUC 0.495   P@300 0.457   R@300 0.511              3
```

1 Four detectors, rank-averaged

Isolation Forest, LOF, autoencoder and PCA.

2 The ensemble is worse

PR-AUC 0.326 versus 0.495 for Isolation Forest alone.

3 Why

Equal weights let weak detectors (LOF at 0.082) dilute the strong one. Ensembles help when members are *comparably good and different* — validate weights, do not just average.

Check your understanding

Q1 **Why does Isolation Forest give anomalies short paths?**

Answer: Few and different points are separated from everything else by very few random cuts.

Q2 **Changing contamination from 1% to 5% changes what?**

Answer: Only the threshold used by predict(); the scores stay identical.

Q3 **Why did every unsupervised method miss lost/stolen fraud?**

Answer: It looks like the owner's normal behaviour: it is not anomalous.

Q4 **Why can fewer PCA components detect better?**

Answer: Harder compression stops the model from rebuilding anomalies well, so their errors stay high.

Key takeaways

01 Unsupervised detectors define ‘normal’ by isolation, density, boundary or reconstruction.

02 Isolation Forest was the strongest and fastest here — a sound default.

03 Settings (k, nu, gamma, components, sample size) change results dramatically.

04 ROC-AUC hides large differences; PR-AUC and per-scheme recall reveal them.

05 Anomalies that mimic normal behaviour are invisible without labels.

06 Ensembles need weighting and validation; naive averaging can hurt.

PART 10

Supervised Machine Learning

When confirmed fraud labels exist, models can learn what fraud looks like — including the quiet schemes that unsupervised methods could not see.

IN THIS PART

- The accuracy trap
- Class weights for imbalance
- Logistic regression
- Random forests and gradient boosting
- Undersampling and hybrid models
- Precision–recall and ROC curves

SUPERVISED MACHINE LEARNING

When labels exist, learn from them

What changes

The model sees is_fraud during training and learns which feature combinations predict it.

What it gains

Faint, specific patterns: a lost card used at a fuel station at 2.5× the usual spend.

What it risks

It only knows schemes present in past labels. New tactics, and fraud nobody reported, remain invisible.

KEY INSIGHT

Labels are what we have from *past* investigations. That is why unsupervised scores remain valuable as a second opinion — and even as an input feature.

The accuracy trap, demonstrated

PYTHON

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, recall_score
d = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
p = d.predict(X_te)
print("accuracy: %.4f" % accuracy_score(y_te, p))
print("recall  : %.4f" % recall_score(y_te, p))
print("frauds caught:", int((p & y_te).sum()))
```

OUTPUT

```
accuracy: 0.9871
recall  : 0.0000
frauds caught: 0
```

1 A model that always says ‘legitimate’

DummyClassifier predicts the most frequent class for every payment.

2 98.71% accurate

3 Zero fraud caught

Recall 0. This is why accuracy never appears on a fraud dashboard.

Three ways to handle imbalance

1 Class weights

Tell the model each fraud counts as much as ~69 legitimate payments. No data is thrown away. Our default.

2 Resampling

Train on fewer legitimate rows (undersampling) or more copies of fraud (oversampling). Faster, but distorts probabilities.

3 Threshold moving

Train normally, then lower the decision threshold. Often the simplest and most honest fix.

KEY INSIGHT

All three change *where* the model draws the line. None creates new information. Good features matter more than any balancing trick.

SUPERVISED MACHINE LEARNING

Logistic regression, in words

The idea

Give every feature a weight. Add up the weighted evidence. Squash the total into a number between 0 and 1 — a probability-like score.

Reading the weights

A positive weight pushes towards fraud; negative towards legitimate. Bigger magnitude = stronger push (on scaled features).

ANALOGY

Like a judge adding up points: +3 for night time, +1.5 for a new device, −2 for a grocery merchant… then converting the total into a likelihood.

WATCH OUT

It can only add evidence, not combine it: it cannot learn ‘night is only suspicious when the customer is not a night owl’ unless we build that interaction ourselves.

Logistic regression with balanced class weights

PYTHON

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
lr = LogisticRegression(max_iter=2000, class_weight="balanced")
lr.fit(Xs_tr, y_tr)
p_lr = lr.predict_proba(Xs_te)[:, 1]
print(classification_report(y_te, p_lr > 0.5, digits=3))
```

OUTPUT

	precision	recall	f1-score	support
0	0.999	0.928	0.962	20586
1	0.142	0.914	0.245	268
accuracy			0.928	20854
macro avg	0.570	0.921	0.604	20854
weighted avg	0.988	0.928	0.953	20854

1 class_weight='balanced'

Automatically weights fraud by the inverse of its frequency.

2 Probabilities, then a 0.5 cut-off

predict_proba gives scores; the report applies 0.5.

3 Fraud recall 0.914

Most fraud is caught…

4 …but precision is 0.142

Seven false alarms per catch. Balanced weights push scores up, so 0.5 is far too low a cut-off here.

Ranking quality and the strongest weights

PYTHON

```
print(evaluate("Logistic Regression", p_lr))
coef = pd.Series(lr.coef_[0], index=FEATS).sort_values()
print(coef.tail(5).round(2).to_string())
```

OUTPUT

```
Logistic Regression
ROC-AUC 0.979  PR-AUC 0.626  P@300 0.563  R@300 0.631
log_dist          1.30
foreign           1.42
new_device        1.44
cat_electronics   1.50
is_night          2.80
```

1 **PR-AUC 0.626**

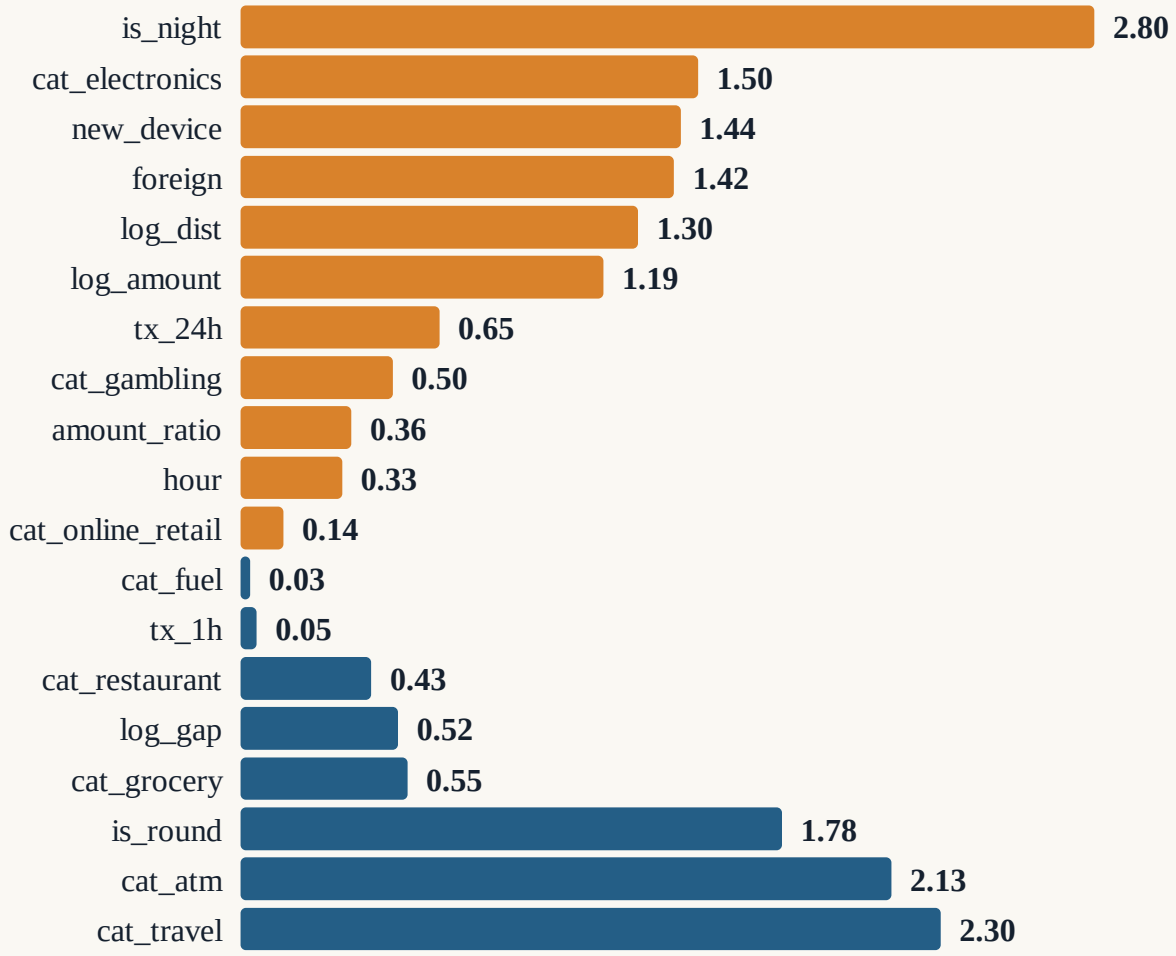
Already better than every unsupervised method (best 0.495).

2 **Night is the strongest push**

Weight 2.80, followed by electronics, new device, foreign and distance.

Logistic-regression weights

Scaled features, so magnitudes are comparable.



PRO TIP

Orange = pushes towards fraud; blue = pushes towards legitimate. Bar length is the size of the push.

KEY INSIGHT

Travel, ATM and round amounts push *against* fraud — because legitimate travellers and cash users are common. The model is learning the hard negatives.

What class weights actually do

PYTHON

```
lr0 = LogisticRegression(max_iter=2000).fit(Xs_tr, y_tr)
p0 = lr0.predict(Xs_te)
print("no weights  -> recall %.3f, alerts %d" % (
      recall_score(y_te, p0), p0.sum()))
p1 = p_lr > 0.5
print("balanced    -> recall %.3f, alerts %d" % (
      recall_score(y_te, p1), p1.sum()))
```

OUTPUT

```
no weights      -> recall 0.571, alerts 177   1
balanced        -> recall 0.914, alerts 1730  2
```

1 Without weights

Recall 0.571 from 177 alerts: cautious, because fraud is rare.

2 With balanced weights

Recall 0.914 from 1,730 alerts: aggressive.

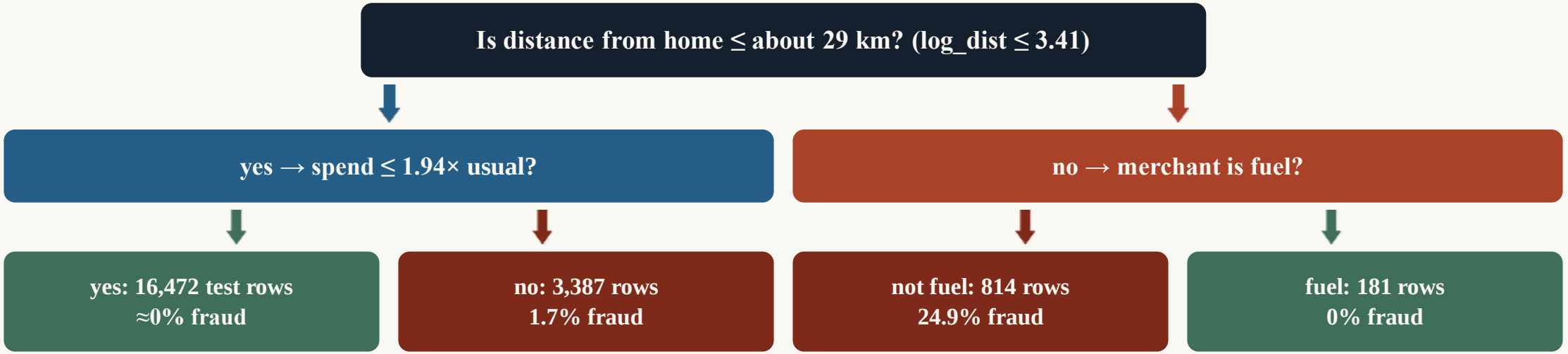
3 Same information, different line

Weights shift the decision boundary; ranking quality changes far less. You can get similar behaviour by moving the threshold.

SUPERVISED MACHINE LEARNING

Decision trees: learning questions from labels

A real depth-2 tree trained on our data (next slide). The questions are chosen from labels, not at random.



KEY INSIGHT

A tree asks the yes/no question that best separates fraud from legitimate, then repeats inside each branch. It discovered on its own that *far from home* matters, and that far-away *fuel* purchases are travellers, not fraud.

Training and reading a small decision tree — the code

PYTHON

```
from sklearn.tree import DecisionTreeClassifier, export_text
dt = DecisionTreeClassifier(max_depth=2, class_weight="balanced",
                            random_state=0).fit(X_tr, y_tr)
print(export_text(dt, feature_names=FEATS, decimals=2))
leaf = dt.apply(X_te)
print(pd.Series(y_te).groupby(leaf).agg(["size", "mean"])
      .round(3).to_string())
```

1

1 Depth 2: just two questions

Kept tiny so it can be read; real models use far deeper trees.

Training and reading a small decision tree — the output

PYTHON · LAST LINES OF THE CELL ABOVE

```
leaf = dt.apply(X_te)
print(pd.Series(y_te).groupby(leaf).agg(["size", "mean"])
      .round(3).to_string())
```

OUTPUT

```
|--- log_dist <= 3.41                                                    2
|       |--- amount_ratio <= 1.94
|       |       |--- class: 0
|       |--- amount_ratio >           1.94
|       |       |--- class: 1
|--- log_dist >           3.41
|       |--- cat_fuel <= 0.50
|       |       |--- class: 1
|       |--- cat_fuel >           0.50
|       |       |--- class: 0

      size       mean
2    16472    0.000
3     3387    0.017
5      814    0.249                                                       3
6      181    0.000
```

2 The learned rules

Distance first, then spend ratio or merchant. Note the thresholds are on log and scaled-ratio features.

3 Test rows per leaf and fraud rate

Leaf 5 (far away, not fuel) concentrates fraud at 24.9% — almost 20× the base rate.

SUPERVISED MACHINE LEARNING

Forests and boosting: many trees are better than one

Random forest

Hundreds of deep trees, each trained on a random sample of rows and features. They vote. Errors of individual trees cancel out.

Gradient boosting

Shallow trees built one after another, each focusing on the mistakes of the trees before it. The industry workhorse for fraud.

Why they suit fraud

Handle mixed feature types, interactions and skew without scaling; fast to score; strong on tabular data.

ANALOGY

Think of a forest as a panel of independent experts voting, and boosting as a team where each new member is trained specifically to fix the team’s current blind spots.

Random forest

PYTHON

```
from sklearn.ensemble import RandomForestClassifier
t = time.time()
rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=2,
    class_weight="balanced_subsample", n_jobs=-1,
    random_state=0).fit(X_tr, y_tr)
p_rf = rf.predict_proba(X_te)[:, 1]
print("fit %.1fs" % (time.time() - t))
print(evaluate("Random Forest", p_rf))
```

OUTPUT

```
fit 4.9s
Random Forest
ROC-AUC 0.995   PR-AUC 0.858    P@300 0.723     R@300 0.810
```

1 300 trees, balanced per sample

Each tree's bootstrap sample is re-weighted to balance the classes.

2 No scaling needed

Trees split on thresholds, so raw features work.

3 PR-AUC 0.858

A big jump from logistic regression (0.626).

4 P@300 = 0.723, R@300 = 0.810

Seven in ten of the top 300 alerts are fraud, and they contain 81% of all test fraud.

Histogram gradient boosting

PYTHON

```
from sklearn.ensemble import HistGradientBoostingClassifier
t = time.time()
hgb = HistGradientBoostingClassifier(max_iter=300,
      learning_rate=0.05, max_leaf_nodes=31,
      class_weight="balanced", random_state=0)
hgb.fit(X_tr, y_tr)
p_hgb = hgb.predict_proba(X_te)[:, 1]
print("fit %.1fs, iterations %d" % (time.time() - t, hgb.n_iter_))
print(evaluate("Gradient Boosting", p_hgb))
```

OUTPUT

```
fit 1.5s, iterations 115
Gradient Boosting
ROC-AUC 0.995   PR-AUC 0.843    P@300 0.680     R@300 0.761
```

1 **300 boosting rounds at most**

Small learning rate (0.05); early stopping ended at 115 rounds.

2 **Fast**

Features are bucketed into histograms, so training took 1.5 seconds.

3 **PR-AUC 0.843**

On a par with the random forest. On real, larger data, tuned boosting usually edges ahead.

Gradient boosting’s confusion matrix at 0.5

PYTHON

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_te, p_hgb > 0.5)
print(pd.DataFrame(cm, index=["actual legit", "actual fraud"],
                   columns=["pred legit", "pred fraud"]))
tn, fp, fn, tp = cm.ravel()
print("\nprecision %.3f  recall %.3f" % (tp/(tp+fp), tp/(tp+fn)))
```

OUTPUT

	pred legit	pred fraud	
actual legit	20283	303	2
actual fraud	25	243	3
precision 0.445	recall 0.907		4

1. **243 frauds caught**
   Of 268 — recall 0.907.
2. **303 false alarms**
   Out of 20,586 legitimate payments: just 1.5% of customers inconvenienced.
3. **25 frauds missed**
4. **Precision 0.445**
   Nearly one alert in two is real fraud — a workable queue.

The same matrix, drawn

	Predicted legitimate	Predicted fraud
Actual legitimate	**20,283** true negatives	**303** false alarms
Actual fraud	**25** missed fraud	**243** frauds caught

KEY INSIGHT

The giant top-left number is why accuracy is meaningless: the model would be ‘98%+ accurate’ even if it caught nothing.

PRO TIP

Every decision about thresholds trades the gold box against the dark-red box.

Undersampling: train on a smaller, balanced sample

How

Keep every fraud, plus a random 20 legitimate rows per fraud (1:20 instead of 1:69).

Why

Training is ~3× faster; on huge data (billions of payments) it is often the only practical option.

Catch

The model believes fraud is more common than it really is, so its probabilities are inflated. Rankings survive; raw probabilities do not.

Undersampling in code

PYTHON

```
rs = np.random.default_rng(7)
pos = np.where(y_tr == 1)[0]                                          1
neg = rs.choice(np.where(y_tr == 0)[0], len(pos) * 20,                2
                replace=False)
ix = np.concatenate([pos, neg])
hgb_u = HistGradientBoostingClassifier(max_iter=300,
            learning_rate=0.05, random_state=0)
hgb_u.fit(X_tr[ix], y_tr[ix])
p_u = hgb_u.predict_proba(X_te)[:, 1]
print("train rows used:", len(ix), "(1:20 ratio)")
print(evaluate("GBM undersampled", p_u))
```

OUTPUT

```
train rows used: 12852 (1:20 ratio)                                   3
GBM undersampled
ROC-AUC 0.995   PR-AUC 0.854    P@300 0.710     R@300 0.795           4
```

1. **All 612 training frauds**

2. **Twenty legitimate rows per fraud**
   Sampled without replacement.

3. **12,852 rows instead of 41,319**

4. **PR-AUC 0.854**
   Ranking quality is unchanged (even slightly higher) with less than a third of the data.

Probabilities tell a different story

PYTHON

```
print("mean P(fraud), undersampled model: %.4f" % p_u.mean())
print("mean P(fraud), full-data model   : %.4f" % p_hgb.mean())
print("true test fraud rate             : %.4f" % y_te.mean())
print("ROC-AUC  under %.3f vs full %.3f" % (
      roc_auc_score(y_te, p_u), roc_auc_score(y_te, p_hgb)))
```

OUTPUT

```
mean P(fraud), undersampled model: 0.0176   1
mean P(fraud), full-data model             : 0.0423
true test fraud rate                       : 0.0129   2
ROC-AUC under 0.995 vs full 0.995                     3
```

1 Average predicted fraud probability

Undersampled: 1.76%. Full data with balanced weights: 4.23%.

2 The truth is 1.29%

Both are inflated — class weights distort probabilities too.

3 Identical ROC-AUC

Rankings are as good as each other. If you need true probabilities (for expected-loss maths), calibrate (Part 11).

SUPERVISED MACHINE LEARNING

Hybrid models: give the classifier an anomaly score

Feed the Isolation Forest score in as one extra feature. The supervised model can then learn when ‘strangeness’ predicts fraud.

Why it can help

The unsupervised score summarises many weak oddities into one number the classifier might not build itself.

Why it may not

If the classifier already has strong features, the extra score adds little and can add noise.

How to decide

Test it on a later validation period — exactly what we do next.

Gradient boosting plus the Isolation Forest score

PYTHON

```
X_tr_h = np.column_stack([X_tr, -iso.score_samples(Xs_tr)])
X_te_h = np.column_stack([X_te, s_iso])
hgb_h = HistGradientBoostingClassifier(max_iter=300,
            learning_rate=0.05, class_weight="balanced",
            random_state=0).fit(X_tr_h, y_tr)
p_h = hgb_h.predict_proba(X_te_h)[:, 1]
print(evaluate("GBM + IForest score", p_h))
```

OUTPUT

```
GBM + IForest score
ROC-AUC 0.992   PR-AUC 0.847    P@300 0.713     R@300 0.799
```

1 Append one column

Training rows get their own Isolation Forest scores; test rows get the test scores computed earlier.

2 PR-AUC 0.847 vs 0.843

Essentially no gain here: the boosted trees already capture what Isolation Forest knows.

3 Where hybrids shine

Thin labels, many new attack variants, or when the anomaly model sees data the classifier does not (e.g. device graphs).

Supervised scoreboard

PYTHON

```
sup = pd.DataFrame(RES2 + [evaluate("IsolationForest", s_iso)])
print(sup[["model", "roc_auc", "pr_auc", "prec_at_k",
           "recall_at_k"]].to_string(index=False))
```

OUTPUT

model	roc_auc	pr_auc	prec_at_k	recall_at_k	
Logistic Regression	0.979	0.626	0.563	0.631	
Random Forest	0.995	0.858	0.723	0.810	1
Gradient Boosting	0.995	0.843	0.680	0.761	
GBM + IForest score	0.992	0.847	0.713	0.799	
IsolationForest	0.961	0.495	0.457	0.511	2

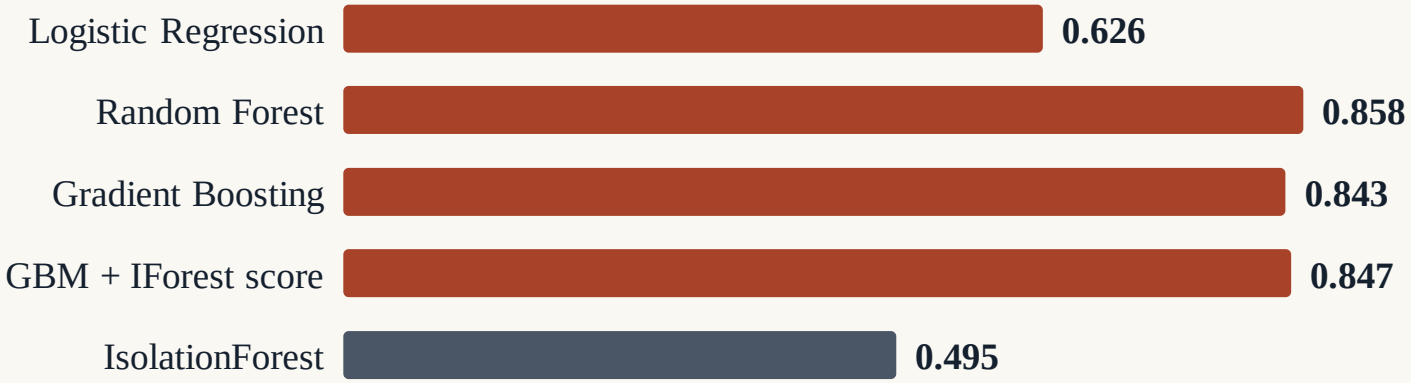
1 Trees beat linear

Forest and boosting reach PR-AUC ~0.85 versus 0.63 for logistic regression.

2 Labels beat no labels

Best unsupervised method: 0.495. Supervision nearly doubles ranking quality on known schemes.

Supervised versus unsupervised (PR-AUC)



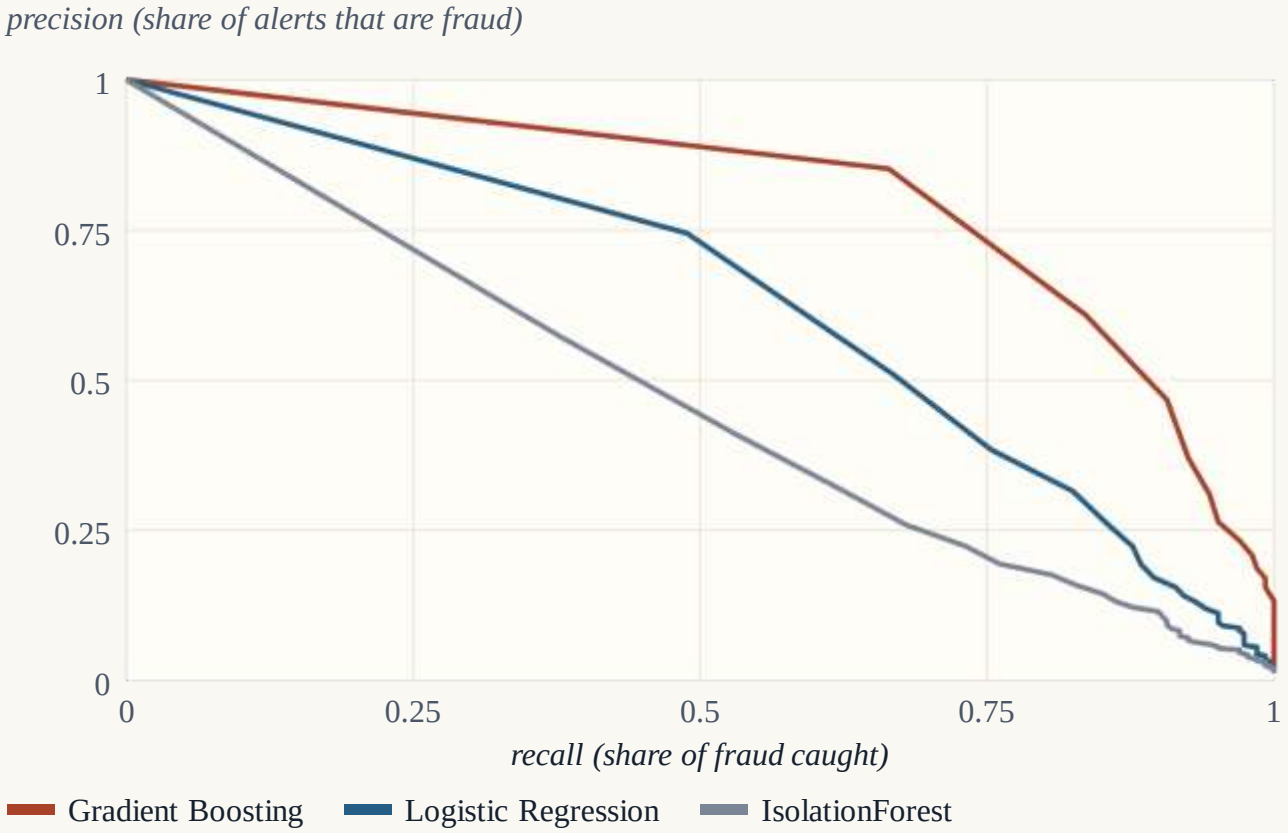
WHAT THE OUTPUT SHOWS

All four supervised models beat the best unsupervised one.

KEY INSIGHT

But remember: Isolation Forest ranked the *never-seen* gambling card-testing first. Use both.

Precision–recall curves



KEY INSIGHT

Each curve traces every possible threshold. Up and to the right is better.

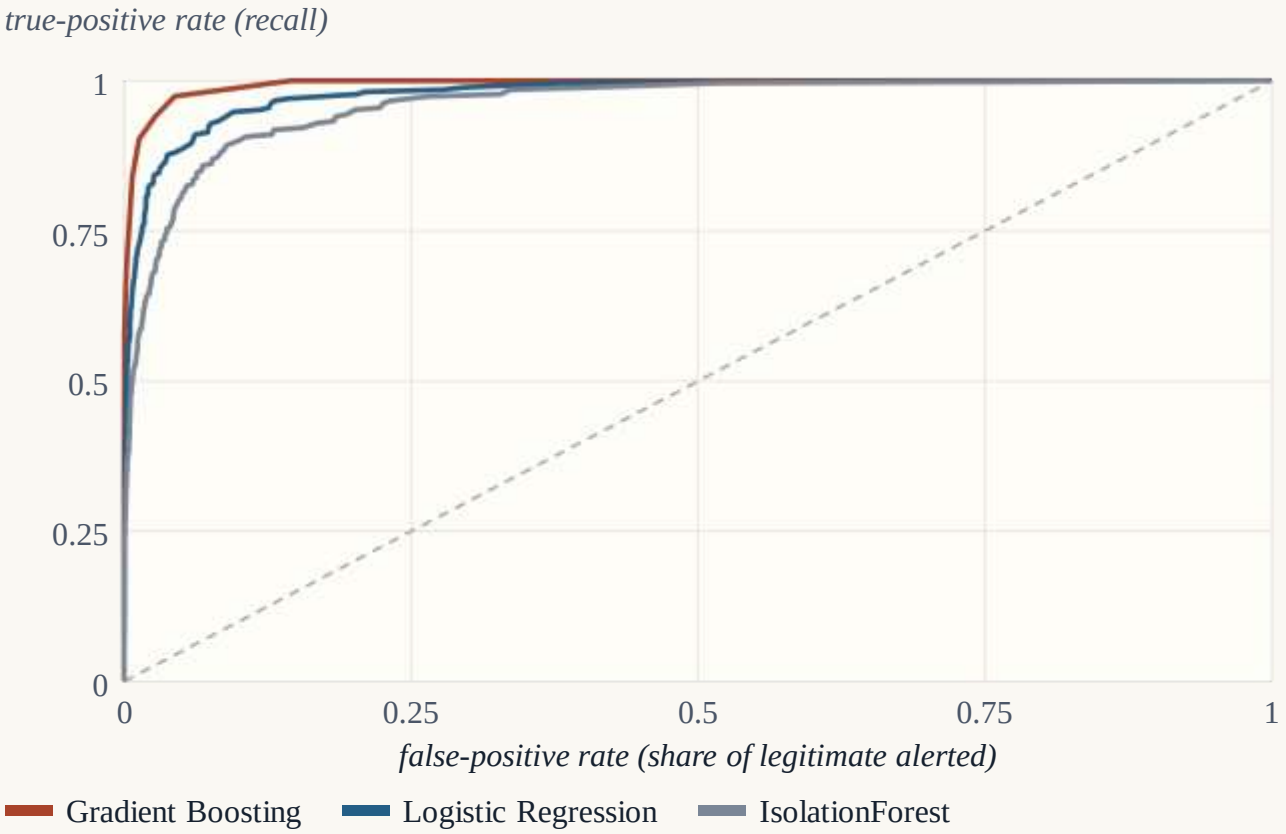
WHAT THE OUTPUT SHOWS

Gradient boosting holds precision above 0.6 until recall passes 0.8; Isolation Forest falls below 0.5 early.

PRO TIP

PR-AUC is the area under each curve.

ROC curves: why they flatter



WHAT THE OUTPUT SHOWS

All three curves hug the top-left corner — they look almost equally excellent.

WATCH OUT

The x-axis is a share of 20,586 legitimate payments: 5% sounds tiny but is over 1,000 false alarms. The PR curve made the differences obvious; ROC hides them.

Why unsupervised detection still matters

Supervised models are backward-looking

They recognise the fraud of the past. Brand-new schemes have no labels yet.

Unsupervised models are a safety net

They flag whatever is strange today, labelled or not — exactly how the drifted gambling card-testers were caught first.

Labels are biased

They only exist for cases someone investigated — usually those an earlier model or rule flagged.

KEY INSIGHT

The robust architecture: supervised model for known fraud + anomaly detector for novelty + rules for policy, all feeding one ranked queue.

Key takeaways

01 Accuracy is useless for rare events: a do-nothing model scores 98.7%.

02 Class weights, resampling and thresholds all move the decision line.

03 Tree ensembles (forests, boosting) beat linear models on fraud features.

04 Supervision nearly doubled PR-AUC over the best unsupervised method.

05 Balancing distorts probabilities; rankings survive.

06 Keep an unsupervised layer for patterns that have no labels yet.

PART 11

Thresholds, Cost and Calibration

A model produces scores; a business needs decisions. This part turns scores into alerts using costs, capacity and calibrated probabilities.

IN THIS PART

- Precision and recall at each threshold
- Choosing the threshold by money
- Alert budgets
- Calibration: honest probabilities

Precision, recall and alerts at six thresholds

PYTHON

```
from sklearn.metrics import precision_recall_curve
prec, rec, thr = precision_recall_curve(y_te, p_hgb)
for t_ in [0.1, 0.3, 0.5, 0.7, 0.9, 0.97]:
    i = np.searchsorted(thr, t_)
    print("thr %.2f  precision %.3f  recall %.3f  alerts %5d" % (
          t_, prec[i], rec[i], int((p_hgb >= t_).sum())))
```

1

OUTPUT

```
thr 0.10   precision 0.159   recall 0.993   alerts  1674   2
thr 0.30   precision 0.294   recall 0.944   alerts   861
thr 0.50   precision 0.445   recall 0.907   alerts   546
thr 0.70   precision 0.620   recall 0.836   alerts   361   3
thr 0.90   precision 0.899   recall 0.597   alerts   178
thr 0.97   precision 0.993   recall 0.530   alerts   143   4
```

1 The full curve

precision_recall_curve returns precision and recall at every distinct threshold.

2 0.10: catch almost everything

Recall 0.993 but 1,674 alerts at precision 0.159.

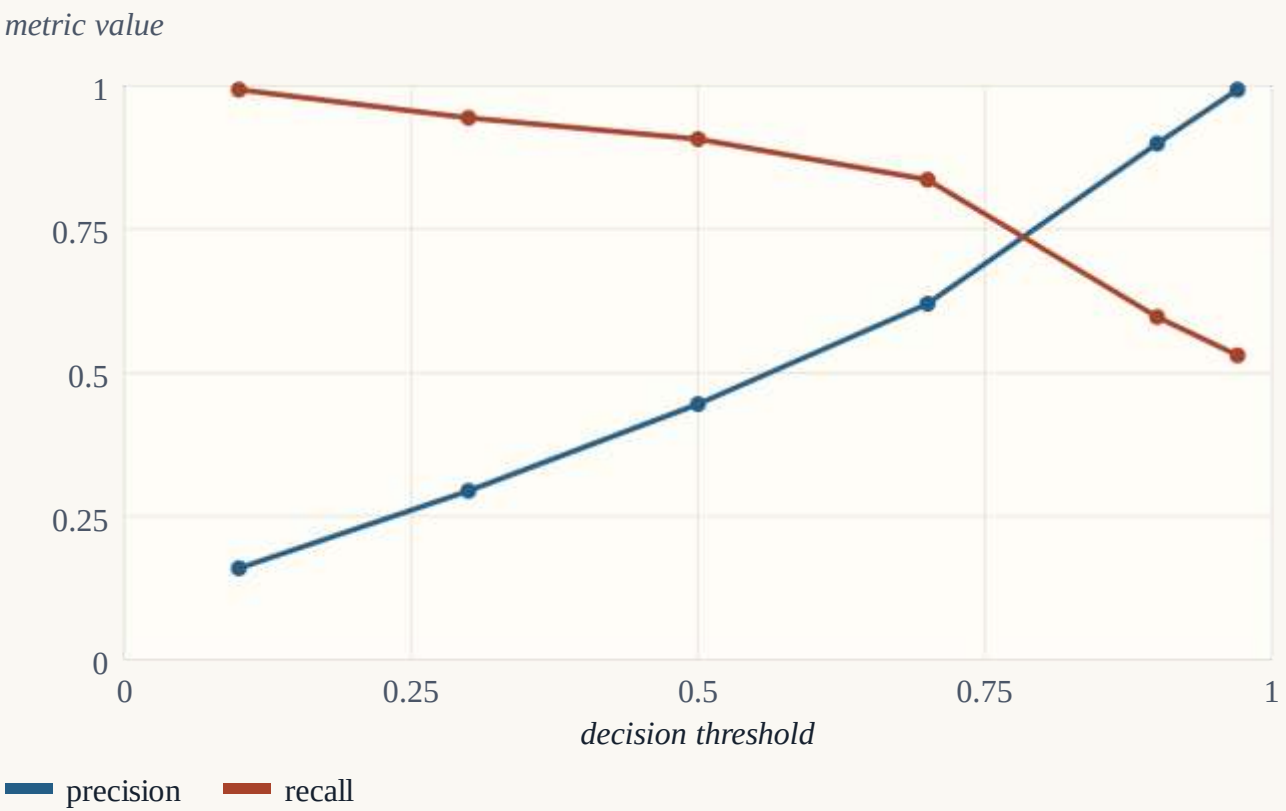
3 0.70: a balanced point

361 alerts, precision 0.620, recall 0.836.

4 0.97: almost no false alarms

Precision 0.993 — but half the fraud is missed.

The threshold dial



WHAT THE OUTPUT SHOWS

Turning the dial right trades recall for precision. The two lines cross between 0.7 and 0.9.

KEY INSIGHT

Alert volume falls from 1,674 to 143 across the dial — the number that analyst managers care about most.

Choosing a threshold by money

Cost of an alert

Every alert costs analyst time — here we assume **£5 per review** (an illustrative figure; use your own).

Cost of a miss

Every missed fraud costs its **full amount** (a simplification: real losses include fees and recovery).

KEY INSIGHT

Total cost = (number of alerts × £5) + (value of fraud missed). Try every threshold; pick the cheapest.

PRO TIP

This turns a statistical choice into a business decision everyone can audit and debate.

The cheapest threshold

PYTHON

```
amt = te.amount.values
REVIEW_COST = 5.0                # cost of one review, GBP
def total_cost(t_):
    alert = p_hgb >= t_
    missed = amt[(~alert) & (y_te == 1)].sum()
    return alert.sum() * REVIEW_COST + missed
grid = np.linspace(0.02, 0.98, 49)
costs = [total_cost(t_) for t_ in grid]
best = grid[int(np.argmin(costs))]
print("best threshold %.2f -> cost GBP %.0f" % (best, min(costs)))
print("threshold 0.50  -> cost GBP %.0f" % total_cost(0.5))
print("no model at all -> cost GBP %.0f" % amt[y_te == 1].sum())
```

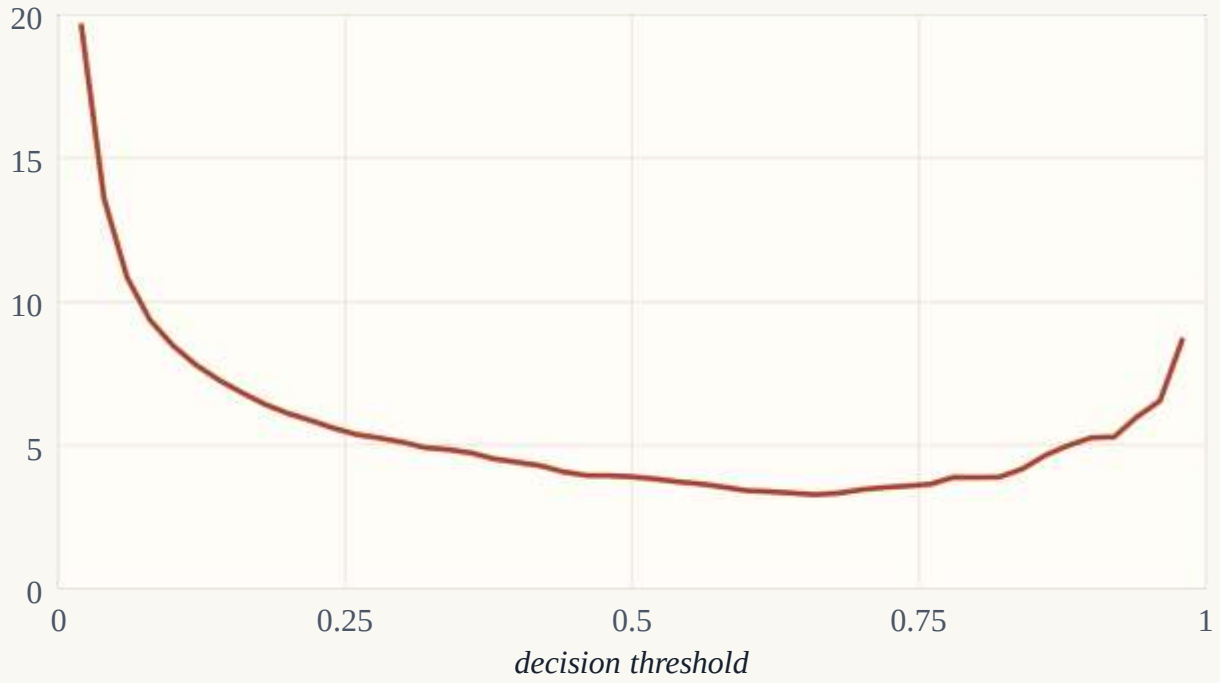
OUTPUT

```
best threshold 0.66 -> cost GBP 3274
threshold 0.50          -> cost GBP 3893
no model at all -> cost GBP 22257
```

1. **Review cost: £5**
2. **Total cost for any threshold**
   Alerts times £5, plus missed fraud value.
3. **Search 49 thresholds**
4. **Best: 0.66 → £3,274**
5. **Default 0.5 → £3,893**
   Using the library default costs about 19% more.
6. **No model → £22,257**
   The model saves about 85% of fraud cost in the test month.

The cost curve

total cost in the test month (£ thousands)



WHAT THE OUTPUT SHOWS

Left: too many alerts — review costs explode. Right: too few — missed fraud dominates.

KEY INSIGHT

The floor is broad between about 0.5 and 0.8: small threshold changes there cost little. That is reassuring for operations.

Alert budgets: capacity first

PYTHON

```
daily = te.assign(p=p_hgb).groupby("day")
for budget in [3, 5, 10, 20]:      # alerts analysts review/day
    caught = sum(d.nlargest(budget, "p").is_fraud.sum()                 1
                 for _, d in daily)
    print("budget %2d/day -> caught %3d of %d frauds" % (
          budget, caught, y_te.sum()))
```

OUTPUT

```
budget  3/day -> caught  79 of 268 frauds                              2
budget  5/day -> caught 116 of 268 frauds
budget 10/day -> caught 180 of 268 frauds                              4
budget 20/day -> caught 233 of 268 frauds                              3
```

1 Top-N per day

Each day, analysts review only the N highest-scored payments.

2 3 per day → 79 frauds

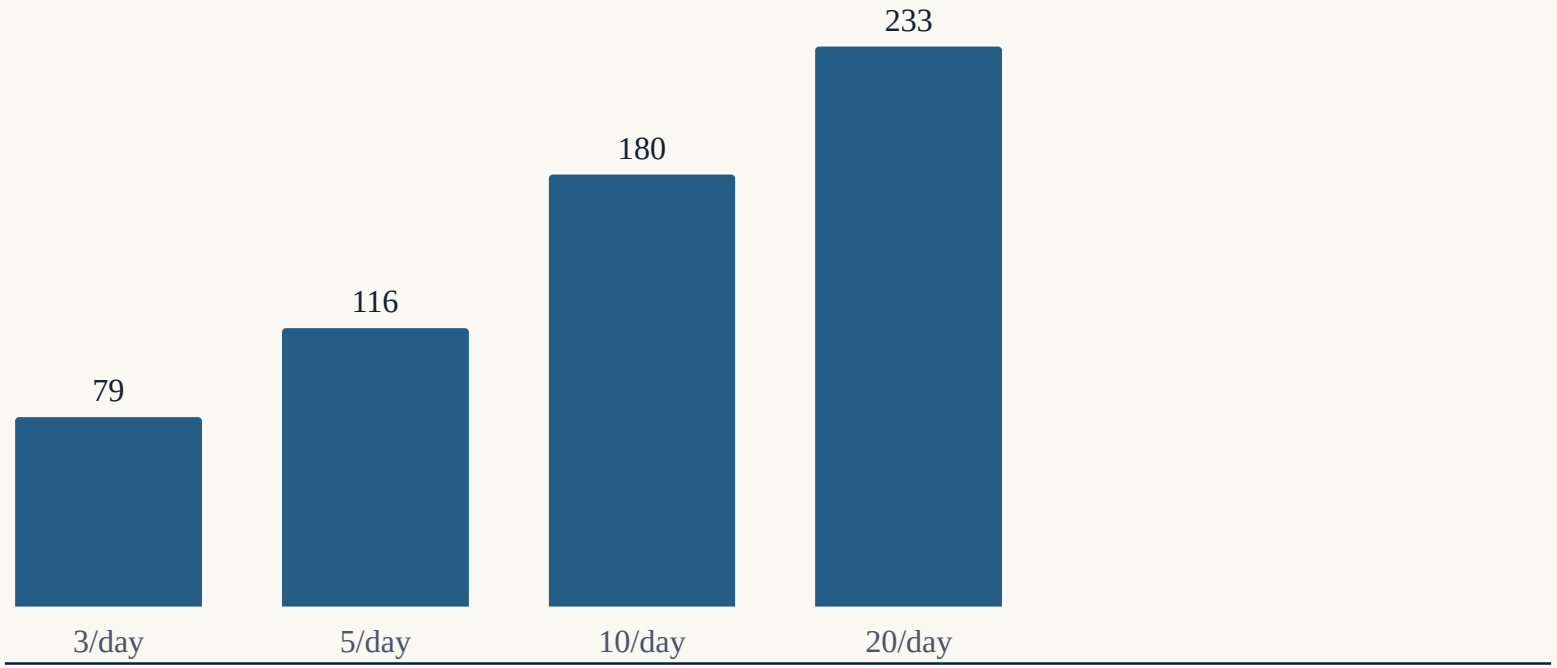
3 20 per day → 233 of 268

87% of fraud with 600 reviews a month.

4 Use this to size the team

Each extra reviewer-hour buys a measurable number of extra catches.

Frauds caught per daily review budget



WHAT THE OUTPUT SHOWS

Diminishing returns: three reviews a day already catch 79 frauds in the month; doubling from 10 to 20 a day adds 53 more.

KEY INSIGHT

Scores let you pick the budget explicitly instead of discovering it from an arbitrary threshold.

Calibration: can we trust the probability?

Calibrated

Of all payments scored ‘30%’, about 30% turn out to be fraud.

Why it matters

Expected-loss decisions (‘block if probability × amount > £20’) and combining models need honest probabilities.

WATCH OUT

Class weights and undersampling deliberately distort probabilities. Rankings are fine; the numbers themselves are not.

PRO TIP

A reliability table compares predicted scores with what actually happened, band by band.

A reliability table for gradient boosting

PYTHON

```
bins = pd.cut(p_hgb, [0, .01, .05, .2, .5, .8, 1.0])
cal = pd.DataFrame({"bin": bins, "y": y_te}).groupby(
    "bin", observed=True).y.agg(["size", "mean"])
cal.columns = ["n", "observed_fraud_rate"]
print(cal.round(3))
```

OUTPUT

bin	n	observed_fraud_rate	
(0.0, 0.01]	14941	0.000	
(0.01, 0.05]	3506	0.000	
(0.05, 0.2]	1267	0.006	
(0.2, 0.5]	594	0.030	
(0.5, 0.8]	267	0.157	1
(0.8, 1.0]	279	0.720	2

1 Scores between 0.5 and 0.8

Only 15.7% of these were fraud — far below what the score suggests.

2 Scores above 0.8

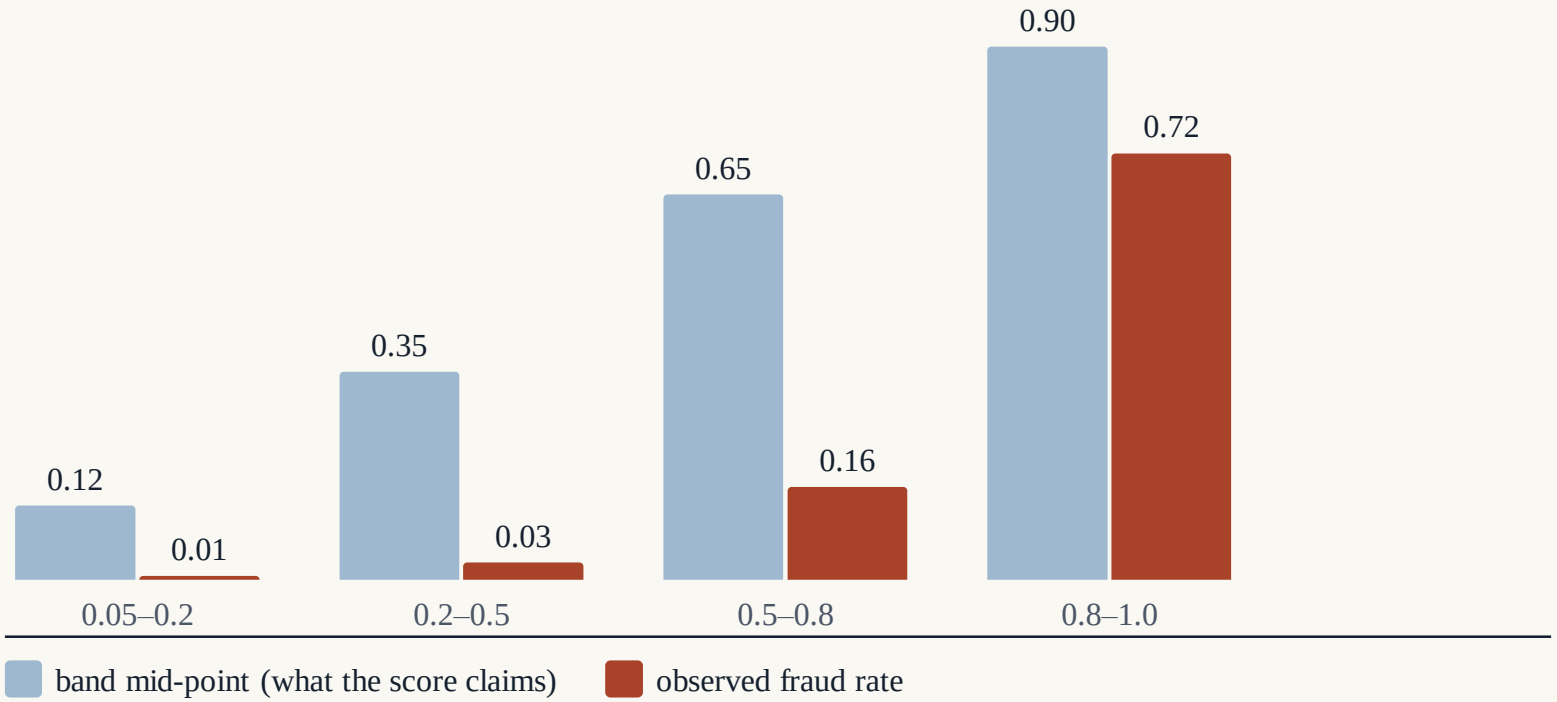
72% fraud — over-confident, but closer.

3 Verdict: over-estimates risk

Expected, because we trained with balanced class weights.

Predicted versus observed fraud rate

Bands of predicted probability from the reliability table.



WHAT THE OUTPUT SHOWS

For a calibrated model, each pair of bars would be about equal. Here the orange (reality) is far shorter in every band.

Fixing calibration with isotonic regression

PYTHON

```
from sklearn.calibration import CalibratedClassifierCV
cal_m = CalibratedClassifierCV(
    HistGradientBoostingClassifier(max_iter=300,
        learning_rate=0.05, random_state=0),
    method="isotonic", cv=3).fit(X_tr, y_tr)                    1
p_cal = cal_m.predict_proba(X_te)[:, 1]
print("mean P before: %.4f  after: %.4f  truth: %.4f" % (
      p_hgb.mean(), p_cal.mean(), y_te.mean()))
from sklearn.metrics import brier_score_loss
print("Brier before %.4f  after %.4f" % (
      brier_score_loss(y_te, p_hgb), brier_score_loss(y_te, p_cal)))
```

OUTPUT

```
mean P before: 0.0423        after: 0.0120        truth: 0.0129    2
Brier before 0.0121          after 0.0045                          3
```

1 Calibrated wrapper

Trains the model on part of the data and learns a monotone mapping from scores to observed rates on the rest (3-fold).

2 Average probability now honest

1.20% predicted versus 1.29% actual (was 4.23%).

3 Brier score drops 0.0121 → 0.0045

The Brier score measures probability accuracy; lower is better.

Key takeaways

01 Thresholds are business decisions; library defaults are arbitrary.

02 Choose thresholds by total cost: review cost plus missed fraud value.

03 The cheapest threshold (0.66) beat the default (0.5) by about 19%.

04 Alert budgets translate model quality into team size and catches.

05 Weighted or resampled models are mis-calibrated; rankings are still fine.

06 Calibrate when you need true probabilities for expected-loss decisions.

PART 12

Leakage and Honest Validation

The most common reason fraud models fail in production is not a weak algorithm — it is an evaluation that quietly cheated.

IN THIS PART

- What leakage is
- A leaky feature, demonstrated
- Random versus time-based splits
- A validation checklist

Data leakage: when the future sneaks into training

1 Target leakage

A feature built from the label or from information only known after the outcome (e.g. ‘chargeback filed’).

2 Temporal leakage

Training on data that happened after the test period, or computing statistics over all time.

3 Group leakage

Rows from the same card-testing burst land in both train and test, so the model memorises the attack.

4 Preprocessing leakage

Scalers, encoders or medians fitted on the full data set, including test rows.

A leaky feature, demonstrated

PYTHON

```
# MISTAKE: customer's fraud count over ALL days (incl. own label)
leak_all = tx.groupby("customer_id").is_fraud.transform("sum")                1
# SAFER: frauds on this card strictly BEFORE this payment
prior = tx.groupby("customer_id").is_fraud.cumsum() - tx.is_fraud             2
def with_feature(col):
    A = np.column_stack([X_tr, col[tr.index]])
    B = np.column_stack([X_te, col[te.index]])
    m = HistGradientBoostingClassifier(max_iter=300,
            learning_rate=0.05, class_weight="balanced",
            random_state=0).fit(A, y_tr)
    return average_precision_score(y_te, m.predict_proba(B)[:, 1])
base = average_precision_score(y_te, p_hgb)
print("no extra feature       PR-AUC %.3f" % base)
print("leaky (all days)       PR-AUC %.3f" % with_feature(leak_all))
print("prior frauds (earlier) PR-AUC %.3f" % with_feature(prior))
```

OUTPUT

```
no extra feature        PR-AUC 0.843
leaky (all days)        PR-AUC 0.923   3
prior frauds (earlier)  PR-AUC 0.869   4
```

1 The mistake

‘How many frauds does this customer have?’ counted over all 90 days — including the current payment’s own label and future ones.

2 The safer version

Only frauds on this card strictly *before* this payment.

3 Leak: 0.843 → 0.923

A tempting jump that would vanish in production, where future labels do not exist.

4 Honest gain: 0.869

Prior fraud history genuinely helps — but only if labels really are available that quickly. In practice they arrive weeks later (Part 1).

How big the illusion is



WATCH OUT

The leaky feature looks like a 9.5% improvement. In deployment it is not merely useless — the model would rely on a signal that is missing, and perform *worse* than the honest model.

Random split versus time split

PYTHON

```
from sklearn.model_selection import train_test_split
Xa, ya = tx[FEATS].values, tx.is_fraud.values
a_tr, a_te, b_tr, b_te = train_test_split(Xa, ya, test_size=.33,
                        stratify=ya, random_state=0)
m = HistGradientBoostingClassifier(max_iter=300,
        learning_rate=0.05, random_state=0).fit(a_tr, b_tr)
print("random split PR-AUC %.3f" % average_precision_score(
      b_te, m.predict_proba(a_te)[:, 1]))
print("time split   PR-AUC %.3f" % average_precision_score(
      y_te, p_hgb))
```

OUTPUT

```
random split PR-AUC 0.866
time split          PR-AUC 0.843
```

1 A random 67/33 split

Mixes January, February and March rows in both sets.

2 Random split looks better

PR-AUC 0.866 versus 0.843 — partly from memorising attack bursts that span both sets.

3 The gap would grow with real drift

Our drift is mild. On real portfolios, random splits regularly overstate performance badly.

A validation checklist for fraud and security models

Split by time; test on the most recent period	Models always face the future
Fit scalers, encoders, medians on training rows only	Prevents preprocessing leakage
Build features only from information available at decision time	Prevents target and temporal leakage
Respect label delay: drop the last weeks of training if labels are incomplete	Unlabelled recent fraud looks legitimate
Keep whole entities or bursts on one side of the split when appropriate	Prevents memorisation
Report per segment (scheme, merchant, channel)	Averages hide blind spots
Be suspicious of near-perfect results	Usually a leak or an unrealistic dataset

Key takeaways

01 Leakage makes models look better in testing and worse in production.

02 A single leaky feature inflated PR-AUC from 0.843 to 0.923.

03 Only use information available at the moment of decision.

04 Fit all preprocessing on the training period only.

05 Split by time; random splits overstate performance.

06 Suspect perfection.

PART 13

Explainability

An alert without a reason wastes analyst time and fails regulators. This part opens the model up — globally and alert by alert.

IN THIS PART

- Why explanations matter
- Global importance by permutation
- Reason codes for individual alerts
- Partial dependence

EXPLAINABILITY

Who needs explanations?

Analysts

‘Why is this flagged?’ decides where to look first and how fast a case closes.

Customers

A declined payment needs a plain explanation and a route to resolve it.

Regulators and auditors

Automated decisions affecting people must be explainable and shown not to discriminate.

Model builders

Explanations expose leaks, spurious shortcuts and drift.

EXPLAINABILITY

Global versus local explanations

Global

What drives the model overall? Which features matter most across all payments? Used for validation and governance.

Tools here: **permutation importance**, partial dependence.

Local

Why did *this* payment get *this* score? Used on every alert.

Tools here: **reason codes** from deviations. Popular alternatives: SHAP and LIME (separate libraries).

EXPLAINABILITY

Permutation importance, in words

KEY INSIGHT

Shuffle one feature’s values randomly across the test set, breaking its link with fraud. Re-score. If performance collapses, the model relied on that feature.

ANALOGY

Like removing one witness from a trial and seeing whether the verdict changes.

Strengths

Works for any model; measured on the metric you care about (PR-AUC here).

Caveat

Correlated features share credit: shuffle one and its twin still carries the signal, so both can look unimportant.

EXPLAINABILITY

Permutation importance for gradient boosting

PYTHON

```
from sklearn.inspection import permutation_importance
pi = permutation_importance(hgb, X_te, y_te, n_repeats=5,
        scoring="average_precision", random_state=0, n_jobs=-1)
imp = pd.Series(pi.importances_mean, index=FEATS)
print(imp.sort_values(ascending=False).head(8).round(4)
         .to_string())
```

1

OUTPUT

log_dist	0.725
cat_restaurant	0.092
amount_ratio	0.088
cat_fuel	0.085
cat_online_retail	0.082
hour	0.082
log_gap	0.058
cat_grocery	0.034

2

3

1 Scored on PR-AUC

Five shuffles per feature, on the test month.

2 Distance dominates

Shuffling distance drops PR-AUC by 0.725 — the model leans on it heavily.

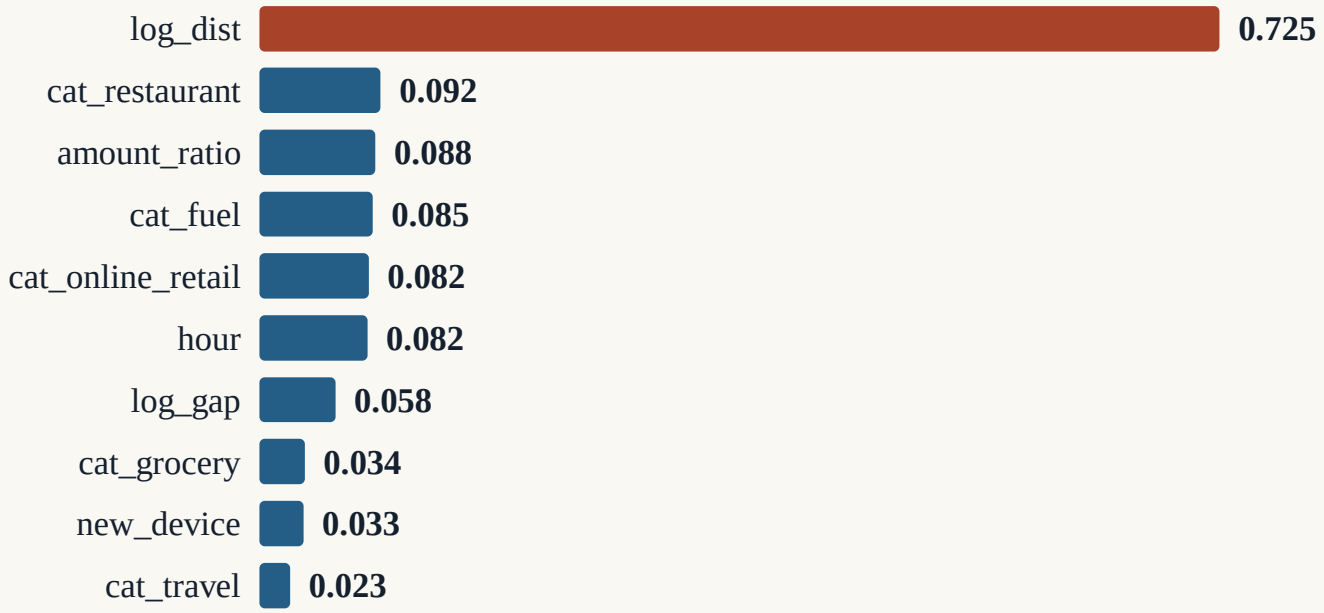
3 Merchant columns next

Everyday categories (restaurant, fuel) help the model *rule out* fraud.

EXPLAINABILITY

What the model relies on

Drop in PR-AUC when each feature is shuffled.



WATCH OUT

A single dominant feature is a risk: if distance data becomes unavailable or is spoofed (VPNs, GPS faking), performance would collapse.

PRO TIP

Governance teams should track concentration of importance over time.

Reason codes for one alert

PYTHON

```
def reason_codes(row, k=3):
    ref = tr[NUM].median()                                          1
    spread = (tr[NUM].quantile(.9) - tr[NUM].quantile(.1)
              ).replace(0, 1)
    dev = ((row[NUM] - ref) / spread).abs()                         2
    return dev.sort_values(ascending=False).head(k)
alert = te.assign(p=p_hgb).sort_values("p").iloc[-1]
print("alert p=%.3f  scheme=%s" % (alert.p, alert.scheme))
print(reason_codes(alert).round(2).to_string())
```

OUTPUT

```
alert p=0.998        scheme=cnp_foreign                             3
log_dist          2.92
new_device         1.0
foreign            1.0
```

1 Typical value and spread

From the training period: median and the 10th–90th percentile range of each numeric feature.

2 Deviation

How many typical ranges away from normal is this payment? Report the three largest.

3 Top alert: foreign card-not-present

Distance is 2.9 ranges beyond normal, plus a new device and a foreign merchant.

EXPLAINABILITY

Reason codes for an account takeover

PYTHON

```
row = te.assign(p=p_hgb)
row = row[row.scheme == "account_takeover"].sort_values("p").iloc[-1]
print("alert p=%.3f  scheme=%s" % (row.p, row.scheme))
for f_, v in reason_codes(row).items():
    print(" %-14s value=%-8s deviation %.1f" % (
        f_, round(row[f_], 2), v))
```

OUTPUT

```
alert p=0.998      scheme=account_takeover                  1
amount_ratio       value=11.3       deviation 5.3           2
log_dist           value=4.32       deviation 1.0
new_device         value=1          deviation 1.0           3
```

1. **Highest-scored takeover**
   Probability 0.998.

2. **11.3× the customer’s usual spend**
   The dominant reason by far.

3. **Plus distance and a new device**

EXPLAINABILITY

Turning reason codes into an analyst-ready alert

ALERT · HIGH | score 0.998 | Account takeover suspected

Customer 1327 · online retail · £103.64 · 21 March 2026, 09:40



Spend 11.3× this customer's usual



Unusually far from home



First payment from this device

Suggested action: step-up authentication; contact customer via known channel.

WHAT THE OUTPUT SHOWS

Real numbers from the previous slide, presented as an analyst would want them: verdict, evidence ranked by strength, suggested next step.

PRO TIP

Keep reason wording consistent and plain — the same phrases can feed customer communications and audit logs.

EXPLAINABILITY

Partial dependence: how does one feature move the score?

PYTHON

```
for r in [0.5, 1, 2, 4, 8, 16]:
    Xp = X_te.copy()
    Xp[:, FEATS.index("amount_ratio")] = r                    1
    print("amount_ratio = %5.1f  -> avg P(fraud) %.4f" % (
        r, hgb.predict_proba(Xp)[:, 1].mean()))
```

OUTPUT

```
amount_ratio =   0.5   -> avg P(fraud) 0.0175
amount_ratio =   1.0   -> avg P(fraud) 0.0226                 2
amount_ratio =   2.0   -> avg P(fraud) 0.0767
amount_ratio =   4.0   -> avg P(fraud) 0.1159                 3
amount_ratio =   8.0   -> avg P(fraud) 0.1749
amount_ratio =  16.0   -> avg P(fraud) 0.1724                 4
```

1 Force one value on everyone

Set amount_ratio to the same value for every test payment, re-score, and average.

2 Up to 2×: little effect

Average fraud probability barely moves.

3 4× to 8×: risk climbs steeply

4 Beyond 8×: flat

The model has learned a ceiling; extreme spend adds no extra suspicion.

EXPLAINABILITY

Partial dependence on spend ratio

average predicted fraud probability



WHAT THE OUTPUT SHOWS

A sensible, monotone shape: more unusual spend means more risk, then a plateau.

PRO TIP

A jagged or reversed curve here would be a red flag for overfitting or leakage.

EXPLAINABILITY

Key takeaways

01 Explanations serve analysts, customers, regulators and model builders.

02 Permutation importance shows global reliance — and concentration risk.

03 Reason codes turn a score into evidence an analyst can act on.

04 Partial dependence checks that the model behaves sensibly.

05 One dominant feature (distance here) is a vulnerability worth monitoring.

06 Consistent, plain-language reasons feed cases, customers and audits.

PART 14

Threat-Intelligence Case Study

The same toolkit, pointed at a security operations centre. Four data sources, seven planted attack behaviours, one prioritised alert queue.

IN THIS PART

- Authentication logs: spraying, brute force, impossible travel
- Network telemetry: exfiltration and scanning
- Global versus contextual anomaly detection
- Beaconing by regularity
- DGA domains: supervised and unsupervised
- ATT&CK mapping and triage

THREAT-INTELLIGENCE CASE STUDY

The SOC dataset blueprint

Authentication

27,000 normal logins by 300 users over 30 days (June 2026), plus a password spray, a brute-force attack and an impossible-travel login.

Network flows

108,000 hourly summaries for 150 hosts: bytes out, connections, distinct destinations and ports. Plus exfiltration, a port scan and a noisy host.

Connections

7,480 timestamped connections across 61 host–destination pairs, one of them a command-and-control beacon.

DNS names

4,000 legitimate-style domain names and 4,000 algorithmically generated ones.

PRO TIP

IP addresses used for attackers come from the ranges reserved for documentation (203.0.113.0/24, 198.51.100.0/24, 192.0.2.0/24), so no real organisation is implicated.

Normal authentication activity

PYTHON

```
rng = np.random.default_rng(11)
USERS = [f"user{i:03d}" for i in range(300)]                          1
DAYS, t0 = 30, pd.Timestamp("2026-06-01")
n = 300 * DAYS * 3
auth = pd.DataFrame({
    "user": rng.choice(USERS, n),
    "ts": (t0 + pd.to_timedelta(rng.integers(0, DAYS, n), "D")
           + pd.to_timedelta(rng.normal(10, 2.5, n).clip(6, 20),     2
                             "h")).floor("s"),
    "src_ip": [f"10.0.{a}.{b}" for a, b in
               rng.integers(1, 40, (n, 2))],
    "country": "GB",
    "success": rng.binomial(1, 0.96, n)})                            3
print(auth.shape)
print(auth.head(3).to_string(index=False))
```

OUTPUT

```
(27000, 5)
user                 ts          src_ip country  success                 4
user040 2026-06-20 09:26:34  10.0.2.32      GB        1
user038 2026-06-10 11:36:42 10.0.14.29      GB        1
user239 2026-06-28 10:53:56 10.0.15.23      GB        1
```

1 300 users, 30 days

Three logins per user per day on average: 27,000 events.

2 Office hours

Login times centred on 10:00, clipped to 06:00–20:00.

3 Internal addresses, UK, 96% success

Ordinary typos give a 4% failure rate.

4 One row per login attempt

Attack 1 — password spraying

PYTHON

```
spray_t = t0 + pd.Timedelta(days=12, hours=2)
spray = pd.DataFrame({
    "user": rng.choice(USERS, 260, replace=False),                    1
    "ts": spray_t + pd.to_timedelta(np.sort(
          rng.integers(0, 7200, 260)), "s"),
    "src_ip": "203.0.113.45", "country": "NL",
    "success": 0})
spray.loc[spray.index[[57, 190]], "success"] = 1  # 2 weak pwds    2
print(spray.shape, "| successes:", int(spray.success.sum()))
```

OUTPUT

```
(260, 5) | successes: 2                                                3
```

1 260 different accounts

One attempt each, from one external address over two hours, starting at 02:00.

2 Two weak passwords

Two accounts actually let the attacker in — the reason spraying works.

3 260 attempts, 2 successes

Attacks 2 and 3 — brute force and impossible travel

PYTHON

```
bt = t0 + pd.Timedelta(days=20, hours=23)
brute = pd.DataFrame({
    "user": "user042",                                                    1
    "ts": bt + pd.to_timedelta(np.arange(450) * 7, "s"),
    "src_ip": "198.51.100.23", "country": "RU",
    "success": 0})
brute.loc[brute.index[-1], "success"] = 1   # finally cracked
travel = pd.DataFrame({"user": ["user117", "user117"],                    2
    "ts": [t0 + pd.Timedelta(days=8, hours=9),
           t0 + pd.Timedelta(days=8, hours=9, minutes=40)],
    "src_ip": ["10.0.3.7", "192.0.2.99"],
    "country": ["GB", "BR"], "success": [1, 1]})
auth = (pd.concat([auth, spray, brute, travel])
          .sort_values("ts").reset_index(drop=True))
print(auth.shape)
```

OUTPUT

```
(27712, 5)                                                                3
```

1 450 guesses against user042

One every 7 seconds, late at night, from a foreign address. The last guess succeeds.

2 Impossible travel

user117 logs in from the UK, then from Brazil 40 minutes later — no flight is that fast.

3 27,712 events in total

Behaviour per source address

PYTHON

```python
ip = auth.groupby("src_ip").agg(                                    # 1
    attempts=("success", "size"),
    failures=("success", lambda s: int((s == 0).sum())),
    users=("user", "nunique"),
    countries=("country", "nunique"))
ip["fail_ratio"] = (ip.failures / ip.attempts).round(3)             # 2
print(ip.sort_values("failures", ascending=False).head(6)
        .drop(columns="countries"))
```

OUTPUT

src_ip	attempts	failures	users	fail_ratio	
198.51.100.23	450	449	1	0.998	3
203.0.113.45	260	258	260	0.992	
10.0.4.3	25	5	25	0.200	4
10.0.4.25	18	4	17	0.222	
10.0.1.4	18	4	18	0.222	
10.0.12.4	20	4	19	0.200	

1 Aggregate by source IP

Attempts, failures, distinct users and countries per address.

2 Failure ratio

3 Two addresses stand out instantly

449 failures against one user, and 258 failures across 260 users — two different shapes of attack.

4 Normal shared addresses

Internal IPs used by many staff: many users, few failures.

Behavioural rules mapped to ATT&CK

PYTHON

```
spray_hits = ip[(ip.users >= 20) & (ip.fail_ratio > 0.8)]                    1
print("Password-spray suspects (T1110.003):")
print(spray_hits.drop(columns="countries"))
brute_hits = ip[(ip.users <= 2) & (ip.failures >= 50)]                       2
print("\nBrute-force suspects (T1110.001):")
print(brute_hits.drop(columns="countries"))
```

OUTPUT

```
Password-spray suspects (T1110.003):
attempts      failures      users      fail_ratio
src_ip
203.0.113.45          260          258          260          0.992          3

Brute-force suspects (T1110.001):
attempts      failures      users      fail_ratio
src_ip
198.51.100.23         450          449          1            0.998
```

1 Spray signature

Many distinct users (≥ 20) and mostly failures (> 80%).

2 Brute-force signature

Few users (≤ 2) but many failures (≥ 50).

3 Both attacks found, nothing else

Precise rules work when the behaviour is well understood. These encode TTPs, not IOCs — the attacker cannot dodge them by changing IP address.

The same question, without rules

PYTHON

```
from sklearn.ensemble import IsolationForest
F = np.log1p(ip[["attempts", "failures", "users"]]).assign(
        fail_ratio=ip.fail_ratio)
ipf = IsolationForest(n_estimators=200, random_state=0).fit(F)
ip["score"] = -ipf.score_samples(F)
print(ip.sort_values("score", ascending=False)
        .head(5)[["attempts", "users", "fail_ratio", "score"]])
```

1

OUTPUT

src_ip	attempts	users	fail_ratio	score	
198.51.100.23	450	1	0.998	0.808	2
203.0.113.45	260	260	0.992	0.798	
192.0.2.99	1	1	0.000	0.746	3
10.0.35.1	6	6	0.167	0.736	
10.0.16.18	6	6	0.000	0.721	

1 Isolation Forest on IP behaviour

Log counts plus failure ratio; no thresholds written by hand.

2 Both attackers ranked first and second

Scores 0.808 and 0.798.

3 Third place: 192.0.2.99

The single foreign login of the impossible-travel event — found without anyone writing a travel rule.

Failures per user per hour

PYTHON

```
w = (auth[auth.success == 0].set_index("ts")
      .groupby("user").resample("1h").size()
      .rename("fails_per_hour").reset_index())
print(w.fails_per_hour.describe().round(2).to_string())
print(w.sort_values("fails_per_hour").tail(3)
      .to_string(index=False))
```

1

OUTPUT

```
count    116214.00
mean          0.02
std           1.32
min           0.00
25%           0.00
50%           0.00
75%           0.00
max         449.00
user                    ts  fails_per_hour
user285 2026-06-05 09:00:00               2
user086 2026-06-11 08:00:00               2
user042 2026-06-21 23:00:00             449
```

2

3

1 Hourly buckets per user

resample('1h') counts failures in each hour for each user.

2 Almost always zero

Median and upper quartile are both 0; the average is 0.02.

3 449 in one hour

user042 at 23:00 on 21 June — the brute-force attack, two hundred times the next highest value.

Impossible travel — and a surprise

PYTHON

```
ok = auth[auth.success == 1].sort_values(["user", "ts"])
ok = ok.assign(prev_country=ok.groupby("user").country.shift(),                1
               prev_ts=ok.groupby("user").ts.shift())
gap_h = (ok.ts - ok.prev_ts).dt.total_seconds() / 3600
imp = ok[(ok.country != ok.prev_country)
         & ok.prev_country.notna() & (gap_h < 4)]                              2
print(imp[["user", "prev_country", "country", "ts"]]
      .to_string(index=False))
```

OUTPUT

user	prev_country	country	ts	
user117	GB	BR	2026-06-09 09:40:00	3
user117	BR	GB	2026-06-09 10:16:19	
user297	NL	GB	2026-06-13 07:01:39	4

1 Consecutive successful logins

For each user, compare each successful login with the previous one.

2 Country change within four hours

3 user117: UK → Brazil in 40 minutes

The planted event — and the return trip half an hour later.

4 user297: Netherlands → UK

Not planted as travel! user297's previous success was the *password spray* from 203.0.113.45. This account was compromised by the spray.

Detectors cross-check each other

The impossible-travel query independently exposed an account the password spray had compromised. Correlation across detectors turns weak signals into confident findings.

Success after a burst of failures

PYTHON

```
# success shortly after many failures = possible compromise
a = auth.sort_values(["user", "ts"]).reset_index(drop=True)
a["fail"] = 1 - a.success
r = a.groupby("user").rolling("1h", on="ts").fail.sum()                1
a["fails_1h"] = r.values - a.fail       # failures before it
sus = a[(a.success == 1) & (a.fails_1h >= 10)]                         2
print(sus[["user", "ts", "src_ip", "country", "fails_1h"]]
        .to_string(index=False))
```

OUTPUT

```
user                  ts             src_ip country  fails_1h
user042 2026-06-21 23:52:23 198.51.100.23      RU     449.0   3
```

1 Failures in the hour before

A time-based rolling window per user, excluding the current attempt.

2 Success with ≥ 10 prior failures

3 One hit: the cracked password

user042 from 198.51.100.23 (RU) after 449 failures — a probable account compromise requiring immediate reset.

THREAT-INTELLIGENCE CASE STUDY

From logins to network behaviour

Authentication shows who got in. Network telemetry shows what they did next. Each host gets one summary row per hour:

bytes_out
data sent from the host

conns
number of connections

dst_ips
distinct destinations

dst_ports
distinct destination ports

KEY INSIGHT

Work hours (08:00–18:00) are far busier than nights, so ‘high’ must always be judged against the host’s normal *at that hour*.

Simulating 150 hosts for 30 days

PYTHON

```
HOSTS = [f"host-{i:03d}" for i in range(150)]
idx = pd.MultiIndex.from_product([HOSTS, range(30 * 24)],
                                 names=["host", "hour_idx"])
net = pd.DataFrame(index=idx).reset_index()
hod = net.hour_idx % 24
work = ((hod >= 8) & (hod <= 18)).astype(float)
k = len(net)
net["bytes_out"] = (rng.lognormal(13 + 2 * work, 0.7, k)).round()
net["conns"] = rng.poisson(15 + 60 * work)
net["dst_ips"] = rng.poisson(4 + 12 * work) + 1
net["dst_ports"] = rng.poisson(2 + 3 * work) + 1
print(net.shape)
print(net.describe().loc[["mean", "50%", "max"]].round(0))
```

OUTPUT

```
(108000, 6)
```

hour_idx	bytes_out	conns	dst_ips	dst_ports	
mean	360.0	2.208e+06	43.0	10.0	4.0
50%	360.0	1.013e+06	21.0	8.0	4.0
max	719.0	6.045e+07	114.0	37.0	17.0

1 One row per host per hour

150 × 720 = 108,000 rows.

2 Work-hours effect

Every measure is boosted between 08:00 and 18:00.

3 Typical hour: ~1 MB out

Median 1.0 MB; the largest normal hour is 60 MB.

Planting three network attacks

PYTHON

```
def hit(host, day, hours, **vals):
    m = (net.host == host) & net.hour_idx.isin(
        [day * 24 + h for h in hours])
    for c, v in vals.items():
        net.loc[m, c] = v
hit("host-077", 25, [1, 2, 3], bytes_out=4.5e9, dst_ips=1)   # 1
hit("host-033", 18, [14], dst_ports=950, dst_ips=230,        # 2
    conns=2400)
hit("host-101", 9, range(0, 24), conns=600, dst_ips=1)       # 3
print("Injected: exfiltration, port scan, noisy beacon host")
```

OUTPUT

```
Injected: exfiltration, port scan, noisy beacon host
```

1 Exfiltration

host-077 sends 4.5 GB per hour to a single destination, 01:00–03:59 on day 25.

2 Port scan

host-033 touches 950 ports on 230 destinations in one afternoon hour.

3 Noisy host

host-101 opens 600 connections per hour to one destination all day on day 9.

Per-host, per-hour baselines (robust)

PYTHON

```
net["hod"] = net.hour_idx % 24
net["log_bytes"] = np.log1p(net.bytes_out)
base = net.groupby(["host", "hod"]).log_bytes.agg(["median"])
mad = net.groupby(["host", "hod"]).log_bytes.agg(
    lambda s: (s - s.median()).abs().median() * 1.4826)
net = net.join(base, on=["host", "hod"]).join(
    mad.rename("mad"), on=["host", "hod"])
net["bytes_rz"] = (net.log_bytes - net["median"]) / net.mad
print(net.sort_values("bytes_rz").tail(4)
      [["host", "hour_idx", "bytes_out", "bytes_rz"]]
      .to_string(index=False))
```

OUTPUT

host	hour_idx	bytes_out	bytes_rz	
host-079	366	4.152e+06	7.941	4
host-077	603	4.500e+09	15.181	
host-077	602	4.500e+09	17.373	3
host-077	601	4.500e+09	19.568	

1 Baseline per host and hour of day

Median and median absolute deviation of log bytes for each of 150 × 24 combinations.

2 Robust z-score

How many typical deviations above this host's normal for this hour?

3 Exfiltration: deviations of 15–20

Three consecutive hours of host-077 top the list.

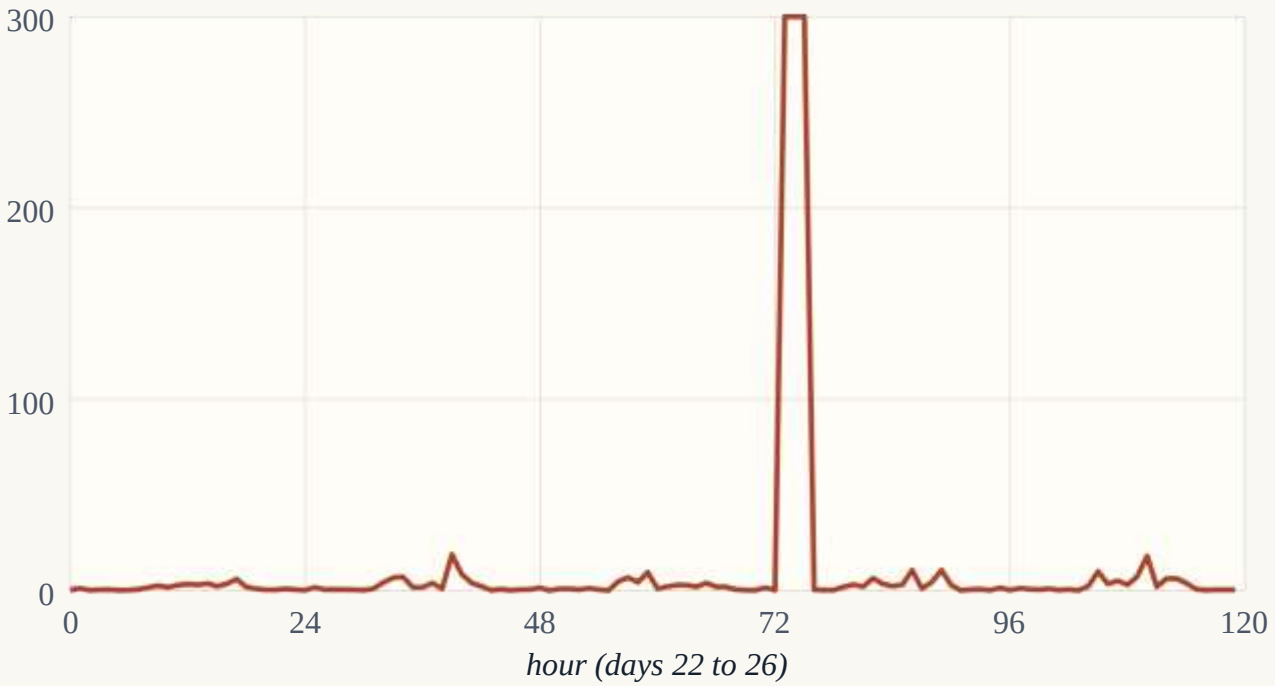
4 The next is a far weaker 7.9

A one-off busy hour — a gap that makes the real event unmistakable.

THREAT-INTELLIGENCE CASE STUDY

host-077: five days of outbound traffic

MB sent per hour (capped at 300 for display)



WHAT THE OUTPUT SHOWS

Daily humps are ordinary working hours (a few MB).

KEY INSIGHT

On day 25 the line hits the ceiling: 4,500 MB per hour in the small hours — over a thousand times this host's normal night traffic.

Global Isolation Forest on network features

PYTHON

```
cols = ["log_bytes", "conns", "dst_ips", "dst_ports"]
Xn = net[cols].assign(conns=np.log1p(net.conns),
                      dst_ips=np.log1p(net.dst_ips),
                      dst_ports=np.log1p(net.dst_ports))
nif = IsolationForest(n_estimators=300, contamination=0.0005,
                      random_state=0).fit(Xn)
net["if_score"] = -nif.score_samples(Xn)
net["if_flag"] = nif.predict(Xn) == -1
print("flagged host-hours:", int(net.if_flag.sum()))
print(net[net.if_flag].groupby("host").size()
      .sort_values(ascending=False).head(6).to_string())
```

1

OUTPUT

```
flagged host-hours: 54
host
host-101    8
host-023    3
host-077    3
host-099    3
host-026    2
host-114    2
```

2

3

1 Raw behaviour, all hosts together

Four log-scaled features; contamination 0.05% → 54 flags.

2 host-101 dominates

8 of its 24 noisy hours are flagged.

3 Exfiltration only partly

3 flags for host-077 — but mixed with many ordinary hosts.

Where are the port scan and the exfiltration?

PYTHON

```
top = net.sort_values("if_score", ascending=False).head(8)
top = top.assign(day=top.hour_idx // 24)
print(top[["host", "day", "hod", "dst_ports", "dst_ips",
           "if_score"]].to_string(index=False))
```

OUTPUT

host	day	hod	dst_ports	dst_ips	if_score
host-101	9	23	1	1	0.689
host-101	9	6	1	1	0.684
host-023	29	6	7	1	0.681
host-014	4	14	1	23	0.677
host-063	6	11	1	11	0.676
host-077	25	3	4	1	0.672
host-004	10	7	1	7	0.671
host-101	9	13	8	1	0.671

1 Top eight global anomalies

No port scan in sight

host-033's 950-port hour is missing from the top.

3 Exfiltration in sixth place

Below several ordinary hosts that simply had unusual-but-harmless hours.

4 Why

Isolation Forest scores saturate: once a point is extreme, *how* extreme stops mattering, and global normal ranges mix quiet and busy hosts.

Contextual Isolation Forest — the code

PYTHON

```
# Contextual features: deviation from the host's OWN
# behaviour at that hour of day (robust z, clipped)
def ctx(col):                                                          (1)
    v = np.log1p(net[col])
    g = v.groupby([net.host, net.hod])
    med = g.transform("median")
    mad = g.transform(lambda s: (s - s.median()).abs()
                      .median()).replace(0, 0.1) * 1.4826
    return ((v - med) / mad).clip(-50, 50)                             (2)
C = pd.DataFrame({c: ctx(c) for c in ["bytes_out", "conns",
                  "dst_ips", "dst_ports"]})
cif = IsolationForest(n_estimators=300, random_state=0).fit(C)
net["ctx_score"] = -cif.score_samples(C)
top = net.sort_values("ctx_score", ascending=False).head(8)
print(top.assign(day=top.hour_idx // 24)[["host", "day", "hod",
      "ctx_score"]].to_string(index=False))
```

1 Deviation from the host’s own normal

Each feature becomes a robust z-score against the same host at the same hour of day.

2 Clipped at ±50

Keeps a single astronomical value from dominating.

Contextual Isolation Forest — the output

PYTHON · LAST LINES OF THE CELL ABOVE

```
top = net.sort_values("ctx_score", ascending=False).head(8)
print(top.assign(day=top.hour_idx // 24)[["host", "day", "hod",
      "ctx_score"]].to_string(index=False))
```

OUTPUT

host	day	hod	ctx_score	
host-033	18	14	0.701	3
host-101	9	8	0.699	4
host-020	15	18	0.695	
host-042	29	6	0.687	
host-101	9	17	0.682	
host-032	11	13	0.681	
host-054	7	20	0.677	
host-051	20	14	0.676	

3 Port scan now first

4 Noisy host second

Exfiltration is still not in the top eight — scores saturate again.

An explainable companion: the largest deviation

PYTHON

```
# Simple, explainable companion: the single most extreme
# contextual deviation in each host-hour, and which signal
net["max_dev"] = C.abs().max(axis=1).round(1)
net["driver"] = C.abs().idxmax(axis=1)
top = net.sort_values("max_dev", ascending=False)
top = top.drop_duplicates(["host", "driver"]).head(6)
print(top.assign(day=top.hour_idx // 24)[["host", "day", "hod",
      "driver", "max_dev"]].to_string(index=False))
```

OUTPUT

host	day	hod	driver	max_dev
host-033	18	14	conns	33.2
host-101	9	13	conns	26.0
host-077	25	1	bytes_out	19.6
host-073	5	14	dst_ips	12.5
host-078	21	16	dst_ports	10.3
host-086	11	14	dst_ports	10.3

1 Max absolute deviation

For each host-hour, the most extreme of the four contextual scores…

2 …and which signal drove it

3 All three attacks on top

Port scan (conns), noisy host (conns), exfiltration (bytes_out) — each with a plain-language reason.

4 Next: ordinary noise

Deviations of 10–12 from benign hosts form the background.

THREAT-INTELLIGENCE CASE STUDY

Layered detection beats any single model

Port scan (host-033)	not in top 8	rank 1	rank 1 (conns)
Noisy host (host-101)	rank 1	rank 2	rank 2 (conns)
Exfiltration (host-077)	rank 6	not in top 8	rank 3 (bytes_out)

KEY INSIGHT

Context (per-host, per-hour baselines) mattered more than the choice of algorithm.

PRO TIP

The simplest method — largest deviation — ranked every attack in the top three *and* said why. Always benchmark simple against clever.

Simulating connections and a beacon

PYTHON

```
conn = []
for i in range(60):                    # normal pairs                    1
    t_ = np.cumsum(rng.exponential(rng.uniform(200, 2000), 120))
    conn.append(pd.DataFrame({"src": f"host-{i:03d}",
        "dst": f"52.{i}.10.1", "t": t_}))
beacon = 300 + rng.normal(0, 6, 280)      # every ~5 min                    2
conn.append(pd.DataFrame({"src": "host-101",
    "dst": "185.220.1.9", "t": np.cumsum(beacon)}))
conn = pd.concat(conn, ignore_index=True)
print(conn.shape, "| pairs:", conn.groupby(["src", "dst"]).ngroups)
```

OUTPUT

```
(7480, 3) | pairs: 61                                                        3
```

1 60 normal host–destination pairs

Random, irregular gaps (exponentially distributed) between connections.

2 One beacon

host-101 → 185.220.1.9 every 300 seconds with a tiny ±6-second jitter, 280 times.

3 7,480 connections, 61 pairs

Measuring regularity

PYTHON

```
conn = conn.sort_values(["src", "dst", "t"])
conn["gap"] = conn.groupby(["src", "dst"]).t.diff()                    1
pairs = conn.groupby(["src", "dst"]).gap.agg(
    n="count", mean_gap="mean", std_gap="std")
pairs["regularity"] = (pairs.std_gap / pairs.mean_gap).round(3)        2
print(pairs.sort_values("regularity").head(4)
          .round(1).to_string())
```

OUTPUT

src	dst	n	mean_gap	std_gap	regularity	
host-101	185.220.1.9	279	299.6	6.2	0.0	3
host-037	52.37.10.1	119	1910.6	1504.8	0.8	4
host-057	52.57.10.1	119	1966.0	1666.7	0.8	
host-026	52.26.10.1	119	1565.6	1340.4	0.9	

1 Gaps between connections

Per host–destination pair.

2 Regularity = spread ÷ average gap

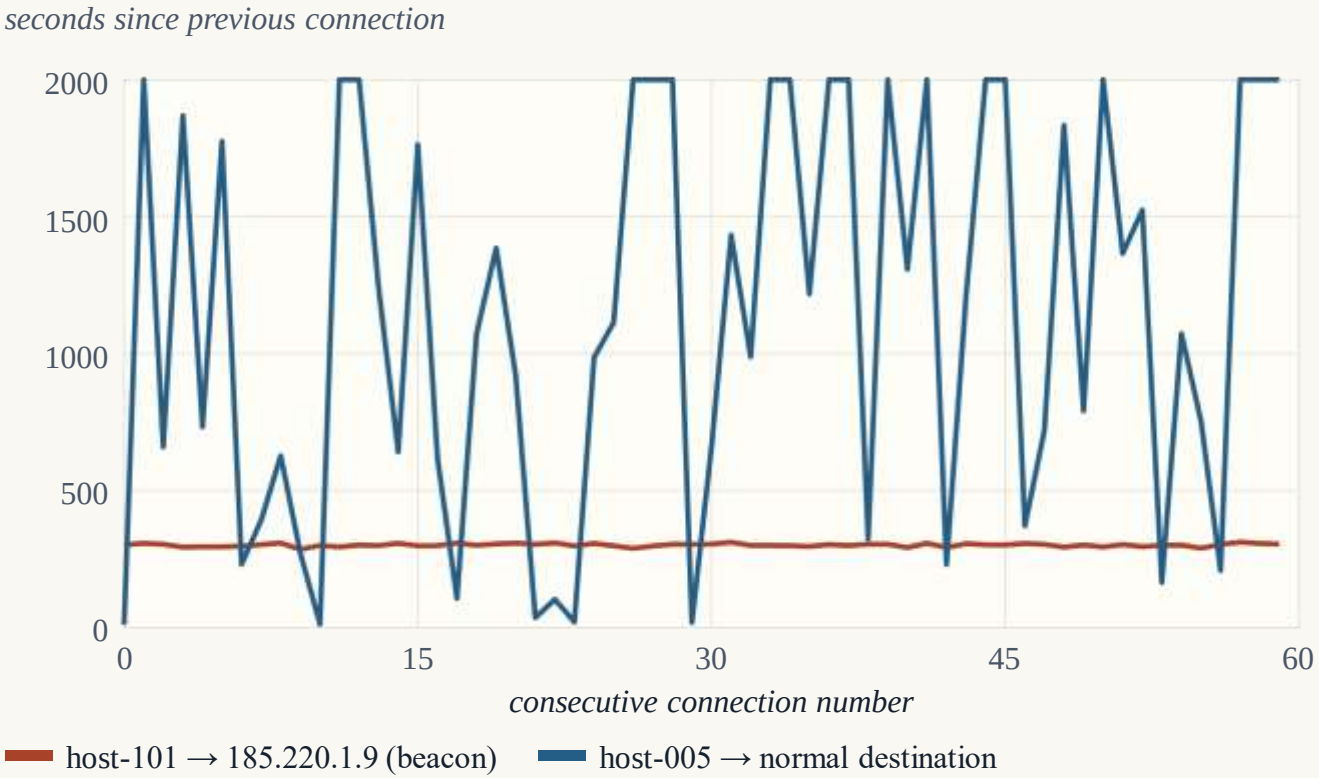
Near 1 for random human-like traffic; near 0 for a metronome.

3 The beacon: 0.02 (shown rounded as 0.0)

Average gap 299.6 s with only 6.2 s of variation.

4 Normal pairs: 0.8 and above

Connection gaps: beacon versus normal



WHAT THE OUTPUT SHOWS

The beacon’s line is flat at about 300 seconds: a timer.

KEY INSIGHT

Normal traffic jumps around unpredictably. Regularity captures this difference in a single number.

Flagging likely command and control

PYTHON

```
beacons = pairs[(pairs.regularity < 0.1) & (pairs.n > 50)]
print("Likely C2 beacons (T1071 / T1573):")
print(beacons.round(1).to_string())
print("\nmedian regularity of normal pairs: %.2f" %
      pairs.regularity.median())
```

1

OUTPUT

```
Likely C2 beacons (T1071 / T1573):
                   n  mean_gap  std_gap  regularity
src        dst
host-101 185.220.1.9         279    299.6         6.2         0.0

median regularity of normal pairs: 0.98
```

2

3

1 Very regular and frequent

Regularity below 0.1 and more than 50 connections.

2 Exactly one pair flagged

The planted beacon — mapped to T1071 (application-layer protocol C2).

3 Normal median: 0.98

THREAT-INTELLIGENCE CASE STUDY

Beaconing in the real world

Attackers add jitter

Malware frameworks randomise sleep times (for example ±20–50%) to defeat exactly this test.

Legitimate software beacons too

Update checkers, monitoring agents and chat apps poll on timers — keep an allow-list.

KEY INSIGHT

Combine regularity with other evidence: rare destination, young domain, small fixed-size payloads, activity outside business hours.

PRO TIP

Frequency analysis of gap histograms can still reveal a hidden period under random jitter.

Building legitimate and generated domain names

PYTHON

```
SYL = ["con", "net", "sol", "tech", "mail", "data", "shop", "bank",
       "cloud", "news", "home", "web", "sky", "media", "pay",
       "go", "book", "smart", "trip", "health", "green", "box"]
TLD = [".com", ".co.uk", ".net", ".org", ".io"]
def benign(n):                                                          # 1
    return ["".join(rng.choice(SYL, rng.integers(1, 4)))
            + rng.choice(TLD) for _ in range(n)]
def dga(n):                                                             # 2
    al = list("abcdefghijklmnopqrstuvwxyz0123456789")
    return ["".join(rng.choice(al, rng.integers(10, 22)))
            + rng.choice([".com", ".net", ".info", ".xyz"])
            for _ in range(n)]
dom = pd.DataFrame({"domain": benign(4000) + dga(4000),
                    "is_dga": [0] * 4000 + [1] * 4000})
print(dom.sample(6, random_state=1).to_string(index=False))
```

OUTPUT

```
domain     is_dga                                                       # 3
mstkego6ffupua8mfo6yc.net       1
56jbppxh0o2oqyq.net             1
57uh2rf5ivqpna8bu0a.net         1
a7cs9w88nxg4.com                1
solboxmail.co.uk                0
sky.com                         0
```

1 Legitimate-style names

One to three syllables from a word list: ‘solboxmail’, ‘sky’.

2 DGA-style names

10–21 random letters and digits.

3 A random sample

Describing a domain name with numbers — the code

PYTHON

```
from collections import Counter
def char_entropy(s):                                                    1
    c = np.array(list(Counter(s).values())) / len(s)
    return float(-(c * np.log2(c)).sum())
def feats(d):
    name = d.split(".")[0]
    vow = sum(ch in "aeiou" for ch in name)
    runs = max(len(r) for r in
               "".join(" " if ch in "aeiou0123456789" else "x"
                       for ch in name).split()) if name else 0
    return dict(length=len(name),                                       2
                digits=sum(ch.isdigit() for ch in name) / len(name),
                vowels=vow / len(name), entropy=char_entropy(name),
                consonant_run=runs, distinct=len(set(name)))
D = pd.DataFrame([feats(d) for d in dom.domain])
print(D.groupby(dom.is_dga).mean().round(2).T)
```

1 Character randomness (entropy)

How unpredictable the next character is: high for random strings, lower for words.

2 Six lexical features

Length, share of digits, share of vowels, randomness, longest consonant run, distinct characters.

Describing a domain name with numbers — the output

PYTHON · LAST LINES OF THE CELL ABOVE

```
            consonant_run=runs, distinct=len(set(name)))
D = pd.DataFrame([feats(d) for d in dom.domain])
print(D.groupby(dom.is_dga).mean().round(2).T)
```

OUTPUT

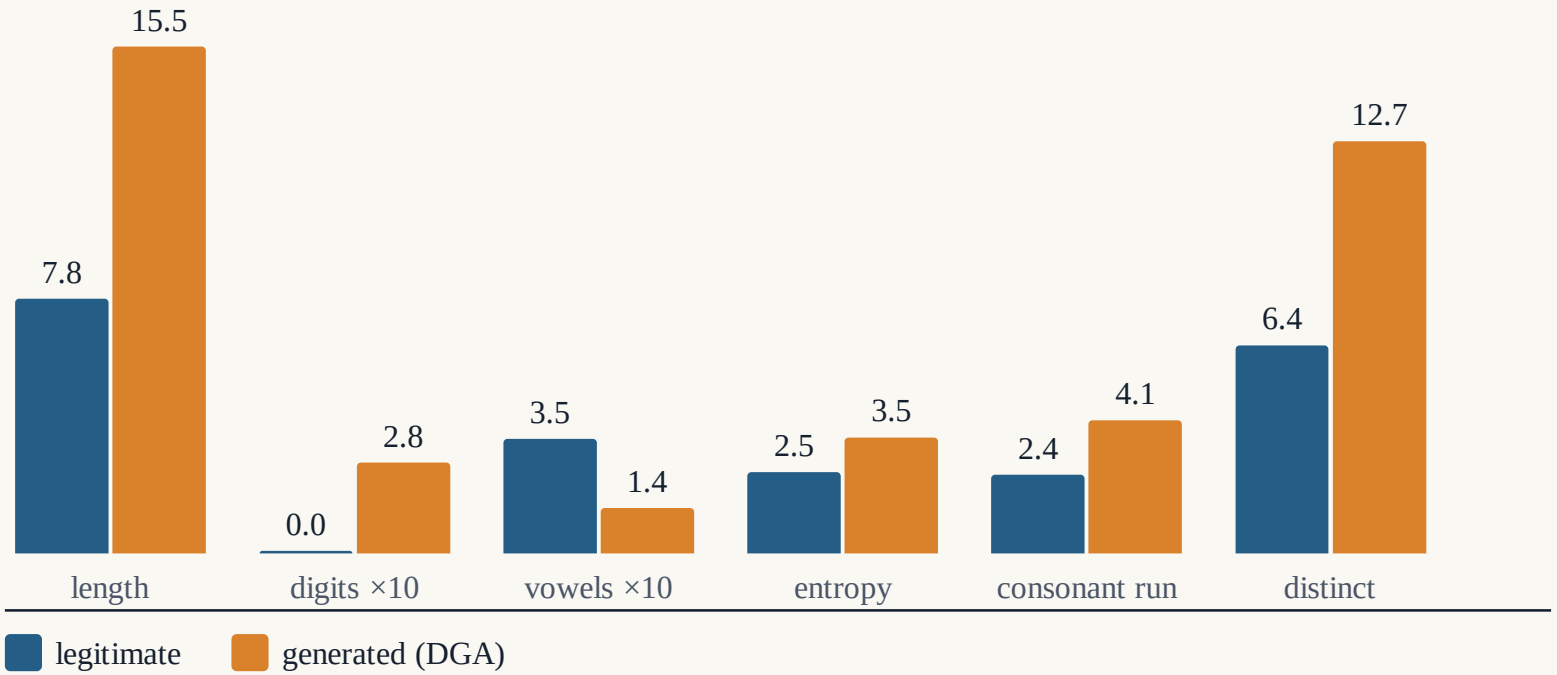
is_dga	0	1
length	7.80	15.54
digits	0.00	0.28
vowels	0.35	0.14
entropy	2.50	3.55
consonant_run	2.41	4.10
distinct	6.38	12.65

3

3 Clear separation on average

Generated names are twice as long, 28% digits, half the vowels and much more random.

Legitimate versus generated: feature averages



WHAT THE OUTPUT SHOWS

Every feature separates the groups; digits and vowels most cleanly.

PRO TIP

Shares of digits and vowels are multiplied by 10 so they are visible on the same axis.

A random-forest DGA classifier

PYTHON

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(                    1
    D, dom.is_dga, test_size=0.3, stratify=dom.is_dga,
    random_state=0)
dga_rf = RandomForestClassifier(n_estimators=200,
                                random_state=0).fit(Xd_tr, yd_tr)
print(classification_report(yd_te, dga_rf.predict(Xd_te),
      target_names=["benign", "dga"], digits=3))
```

OUTPUT

```
              precision    recall  f1-score   support

      benign      0.993     0.999     0.996      1200
         dga      0.999     0.993     0.996      1200                2

    accuracy                          0.996      2400                3
   macro avg      0.996     0.996     0.996      2400
weighted avg      0.996     0.996     0.996      2400
```

1 Stratified 70/30 split

Here a random split is fine: each domain is independent and there is no time order.

2 99.6% precision and recall

3 Too easy?

Our generated names are purely random. Real DGA families include *dictionary* DGAs that join real words — far harder, needing character n-gram or neural models.

What the DGA model relies on

PYTHON

```
imp = pd.Series(dga_rf.feature_importances_, index=D.columns)
print(imp.sort_values(ascending=False).round(3).to_string())
```

OUTPUT

```
digits          0.523
vowels          0.207
entropy         0.113
distinct        0.108
length          0.040
consonant_run   0.009
```

1 Digits dominate

52% of the model's importance: legitimate brand names rarely contain digits.

2 Vowels second

Pronounceability matters.

3 Built-in importance

Random forests report how much each feature reduced impurity — a quick but biased measure; permutation importance (Part 13) is more reliable.

Scoring brand-new domains

PYTHON

```
new = ["paybooknews.com", "x7k2q9zr4mfp1.xyz",                    (1)
       "greencloudshop.io", "qwhzkrtpvbn.info",
       "bankhome.co.uk", "a1b2c3d4e5f6g7.net"]
pr = dga_rf.predict_proba(pd.DataFrame([feats(d) for d in new]))
for d, p_ in zip(new, pr[:, 1]):
    print("%-22s P(DGA)=%.2f  %s" % (d, p_,
          "BLOCK" if p_ > .5 else "allow"))
```

OUTPUT

```
paybooknews.com        P(DGA)=0.00   allow     (2)
x7k2q9zr4mfp1.xyz      P(DGA)=1.00   BLOCK
greencloudshop.io      P(DGA)=0.00   allow
qwhzkrtpvbn.info       P(DGA)=0.91   BLOCK     (3)
bankhome.co.uk         P(DGA)=0.00   allow
a1b2c3d4e5f6g7.net     P(DGA)=1.00   BLOCK
```

1 Six unseen names

2 Real-word names allowed

Probability 0.00.

3 Random strings blocked

Including qwhzkrtpvbn.info, which has no digits at all: vowels and consonant runs give it away (0.91).

No labels? Learn legitimate names only

PYTHON

```
iso_d = IsolationForest(random_state=0).fit(D[dom.is_dga == 0])
sd = -iso_d.score_samples(D)
from sklearn.metrics import roc_auc_score
print("IForest trained on benign only, ROC-AUC: %.3f" %
      roc_auc_score(dom.is_dga, sd))
```

1

OUTPUT

```
IForest trained on benign only, ROC-AUC: 0.954
```

2

1 Train on benign names only

An Isolation Forest learns what normal names look like.

2 ROC-AUC 0.954

Without a single DGA example — useful when a brand-new malware family appears.

Mapping every detector to MITRE ATT&CK

PYTHON

```
ATTACK = {                                                          (1)
  "password spray":    ("T1110.003", "Credential Access"),
  "brute force":       ("T1110.001", "Credential Access"),
  "impossible travel":("T1078",      "Initial Access"),
  "port scan":         ("T1046",     "Discovery"),
  "C2 beaconing":      ("T1071",     "Command and Control"),
  "DGA domains":       ("T1568.002", "Command and Control"),
  "bulk exfiltration":("T1048",      "Exfiltration")}
for k_, (tid, tac) in ATTACK.items():
    print("%-18s %-10s %s" % (k_, tid, tac))
```

OUTPUT

password spray	T1110.003	Credential Access	(2)
brute force	T1110.001	Credential Access	
impossible travel	T1078	Initial Access	
port scan	T1046	Discovery	
C2 beaconing	T1071	Command and Control	
DGA domains	T1568.002	Command and Control	
bulk exfiltration	T1048	Exfiltration	

1 One dictionary, one source of truth

Detection name → technique ID → tactic.

2 Five tactics covered

Initial Access, Credential Access, Discovery, Command and Control, Exfiltration.

THREAT-INTELLIGENCE CASE STUDY

Our ATT&CK coverage map

Initial Access	T1078 impossible travel
Credential Access	T1110.001 brute force · T1110.003 spraying
Discovery	T1046 port scan
Command & Control	T1071 beaconing · T1568.002 DGA
Exfiltration	T1048 bulk transfer
Execution · Persistence · Lateral Movement	not covered by these data sources

KEY INSIGHT

Grey rows are gaps. Coverage maps tell a security leader where the next data source (endpoint telemetry, for example) would add the most value.

Turning detections into a prioritised queue

PYTHON

```
alerts = pd.DataFrame([
 ("password spray", "203.0.113.45", 2, 0.95),
 ("brute force", "user042", 1, 0.90),
 ("impossible travel", "user117", 1, 0.70),
 ("port scan", "host-033", 1, 0.80),
 ("C2 beaconing", "host-101", 280, 0.85),
 ("bulk exfiltration", "host-077", 3, 0.99)],
 columns=["detection", "entity", "evidence", "confidence"])                    1
IMPACT = {"Exfiltration": 5, "Credential Access": 4,                           2
          "Command and Control": 4, "Initial Access": 3,
          "Discovery": 2}
alerts["impact"] = [IMPACT[ATTACK[d][1]] for d in alerts.detection]
alerts["priority"] = (alerts.impact * alerts.confidence).round(2)              3
print(alerts.sort_values("priority", ascending=False)
      .drop(columns="evidence").to_string(index=False))
```

OUTPUT

detection	entity	confidence	impact	priority	
bulk exfiltration	host-077	0.99	5	4.95	4
password spray	203.0.113.45	0.95	4	3.80	5
brute force	user042	0.90	4	3.60	
C2 beaconing	host-101	0.85	4	3.40	
impossible travel	user117	0.70	3	2.10	
port scan	host-033	0.80	2	1.60	

1 Each finding gets a confidence

From the strength of the evidence (set by the analyst for this demo).

2 Impact by tactic

Exfiltration is worst; discovery least severe.

3 Priority = impact × confidence

A simple, explainable ordering.

4 Exfiltration first

Data is leaving now: contain host-077 immediately.

5 Then credential attacks

Reset the sprayed and brute-forced accounts (and user297).

Enrichment with threat-intelligence feeds

PYTHON

```
ioc_feed = {"185.220.1.9": "known C2 (feed A)",
            "203.0.113.45": "spray infra (feed B)"}
for e in ["185.220.1.9", "203.0.113.45", "52.7.10.1"]:
    print("%-14s -> %s" % (e, ioc_feed.get(e, "no match")))
```

OUTPUT

```
185.220.1.9    -> known C2 (feed A)
203.0.113.45   -> spray infra (feed B)
52.7.10.1      -> no match
```

1 A stand-in for real feeds

In practice: commercial or open threat-intel platforms, sharing communities, internal case history.

2 Two behavioural findings confirmed

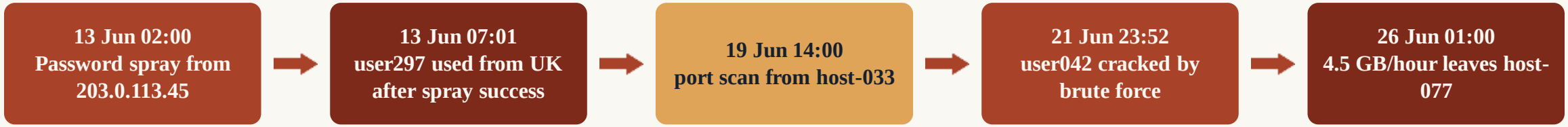
The beacon destination and the spray source are known bad infrastructure.

3 No match is not ‘safe’

Feeds only know yesterday’s infrastructure; behaviour found the attacks first.

THREAT-INTELLIGENCE CASE STUDY

The investigation story, reconstructed



KEY INSIGHT

Individually, each detector produced one alert. Together, on a timeline, they describe an intrusion from first access to data theft — the essence of threat-intelligence analysis.

In our simulation these events were planted independently; in a real case, analysts would now test whether they are linked (shared accounts, hosts and timing).

Check your understanding

Q1 **Why did the global Isolation Forest miss the port scan?**

Answer: Scores saturate and global ranges mix busy and quiet hosts; per-host, per-hour context fixed it.

Q2 **What single number exposes the beacon?**

Answer: Regularity: the spread of gaps divided by their average (0.02 versus ~1).

Q3 **How was user297's compromise found?**

Answer: The impossible-travel query linked it to a successful login from the spray address.

Q4 **Why might our DGA classifier overstate real-world accuracy?**

Answer: Real DGAs include dictionary-based names that look like words.

Key takeaways

01 Security telemetry rewards the same ideas as fraud: entity baselines and context.

02 Rules encode known TTPs precisely; anomaly models catch what rules did not anticipate.

03 Contextual (per-host, per-hour) features mattered more than the algorithm.

04 Simple, explainable scores (largest deviation, regularity) are often the best.

05 Map every detector to ATT&CK to see coverage and gaps.

06 Correlating detections on a timeline turns alerts into an intrusion story.

PART 15

Production and Monitoring

A model is only valuable once it runs reliably on live data — and keeps working as customers, attackers and systems change.

IN THIS PART

- Real-time scoring architecture
- Drift measured with PSI
- Monitoring alert rates
- Feedback loops and retraining
- Governance and model security

A real-time fraud-scoring architecture



Feature store

Keeps per-customer counters and medians up to date so real-time features cost milliseconds.

Decision engine

Combines scores with policy rules and costs; logs every decision and its reasons.

Case management

Routes alerts to analysts; verdicts flow back as labels for retraining.

Batch versus streaming

Batch (hourly / daily)

- Simpler to build and test
- Heavy features: graphs, long histories
- Good for post-authorisation review and SOC hunting

Streaming (event by event)

- Needed for approve/decline in milliseconds
- Features must be incremental counters
- Training–serving skew is the classic bug: features computed differently live than in training

Drift: the Population Stability Index (PSI)

KEY INSIGHT

Cut a feature’s training distribution into ten equal-sized bins. See what share of new data lands in each bin. If the shares have moved, the population has shifted.

WHAT THE OUTPUT SHOWS

PSI adds up those shifts into one number — zero means identical distributions.

below 0.1	stable
0.1 to 0.25	moderate shift — investigate
above 0.25	major shift — act

These bands are an industry rule of thumb, not a law; tune them to your data.

Measuring drift with PSI

PYTHON

```python
def psi(expected, actual, bins=10):
    cuts = np.quantile(expected, np.linspace(0, 1, bins + 1))
    cuts[0], cuts[-1] = -np.inf, np.inf
    e = np.histogram(expected, cuts)[0] / len(expected)
    a = np.histogram(actual, cuts)[0] / len(actual)
    e, a = np.clip(e, 1e-6, None), np.clip(a, 1e-6, None)
    return float(np.sum((a - e) * np.log(a / e)))
print("PSI log_amount train->test : %.4f" % psi(
      tr.log_amount, te.log_amount))
shifted = te.log_amount + 0.4         # simulated inflation shock
print("PSI after +40%% price shock: %.4f" % psi(
      tr.log_amount, shifted))
```

OUTPUT

```
PSI log_amount train->test : 0.0006
PSI after +40% price shock: 0.2495
```

1 Bins from the training data

Ten quantile bins, open-ended at both extremes.

2 Shares, then the PSI sum

Tiny floors avoid dividing by zero for empty bins.

3 Train → test: 0.0006

Spend distribution is stable between January–February and March.

4 A simulated +40% price shock: 0.25

Right at the ‘major shift’ line — exactly what monitoring should catch.

Drift across every numeric feature

PYTHON

```
rows = []
for c in NUM:
    rows.append((c, round(psi(tr[c], te[c]), 4)))
print(pd.DataFrame(rows, columns=["feature", "PSI"])
        .sort_values("PSI", ascending=False)
        .to_string(index=False, float_format="%.4f"))
```

OUTPUT

```
feature      PSI
amount_ratio 0.0176
   log_gap 0.0152
      hour 0.0007
  log_dist 0.0007
log_amount 0.0006
    tx_24h 0.0001
  is_night 0.0000
   foreign 0.0000
new_device 0.0000
     tx_1h 0.0000
 is_round 0.0000
```

1

2

1 All features stable

Every PSI is under 0.02.

2 Largest: amount_ratio

Slightly higher because customer medians were learned only on the training months.

3 PSI on features misses tactic changes

The card-testers' move to gambling barely moves any single feature distribution — another reason to monitor *outcomes* too.

Monitoring the alert rate itself

PYTHON

```
cut = np.quantile(s_iso, 0.99)
weekly = te.assign(a=s_iso > cut).groupby(
    (te.day // 7).rename("week")).a.mean()
print("share of txns above 99th pct score, per week:")
print((weekly * 100).round(2).to_string())
```

OUTPUT

```
share of txns above 99th pct score, per week:
week
8     0.78
9     1.16
10    1.27
11    0.52
12    1.16
```

1 Fixed threshold from the test month

The 99th percentile of the Isolation Forest score.

2 Share above it, per week

Should hover around 1% if behaviour is stable.

3 Week 11 dips to 0.52%

Worth a look: fewer anomalies, a data-feed problem, or attackers lying low? Alert-rate monitoring catches pipeline failures that no accuracy metric would.

Feedback loops and retraining

1 Schedule

Retrain on a regular cadence (e.g. monthly) and whenever drift or performance alarms fire.

2 Label delay

Exclude the most recent weeks whose labels are incomplete, or they teach the model that fraud is legitimate.

3 Selection bias

Labels exist mostly for alerted cases. Review a small random sample of unalerted events to see what you miss.

4 Champion–challenger

Run the new model in shadow mode beside the current one; promote only if it wins on recent data.

Governance: models as accountable systems

Model documentation

Purpose, data, features, metrics by segment, limitations and owners — a ‘model card’ for every detector.

Audit trail

Every decision logged with its score, reasons and model version, so any outcome can be explained later.

Fairness checks

Compare false-decline rates across customer groups and regions; avoid features that proxy for protected characteristics.

PRO TIP

Regulation increasingly requires this: data-protection rules on automated decisions, and risk-management expectations for models in financial services.

Securing the detector itself

Attackers probe fraud and security models just as they probe networks.

Evasion

Fraudsters keep amounts just under thresholds and spread attempts out; attackers add jitter to beacons.

Probing

Repeated small tests reveal what gets declined. Rate-limit and watch for testing behaviour.

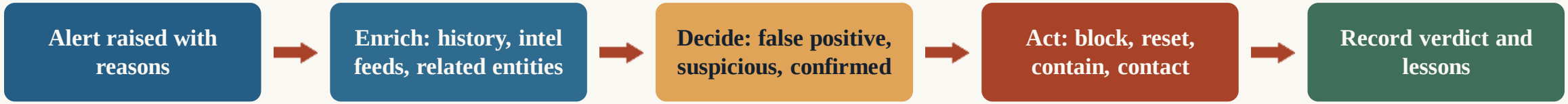
Poisoning

If feedback labels can be manipulated (e.g. false ‘not fraud’ confirmations), the model can be taught to ignore an attack.

Defences

Layered models, randomised thresholds, secret features, label verification and monitoring of model inputs.

An alert-handling runbook



PRO TIP

Time-box each stage. Unclear alerts escalate rather than linger.

KEY INSIGHT

The recorded verdict is tomorrow’s training label — capture it carefully and consistently.

Key takeaways

01 Production systems pair a feature store, models and a decision engine.

02 Avoid training–serving skew: compute features identically everywhere.

03 Monitor input drift (PSI) and output behaviour (alert rates).

04 Retrain on a schedule, respecting label delay and selection bias.

05 Document, log and fairness-test every model.

06 Assume attackers will probe and adapt to the detector itself.

PART 16

Capstone and Resources

Everything in one place: a repeatable method for any new fraud or threat-detection problem.

IN THIS PART

- An end-to-end recipe
- Algorithm and metric cheat sheets
- Ten common mistakes
- Exercises
- Glossary and further reading

The end-to-end recipe

1	Define the decision, its costs and capacity	2	Map the threat: schemes or ATT&CK techniques	3	Gather data; check quality and timestamps
4	Explore: rarity, distributions, context	5	Engineer entity baselines, velocity, context	6	Split by time; fit everything on the past
7	Start with rules and statistics	8	Add unsupervised detectors for novelty	9	Add supervised models where labels exist
10	Set thresholds by cost and budget; calibrate	11	Explain alerts; review with analysts	12	Deploy, monitor drift and alert rates, retrain

Algorithm cheat sheet

Isolation Forest	isolation	no	no	0.495	default tabular detector
k-means distance	clustering	no	yes	0.380	segmenting + detecting
Autoencoder	reconstruction	no	yes	0.316	complex, high-dim data
One-Class SVM	boundary	no	yes	0.267	small clean training sets
PCA reconstruction	reconstruction	no	yes	0.214	fast, per-feature reasons
Gaussian mixture	density	no	yes	0.211	soft segments, likelihoods
Elliptic Envelope	boundary	no	yes	0.182	one bell-shaped group
LOF	density	no	yes	0.082	local outliers, low-dim
Logistic regression	supervised	yes	yes	0.626	transparent baseline
Random forest	supervised	yes	no	0.858	robust strong model
Gradient boosting	supervised	yes	no	0.843	industry workhorse

Metric cheat sheet

Precision	alerts that were truly bad	~1.3%	judging analyst workload
Recall	bad events we alerted on	equals alert share	judging coverage
PR-AUC	precision across all recall levels	0.013	comparing models on rare events
ROC-AUC	chance a bad event outranks a good one	0.5	sanity check only
Precision / recall @ k	quality of the top-k queue	~1.3% / k÷N	fixed review capacity
Total cost	reviews × cost + missed losses	—	choosing thresholds
Brier score	accuracy of probabilities	—	calibration
PSI	distribution shift	0	monitoring drift

Ten common mistakes — every one demonstrated in this tutorial

1. Reporting accuracy on rare events
2. Random splits for time-ordered data
3. Computing statistics on all data (leakage)
4. Trusting default thresholds or contamination
5. Comparing models by ROC-AUC alone
6. Ignoring per-segment results
7. Using global baselines instead of per-entity ones
8. Dividing by a zero spread (MAD on counts)
9. Averaging weak detectors into an ensemble
10. Never monitoring after deployment

Exercises to deepen your skills

1. Beat Isolation Forest

Tune max_samples, max_features and the feature set on a validation window (days 45–59). Can you exceed PR-AUC 0.55 on the test month?

2. Catch lost cards

Engineer a feature that detects them: distance from the customer's *usual merchants*? Category mix change? Measure recall on that scheme.

3. Add jitter

Give the beacon ±40% random jitter. Does regularity still find it? Try a histogram of gaps instead.

4. Label delay

Drop the final 14 days of training labels and retrain. How much does PR-AUC fall?

5. Dictionary DGA

Generate domains by joining random English words. Retrain and compare features.

6. Cost-sensitive threshold

Change the review cost to £1 and £20. How does the best threshold move?

Glossary (1 of 2)

Anomaly score	number expressing how unusual an event is; higher = stranger (in this tutorial)
Base rate	how common the positive class is (1.42% fraud here)
Calibration	agreement between predicted probabilities and observed frequencies
Class weight	extra importance given to the rare class during training
Concept drift	change over time in the data or in what the label means
Contamination	assumed share of anomalies; sets a threshold in scikit-learn
Contextual anomaly	unusual only given its context (time, place, entity)
Feature store	system that computes and serves features consistently for training and live scoring

Glossary (2 of 2)

IOC / TTP	indicator of compromise (artefact) / tactics, techniques and procedures (behaviour)
Leakage	information unavailable at decision time sneaking into training or evaluation
MITRE ATT&CK	public knowledge base of adversary tactics and techniques
Novelty detection	learning clean normal data, then flagging new departures
PR-AUC	area under the precision–recall curve (average precision)
PSI	Population Stability Index: one-number summary of distribution shift
Reason code	short, ranked explanation attached to an alert
UEBA	user and entity behaviour analytics: per-entity baselines in security

Further reading

Liu, Ting & Zhou (2008) — Isolation Forest, IEEE ICDM	the original isolation idea
Breunig, Kriegel, Ng & Sander (2000) — LOF, ACM SIGMOD	density-based local outliers
Schölkopf et al. (2001) — Estimating the support of a high-dimensional distribution	One-Class SVM
Chandola, Banerjee & Kumar (2009) — Anomaly detection: a survey, ACM Computing Surveys	the classic map of the field
Dal Pozzolo et al. (2015) — Calibrating probability with undersampling	imbalance and probabilities in card fraud
MITRE ATT&CK knowledge base (attack.mitre.org)	techniques referenced in Part 14
scikit-learn user guide: novelty and outlier detection	practical API reference for Part 9

YOU HAVE COMPLETED THE TUTORIAL

Find the few. Explain the why. Keep learning.

16
parts

13
algorithms run on real code

12
fraud and attack behaviours planted and hunted

100%
of outputs genuine